

2 T-test

2.1 일표본 T-검정 (One Sample T-Test)

가. 일표본 T-검정 개념

- 단일모집단에서 관심이 있는 연속형 변수의 평균값을 특정 기준값과 비교
- 가정: 모집단의 구성요소들이 정규분포를 이룬다는 가정

- * 종속변수는 연속형이어야함
- * 검증하고자 하는 기준값이 있어야함

나. 일표본 T-검정 단계

1단계:	가설설정
모수	모평균(μ)
귀무가설(H_0)	$\mu = \mu_0$
대립가설(H_1)	(1) 양측검정: $\mu \neq \mu_0$ (2) 우단측검정: $\mu > \mu_0$ (3) 좌단측검정: $\mu < \mu_0$
2단계:	유의수준 설정
3단계:	검정통계량의 값 및 유의확률 계산
4단계:	귀무가설의 기각여부 판단 및 의사결정
p-value:	유의확률
α :	유의수준
p-value < α :	귀무가설을 기각하고, 대립가설을 채택
p-value > α :	귀무가설을 기각하지 않음

[예제]

cats 데이터에서 고양이들의 평균몸무게가 2.6kg인지 아닌지통계적 검정을 수행하고, 결과 해석하기 (양측검정, 유의수준 = 0.05)

In [1]:

```
import pandas as pd
cats=pd.read_csv('cats.csv')
cats
```

Out[1]:

	Sex	Bwt	Hwt
0	F	2.0	7.0
1	F	2.0	7.4
2	F	2.0	9.5
3	F	2.1	7.2
4	F	2.1	7.3
...	...	...	...
139	M	3.7	11.0
140	M	3.8	14.8
141	M	3.8	16.8
142	M	3.9	14.4

In [2]:

```
cats.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 144 entries, 0 to 143
Data columns (total 3 columns):
#   Column  Non-Null Count  Dtype
---  ------  -
0   Sex      144 non-null     object
1   Bwt      144 non-null     float64
2   Hwt      144 non-null     float64
dtypes: float64(2), object(1)
memory usage: 3.5+ KB
```

In [3]:

```
# cats의 Bwt의 정규성 검정 : Shapiro test
#귀무가설 : 정규성을 가진다
#대립가설 : 정규성 x
import scipy.stats as stats
from scipy.stats import shapiro
mu = 2.6
shapiro(cats['Bwt'])
```

Out[3]:

```
ShapiroResult(statistic=0.951878668777466, pvalue=6.730248423991725e-05)
```

[결과해석]

- 첫번째 값 (검정통계치) : 0.95187
- 두번째 값 (p-value) : 6.730e-05
==> 유의수준보다 작으므로 '데이터가 정규분포를 따른다'는 귀무가설을 기각한다.

==> 즉, cats 데이터의 Bwt 변수는 정규분포를 따르지 않음
==> **wilcox test** 로 T-검정 진행

In [4]:

```
# cats의 Bwt t-test (wilcox test)
stats.wilcoxon(cats.Bwt - mu , alternative='two-sided')
```

Out[4]:

```
WilcoxonResult(statistic=3573.0, pvalue=0.02524520294814093)
```

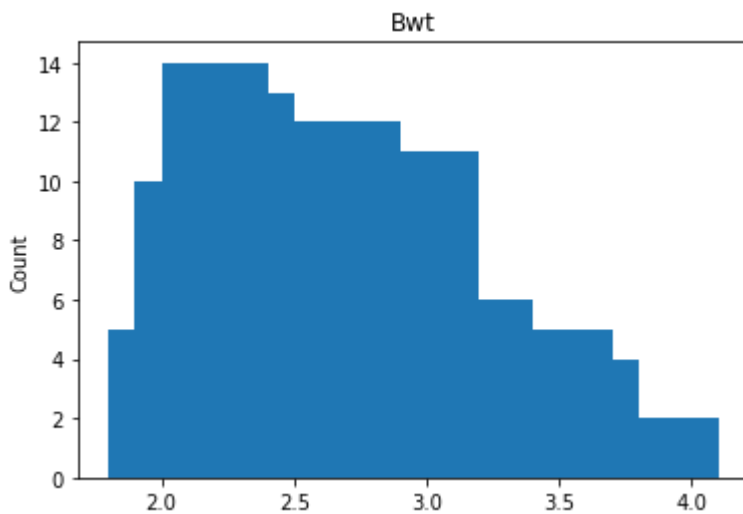
In [5]:

```
import matplotlib.pyplot as plt
cats_Bwt_cnt = pd.value_counts(cats['Bwt'].values, sort=False)

width = 0.4
plt.bar(cats_Bwt_cnt.index, cats_Bwt_cnt.values,width)
plt.title('Bwt')
plt.ylabel('Count')
```

Out[5]:

```
Text(0, 0.5, 'Count')
```



[결과해석]

- 첫번째 값 (검정통계치) : 3573.0
 - 두번째 값 (p-value) : 0.025
- ==> 유의수준보다 작으므로 '고양이들의 평균 몸무게는 2.6kg이다'는 귀무가설을 기각한다
==> 즉, 고양이들의 평균 몸무게는 2.6kg이 아니라는 결론

2.2 대응표본 T-검정 (Paired Sample T-Test)

가. 대응표본 T-검정 개념

- 단일모집단에 대해 두 번의 처리를 가했을 때, 두 개의 처리에 따른 평균의 차이를 비교

- 표본 내의 개체들에 대해 두 번의 측정 실시
- 가정: 모집단의 관측값이 정규성을 만족해야함

나. 대응표본 T-검정 단계

1단계:	가설설정
모수 두 개 모평균 사이의 차이(D)	
귀무가설(H0)	$\mu_x - \mu_y = D = 0$
대립가설(H1)	(1) 양측검정: $\mu_x - \mu_y = D \neq 0$
	(2) 우단측검정: $\mu_x - \mu_y = D > 0$
	(3) 좌단측검정: $\mu_x - \mu_y = D < 0$
2단계: 유의수준 설정	
3단계: 검정통계량의 값 및 유의확률 계산	
4단계:	귀무가설의 기각여부 판단 및 의사결정
p-value:	유의확률
α :	유의수준
p-value < α :	귀무가설을 기각하고, 대립가설을 채택
p-value > α :	귀무가설을 기각하지 않음 --> 사전평균 = 사후평균

[예제]

10명의 환자 대상 수면영양제 복용 전과 후의 수면시간 측정

영양제의 효과가 있는지를 판단하고자 함.

(표본이 정규성을 만족한다는 가정 하에 단측검정 수행, 유의수준 = 0.05)

In [6]:

```
import pandas as pd
data = {'before': [7, 3, 4, 5, 2, 1, 6, 6, 5, 4],
        'after': [8, 4, 5, 6, 2, 3, 6, 8, 6, 5]}
data = pd.DataFrame(data)
data
```

Out[6]:

	before	after
0	7	8
1	3	4
2	4	5
3	5	6
4	2	2
5	1	3
6	6	6
7	6	8
8	5	6
9	4	5

In [7]:

```
data.mean()
```

Out[7]:

```
before    4.3
after     5.3
dtype: float64
```

In [8]:

```
stats.ttest_rel(data['after'], data['before'], alternative='greater')
```

Out[8]:

```
Ttest_relResult(statistic=4.743416490252569, pvalue=0.0005269356285082765)
```

[결과해석]

- 첫번째 값 (검정통계치): 4.7434
- 두번째 값 (p-value): 0.001
==> 유의수준보다 작으므로 '수면영양제를 복용하기 전과 후의 평균 수면시간의 차이는 통계적으로 유의하며, 영양제를 복용한 후 수면시간이 늘었다'는 결론

2.3 독립표본 T-검정 (Independent Sample T-Test)

가. 독립표본 T-검정 개념

- 두 개의 독립된 모집단의 평균을 비교
- 모집단, 모수, 표본이 모두 두 개씩 존재
- 가정
 - 두 모집단은 정규성 만족해야함
 - 두 모집단의 분산이 서로 같아야 함 (등분산성 가정)
 - 독립변수는 범주형, 종속변수는 연속형

나. 독립표본 T-검정 단계

1단계:	가설설정
모수	서로 독립된 두 모집단의 평균(μ_1, μ_2)
귀무가설(H_0)	$\mu_1=\mu_2$
대립가설(H_1)	(1) 양측검정: $\mu_1\neq\mu_2$ (2) 우단측검정: $\mu_1>\mu_2$ (3) 좌단측검정: $\mu_1<\mu_2$
2단계: 유의수준 설정	
3단계: 검정통계량의 값 및 유의확률 계산	
4단계: 귀무가설의 기각여부 판단 및 의사결정	
p-value:	유의확률
α :	유의수준
p-value < α :	귀무가설을 기각하고, 대립가설을 채택
p-value > α :	귀무가설을 기각하지 않음

정규성 검정

In [9]:

```
import scipy.stats as stats
from scipy.stats import shapiro
stats.shapiro(cats.Bwt)
```

Out[9]:

ShapiroResult(statistic=0.9518786668777466, pvalue=6.730248423991725e-05)

[결과해석]

- h_0 : 표본분포가 정규성을 따른다
- p-value : 6.730248423991725e-05

검정결과: 귀무가설 기각 (표본의 모집단이 정규분포를 따르지 않음)

In [10]:

```
cats
```

Out [10]:

	Sex	Bwt	Hwt
0	F	2.0	7.0
1	F	2.0	7.4
2	F	2.0	9.5
3	F	2.1	7.2
4	F	2.1	7.3
...	...	...	...
139	M	3.7	11.0
140	M	3.8	14.8
141	M	3.8	16.8
142	M	3.9	14.4

등분산성 검정

In [11]:

```
female = cats.loc[cats.Sex == 'F', 'Bwt']  
male = cats.loc[cats.Sex == 'M', 'Bwt']  
stats.levene(female, male)
```

Out [11]:

```
LeveneResult(statistic=19.43101190877999, pvalue=2.0435285255189404e-05)
```

[결과해석]

- H0 : 등분성을 만족한다.
- H1: 등분성을 만족하지 않는다.

검정 결과: 귀무가설 기각 (등분산성을 만족하지 않음)

성별에 따른 몸무게가 등분산성을 만족하지 않는다는 조건 하에 독립 t검정 수행

In [12]:

```
stats.ttest_ind(female, male, equal_var=False)
```

Out [12]:

```
Ttest_indResult(statistic=-8.70948849909559, pvalue=8.831034455859356e-15)
```

[결과해석]

- H0 : 고양이들의 성별에 따른 평균 몸무게는 같다.
- H1: 고양이들의 성별에 따른 평균 몸무게는 다르다.

검정 결과: 귀무가설 기각 (고양이들의 성별에 따른 평균 몸무게는 통계적으로 유의한 차이가 존재한다.)

In [13]:

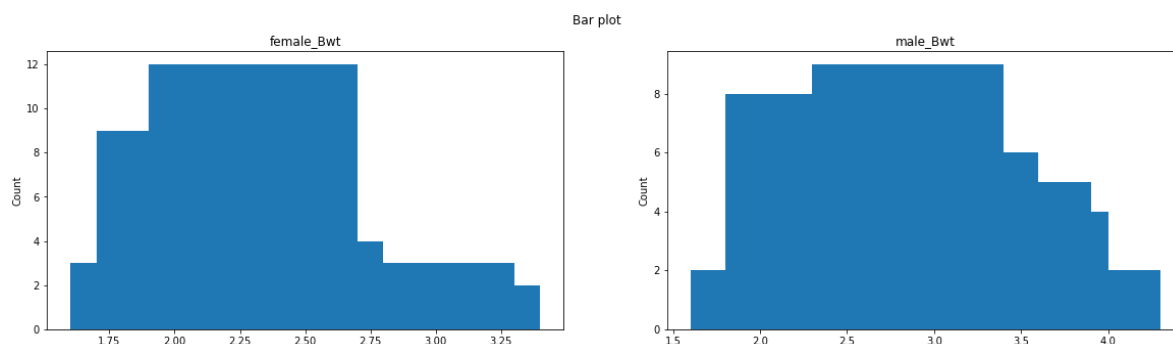
```
print(female.mean())
print(male.mean())
```

```
2.359574468085107
2.8999999999999995
```

In [14]:

```
female_Bwt_cnt = pd.value_counts(female.values, sort=False)
male_Bwt_cnt = pd.value_counts(male.values, sort=False)

fig, axs = plt.subplots(1, 2, figsize=(20,5))
fig.suptitle('Bar plot')
width = 0.4
axs[0].bar(female_Bwt_cnt.index, female_Bwt_cnt.values)
axs[0].set_title('female_Bwt')
axs[0].set_ylabel('Count')
axs[1].bar(male_Bwt_cnt.index, male_Bwt_cnt.values)
axs[1].set_title('male_Bwt')
axs[1].set_ylabel('Count')
plt.show()
```



3 분산분석 (ANOVA)

- T-검정이 두 집단 간의 평균 차이를 비교하는 통계분석 방법이라면 분산분석은 두 개 이상의 다수 집단 간 평균을 비교하는 통계분석 방법이다.
- 분산분석은 개수에 따라 일원배치, 이원배치, 다원배치 분산분석으로 나누어진다.

3.1 분산분석의 개념

- 분산분석은 두 개 이상의 집단에서 그룹 평균 간 차이를 그룹 내 변동에 비교하여 살펴보는 통계 분석 방법이다.
- 즉, 두 개 이상 집단들의 평균 간 차이에 대한 통계적 유의성을 검증하는 방법이다.
- 분산분석의 분류는 아래와 같다.

분석 구분	분석 명칭	독립변수 개수	종속변수 개수
단일변량 분산분석	일원배치 분산분석	1개	1개
	이원배치 분산분석	2개	
	다원배치 분산분석	3개 이상	
다변량 분산분석	MANOVA	1개 이상	2개 이상

3.2 일원배치 분산분석 (One-way ANOVA)

3.2.1 일원배치 분산분석의 개념

- 분산분석에서 반응값에 대한 하나의 범주형 변수의 영향을 알아보기 위해 사용되는 검증 방법이다.
- 모집단의 수에는 제한이 없으며, 각 표본의 수는 같지 않아도 된다.
- F 검정 통계량을 이용한다.

3.2.2 일원배치 분산분석의 가정

- 각 집단의 측정치는 서로 독립적이며, 정규분포를 따른다.
- 각 집단 측정치의 분산은 같다. (등분산 가정)

3.2.3 분산분석 표

요인	제곱합	자유도	평균제곱	F-value	p
집단 간	SSB	집단 수 - 1	집단간제곱합 / 자유도	집단간 평균제곱 / 집단내 평균제곱	
집단 내	SSW	자료 수 - 집단 수	집단내제곱합 / 자유도		
합계	TSS	자료 수 -1			

- SSB == SST 라고도 하며 between treatment Sum of square를 의미한다.
- SSW == SSE 라고도 하며 error or residual sum of square를 의미한다.

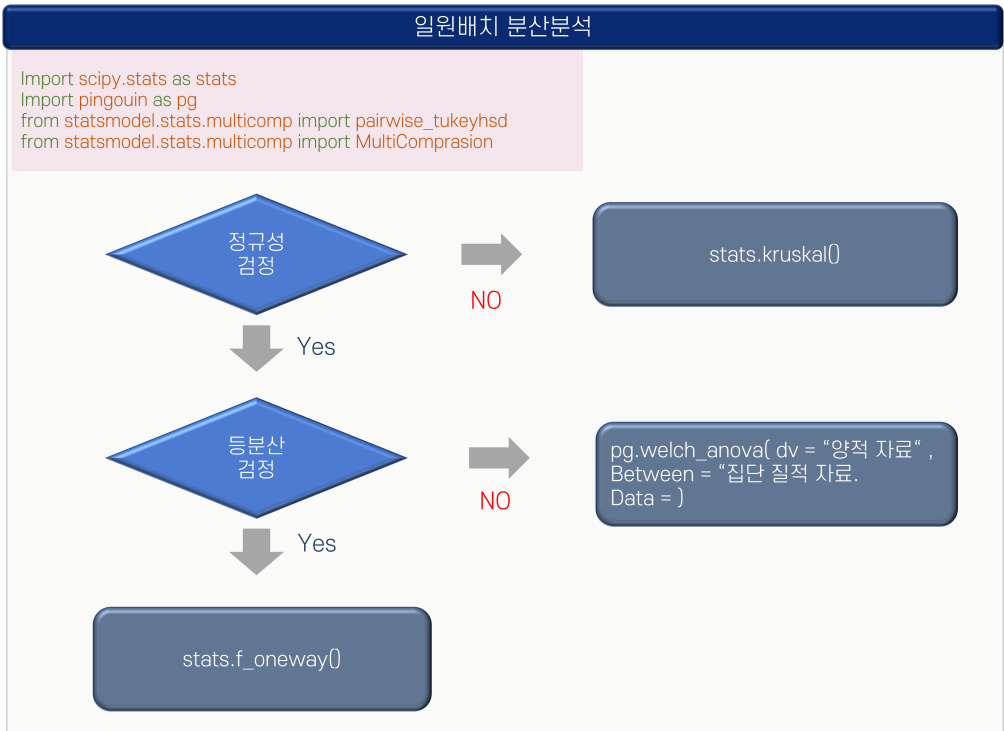
3.2.4 가설 검정

- 귀무가설(H0) : k개의 집단 간 모평균에는 차이가 없다. 즉, 같다고 할 수 있다.
- 대립가설(H1) : k개의 집단 간 모평균이 모두 같다고 할 수 없다.

3.2.5 사후 검정

- 사후검정이란 분산분석의 결과 귀무가설이 기각되어 적어도 한 집단에서 평균의 차이가 있음이 통계적으로 증명되었을 경우(<p-value), 어떤 집단들에 대해서 평균의 차이가 존재하는지를 알아보기 위해 실시하는 분석이다.
- 사후분석의 종류로는 던칸의 MRT, vltudml LSD, 튜키의 HSD방법 등이 있다.

3.2.6 Python을 활용한 일원배치 분산분석



3.3 일원 배치 분산분석 실습

- Python 에 내장되어 있는 iris 데이터를 이용하여 종(Species)별로 꽃받침의 폭(Sepal.Width)의 평균이 같은지 혹은 차이가 있는지를 확인하기 위해 일원배치 분산분석을 수행해보자
- 검정을 수행하기에 앞서 설정할 수 있는 가설은 아래와 같다.
- 귀무가설(H0): 세 가지 종에 대해 Sepal.Width의 평균은 모두 같다.
- 대립가설(H1): 적어도 하나의 종에 대한 Sepal.Width의 평균값에는 차이가 있다.

1. 분산분석

In [1]:

```
import scipy.stats as stats
import pandas as pd

Iris_data = pd.read_csv("../data/iris.csv")
Iris_data.head(100)
```

Out[1]:

	sepal length	sepal width	petal length	petal width	target
0	5.1	3.5	1.4	0.2	Iris-setosa
1	4.9	3.0	1.4	0.2	Iris-setosa
2	4.7	3.2	1.3	0.2	Iris-setosa
3	4.6	3.1	1.5	0.2	Iris-setosa
4	5.0	3.6	1.4	0.2	Iris-setosa
...	...	...	...	...	...
95	5.7	3.0	4.2	1.2	Iris-versicolor
96	5.7	2.9	4.2	1.3	Iris-versicolor
97	6.2	2.9	4.3	1.3	Iris-versicolor
98	5.1	2.5	3.0	1.1	Iris-versicolor

In [2]:

```
Iris_data["target"].unique()
```

Out[2]:

array(['Iris-setosa', 'Iris-versicolor', 'Iris-virginica'], dtype=object)

In [3]:

```
Iris_data.target.value_counts()
```

Out[3]:

```
Iris-virginica    50
Iris-setosa       50
Iris-versicolor   50
Name: target, dtype: int64
```

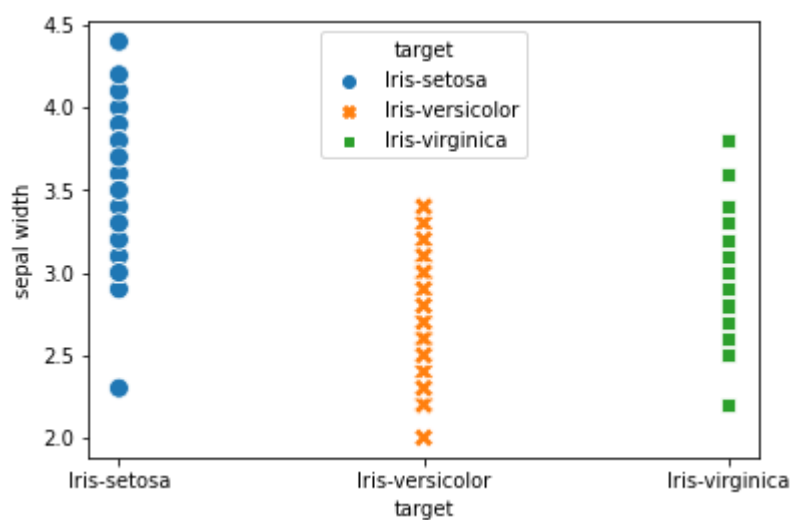
In [4]:

```
import seaborn as sns
import matplotlib.pyplot as plt

# Scatter plot by Groups

sns.scatterplot(x='target',
                y='sepal width',
                hue='target', # different colors by group
                style='target', # different shapes by group
                s=100, # marker size
                data=Iris_data)

plt.show()
```



In [5]:

```
Iris_data
```

Out[5]:

	sepal length	sepal width	petal length	petal width	target
0	5.1	3.5	1.4	0.2	Iris-setosa
1	4.9	3.0	1.4	0.2	Iris-setosa
2	4.7	3.2	1.3	0.2	Iris-setosa
3	4.6	3.1	1.5	0.2	Iris-setosa
4	5.0	3.6	1.4	0.2	Iris-setosa
...	...	...	...	...	...
145	6.7	3.0	5.2	2.3	Iris-virginica
146	6.3	2.5	5.0	1.9	Iris-virginica
147	6.5	3.0	5.2	2.0	Iris-virginica
148	6.2	3.4	5.4	2.3	Iris-virginica
149	5.9	3.0	5.1	1.8	Iris-virginica

150 rows × 5 columns

In [6]:

```
target_list=Iris_data["target"].unique()
```

In [7]:

```
target_list
```

Out[7]:

```
array(['Iris-setosa', 'Iris-versicolor', 'Iris-virginica'], dtype=object)
```

In [8]:

```
setosa = Iris_data[Iris_data["target"]==target_list[0]]["sepal width"]
versicolor = Iris_data[Iris_data["target"]==target_list[1]]["sepal width"]
virginica = Iris_data[Iris_data["target"]==target_list[2]]["sepal width"]
```

In [9]:

```
type(setosa)
```

Out[9]:

```
pandas.core.series.Series
```

In [10]:

```
## 결과에서 첫번째 값은 검정통계치이고, 두번째 값은 p-value입니다.  
## 정규성 검정에서는 p-value가 유의수준 0.05보다 클 경우 표본이 정규분포를 따른다고 판단할 수 있습니다
```

```
print(stats.shapiro(setosa))  
  
print(stats.shapiro(versicolor))  
  
print(stats.shapiro(virginica))
```

```
ShapiroResult(statistic=0.968691885471344, pvalue=0.20465604960918427)  
ShapiroResult(statistic=0.9741330742835999, pvalue=0.33798879384994507)  
ShapiroResult(statistic=0.9673910140991211, pvalue=0.1809043288230896)
```

In [11]:

```
## shapiro test 결과 0.07>0.05 이므로 정규성을 만족함으로 등분산 검정 시행  
## 귀무가설 : 집단간 분산이 같다.  
## 대립가설 : 적어도 두 집단간 분산이 다르다.
```

```
stats.levene(setosa,versicolor,virginica)
```

Out[11]:

```
LeveneResult(statistic=0.6475222363405327, pvalue=0.5248269975064537)
```

P-value가 유의수준(0.05) 보다 크기 때문에 귀무가설 채택하여 세 집단간 분산이 같다고 할 수 있다.

In [12]:

```
## LeveneResult : 등분산성 검정 완료 -> ANOVA분석 시행  
## 귀무가설(H0): 세 가지 종에 대해 Sepal.Width의 평균은 모두 같다.  
## 대립가설(H1): 적어도 하나의 종에 대한 Sepal.Width의 평균값에는 차이가 있다.
```

```
stats.f_oneway(setosa,versicolor,virginica)
```

Out[12]:

```
F_onewayResult(statistic=47.36446140299382, pvalue=1.3279165184572242e-16)
```

P-value가 유의수준(0.05) 작기 때문에 귀무가설을 기각하고 대립가설을 채택하여 적어도 세 집단 중 하나는 평균이 다르다고 할 수 있다.

2. 사후검정

- 그렇다면, 세 가지 종들 중 특히 어떠한 종들 간에 꽃받침의 폭에 차이가 있는지를 파악하기 위해 사후검정을 수행해보자

In [13]:

```
from statsmodels.stats.multicomp import pairwise_tukeyhsd  
from statsmodels.stats.multicomp import MultiComparison  
  
mc = MultiComparison(data= Iris_data["sepal width"], groups=Iris_data["target"] )
```

In [14]:

```
mc
```

Out [14]:

```
<statsmodels.sandbox.stats.multicomp.MultiComparison at 0x2efef34e9c8>
```

In [15]:

```
tuekeyhsd = mc.tuekeyhsd(alpha=0.05)
```

In [16]:

```
tuekeyhsd.summary()
```

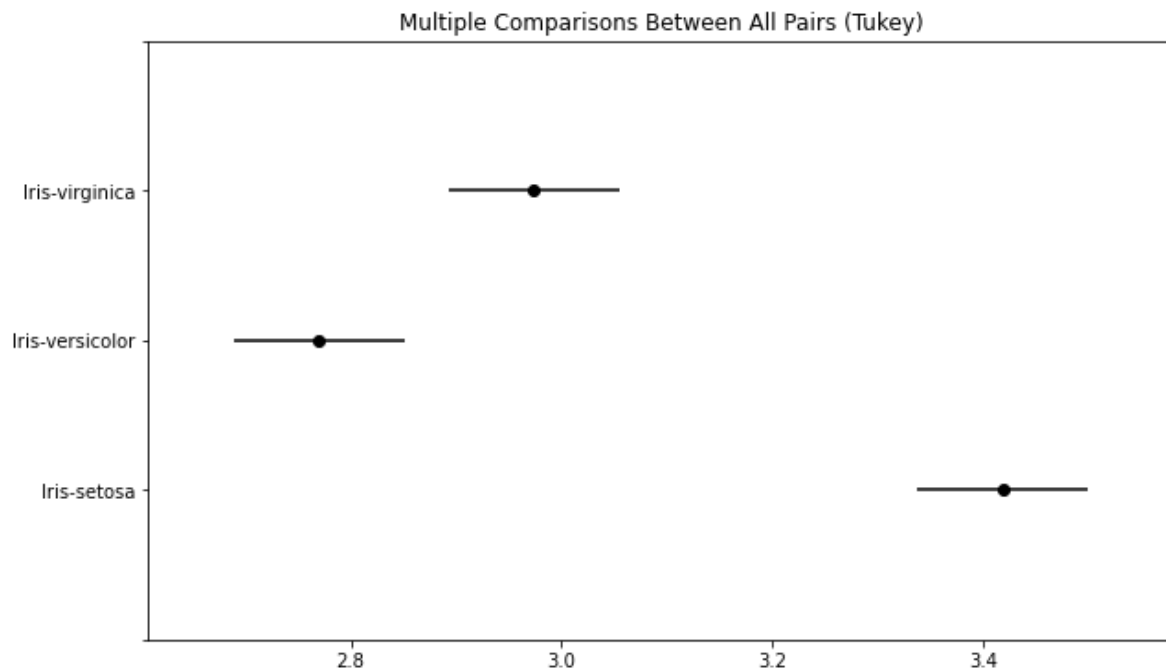
Out [16]:

Multiple Comparison of Means - Tukey HSD, FWER=0.05

group1	group2	meandiff	p-adj	lower	upper	reject
Iris-setosa	Iris-versicolor	-0.648	0.001	-0.8092	-0.4868	True
Iris-setosa	Iris-virginica	-0.444	0.001	-0.6052	-0.2828	True
Iris-versicolor	Iris-virginica	0.204	0.0089	0.0428	0.3652	True

In [17]:

```
fig = tuekeyhsd.plot_simultaneous()
```



- 사후분석에서는 귀무가설을 "집단들 사이의 평균은 같다"로 두고 대립가설을 "집단들 사이의 평균은 같지 않다"로 둔다.
- 그리고 모든 집단 수준에 대하여 두 집단씩 짝을 지어 각각 다중비교를 수행한다.
- 예제의 사후분석 결과를 살펴보면 Iris-setosa Iris-versicolor Iris-virginica 의 세 가지 비교에 대해서 모두 수정된 p-value값(p adj) 이 0.05보다 작으므로, 각각의 비교에 대한 귀무가설을 모두 기각한다. 즉, 모든 종들에 대해서 꽃받침 폭의 평균값은 통계적으로 유의한 차이가 있다는 것을 알 수 있다.

- 또한 diff는 차이(오른쪽 집단 - 왼쪽 집단)값 을 나타내는데 setosa - versicolor 에 대한 diff 값은 음수이므로, versicolor일 때보다 setosa일 때 꽃받침의 폭이 통계적으로 유의하게 큰 값을 가진다고 해석할 수 있다.

3.3.1 추가 함수 사용방법 예시

3.3.1.1 정규성을 만족하지 않을 때

In [18]:

```
stats.kruskal(setosa,versicolor,virginica)
```

Out[18]:

```
KruskalResult(statistic=62.49463010053111, pvalue=2.6882119006774528e-14)
```

3.3.1.2 등분산성을 만족하지 않을 때

In [19]:

```
# !pip install pingouin
# !conda install pingouin
import pingouin as pg
```

welch_anova(data=None, dv=None, between=None) One-way Welch ANOVA.

Parameters

data : :`pandas.DataFrame`

dv : string

종속변수를 포함하는 column 이름, (수치형)

between : string

독립변수를 포함하는 column이름 (범주형)

In [20]:

```
pg.welch_anova(data = Iris_data, dv = 'sepal width', between='target')
```

Out[20]:

	Source	ddof1	ddof2	F	p-unc	np2
0	target	2	97.369597	43.303381	3.519723e-14	0.391881

3.4 이원배치 분산분석 (Two-way ANOVA)

3.4.1 이원배치 분산분석의 개념

- 분산분석에서 반응값에 대해 두 개의 범주형 변수 A,B의 영향을 알아보기 위해 사용되는 검증 방법이다.
- 예를 들어 성별과 학년에 따른 시험점수의 차이에 대해 통계적으로 검정하기 위해 이원배치 분산분석을 사용할 수 있다.
- 두 독립변수 A,B사이의 상관관계가 있는지를 살펴보는 교호작용(두 독립변수의 범주들의 조합으로 인해 반응변수에 미치는 특별한 영향)에 대한 검증이 반드시 진행되어야 한다.

3.4.2 이원배치 분산분석의 가정

- 각 집단 측정치의 분포는 정규분포이어야 한다. (정규성)
- 집단 간 측정치의 분산은 같다. (등분산성)

3.4.3 주효과와 교호작용효과

- 이원배치 분산분석에서는 두 개의 독립변수값에 따르는 데이터의 주효과와 상호작용효과에 대한 검정을 수행한다.
- 주효과란 각각의 독립변수가 종속변수에 미치는 효과를 의미하며, 이를 검정하는 것을 주효과 검정이라 한다.
- 교호작용효과는 여러 독립변수들의 조합이 종속변수에 주는 영향을 의미한다. 즉 교호작용효과 검정은 한 독립변수가 종속변수에 미치는 영향이 다른 독립변수의 수준에 따라서 달라지는 지를 분석하는 것이다.
- 두 독립변수 A,B 사이에 상관관계가 존재할 경우, 교호작용이 있다는 의미이다.
- 교호작용이 없을 경우, 주효과 검정을 진행한다. 반면 교호작용이 있을 경우에는 검정이 무의미하다.

3.4.4 분산분석표

요인	제곱합	자유도	평균제곱합	F	F값
요인 A	SS_A	$I-1$	$MS_A = \frac{SS_A}{I-1}$	$F_A = \frac{MS_A}{MSE}$	$F(I-1, IJ(n-1)) =$
요인 B	SS_B	$J-1$	$MS_B = \frac{SS_B}{J-1}$	$F_B = \frac{MS_B}{MSE}$	$F(J-1, IJ(n-1)) =$
상호 작용	$SS_{A \times B}$	$(I-1)(J-1)$	$MS_{AB} = \frac{SS_{A \times B}}{(I-1)(J-1)}$	$F_{AB} = \frac{MS_{AB}}{MSE}$	$F((I-1)(J-1), IJ(n-1)) =$
오차	SSE	$IJ(n-1)$	$MSE = \frac{SSE}{IJ(n-1)}$		
전체	SST	$IJn-1$			

- SST (Sum of Squares Total) : 총 제곱합
- SSA : 요인A 수준 평균값들 사이의 제곱합
- SSB : 요인 B 수준 평균값들 사이의 제곱합
- SSE(Sum of Squares Error): 오차 제곱합 or SSW== (Sum of square within factor) : 집단 내 변화량
- I : 요인A의 수준의 수
- J : 요인B의 수준의 수

3.4.5 가설 검정

- 귀무가설(H_0) : 3개
 - 1) a와 b변수의 상호작용 효과 (교호작용, interaction)가 없다.
 - 2) a변수에 따른 종속변수의 값에는 차이가 없다.
 - 3) b변수에 따른 종속변수의 값에는 차이가 없다.

- 대립가설(H_1) : 3개
 - 1) a와 b변수의 상호작용 효과 (교호작용, interaction)가 있다. .
 - 2) a변수에 따른 종속변수의 값에는 차이가 있다.
 - 3) b변수에 따른 종속변수의 값에는 차이가 있다.

3.5 Python을 활용한 이원배치 분산분석

함수 사용법 Python으로 양방향 ANOVA를 수행하는 훨씬 더 쉬운 방법이 있습니다. 많은 R- 패키지 (예 : lm)와 유사한 모델 표기법을 가진 Statsmodel을 사용할 수 있습니다. 모델의 공식화부터 시작합니다.

- `formula = 'len ~ C(supp) + C(dose) + C(supp):C(dose)'`
- `model = ols(formula, data).fit()`
- `aov_table = anova_lm(model, typ=2)`

함수 사용 예제

Python에 내장된 mtcars 데이터는 32개의 차종에 대한 다양한 특성과 단위 연료당 주행거리를 담고 있다.
am변수는 변속기 종류이며, cyl변수는 실린더 개수를 의미한다. 데이터를 분석에 적절한 형태로 처리한 후,
변속기 종류(am변수)와 실린더 개수(cyl)에 따라 주행거리(mpg변수) 평균에 유의미한 차이가 존재하는지
이원 분산분석을 수행하고, 그 결과를 해석해보자

- 검정을 수행하기에 앞서 설정할 수 있는 가설은 아래와 같다.
 - 가. 상호작용효과 검정에 대한 가설
 - 1) 귀무가설 H0: 변속기 종류와 실린더 개수 간에는 상호작용 효과가 없다.
 - 2) 대립가설 H1: 변속기 종류와 실린더 개수 간에는 상호작용 효과가 있다.
 - 나. 주효과 검정에 대한 가설
 - 1) 귀무가설 H0: 실린더 개수에 따른 주행거리의 차이는 존재하지 않는다.
 - 2) 대립가설 H1: 실린더 개수에 따른 주행거리의 차이는 존재한다.
 - 3) 귀무가설 H0: 변속기 종류에 따른 주행거리의 차이는 존재하지 않는다.
 - 4) 대립가설 H1: 변속기 종류에 따른 주행거리의 차이는 존재한다.
-

In [1]:

```
import pandas as pd
import numpy as np
# 데이터 불러오기
mtcars = pd.read_csv('mtcars.csv')
mtcars.head()
```

Out[1]:

	Unnamed: 0	mpg	cyl	dis	hp	drat	wt	qsec	vs	am	gear	carb
0	Mazda RX4	21.0	6	160.0	110	3.90	2.620	16.46	0	1	4	4
1	Mazda RX4 Wag	21.0	6	160.0	110	3.90	2.875	17.02	0	1	4	4
2	Datsun 710	22.8	4	108.0	93	3.85	2.320	18.61	1	1	4	1
3	Hornet 4 Drive	21.4	6	258.0	110	3.08	3.215	19.44	1	0	3	1
4	Hornet Sportabout	18.7	8	360.0	175	3.15	3.440	17.02	0	0	3	2

In [2]:

```
## cyl, am, mpg 변수들로만 구성된 분석용 데이터셋 생성
mtcars = mtcars[["mpg", "am", "cyl"]]
mtcars.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 32 entries, 0 to 31
Data columns (total 3 columns):
#   Column  Non-Null Count  Dtype
---  ------  -
0   mpg     32 non-null        float64
1   am      32 non-null        int64
2   cyl     32 non-null        int64
dtypes: float64(1), int64(2)
memory usage: 896.0 bytes
```

In [3]:

```
mtcars.cyl.value_counts()
```

Out[3]:

```
8    14
4    11
6     7
Name: cyl, dtype: int64
```

In [4]:

```
mtcars.am.value_counts()
```

Out[4]:

```
0    19
1    13
Name: am, dtype: int64
```


In [5]:

```
## 독립변수 cyl, am와 종속변수 mpg를 Series로 변경
cyl = mtcars["cyl"]
am = mtcars["am"]
mpg = mtcars["mpg"]
print(type(cyl), type(am))
```

<class 'pandas.core.series.Series'> <class 'pandas.core.series.Series'>

In [6]:

mtcars

Out[6]:

	mpg	am	cyl
0	21.0	1	6
1	21.0	1	6
2	22.8	1	4
3	21.4	0	6
4	18.7	0	8
5	18.1	0	6
6	14.3	0	8
7	24.4	0	4
8	22.8	0	4
9	19.2	0	6
10	17.8	0	6

In [29]:

```
data1 = mtcars[(mtcars['am']==0) & (mtcars['cyl']==4)][ 'mpg' ]
data2 = mtcars[(mtcars['am']==0) & (mtcars['cyl']==6)][ 'mpg' ]
data3 = mtcars[(mtcars['am']==0) & (mtcars['cyl']==8)][ 'mpg' ]
data4 = mtcars[(mtcars['am']==1) & (mtcars['cyl']==4)][ 'mpg' ]
data5 = mtcars[(mtcars['am']==1) & (mtcars['cyl']==6)][ 'mpg' ]
data6 = mtcars[(mtcars['am']==1) & (mtcars['cyl']==8)][ 'mpg' ]
```

In [30]:

```
import scipy.stats as stats
stats.levene(data1,data2,data3,data4,data5,data6)
```

Out[30]:

LeveneResult(statistic=2.735991909675646, pvalue=0.04086062802220418)

2. 분산분석 수행

In [9]:

```
## 분산분석 수행
```

```
from statsmodels.formula.api import ols
from statsmodels.stats.anova import anova_lm

formula = 'mpg ~C(cyl)*C(am)' # 'mpg ~ C(cyl) + C(am) + C(cyl):C(am)' 왼쪽의 공식과 동일
model = ols(formula, mtcars).fit()
aov_table = anova_lm(model, typ=2)
```

In [10]:

```
aov_table
```

Out[10]:

	sum_sq	df	F	PR(>F)
C(cyl)	456.400921	2.0	24.819011	9.354735e-07
C(am)	36.766919	1.0	3.998759	5.608373e-02
C(cyl):C(am)	25.436511	2.0	1.383233	2.686140e-01
Residual	239.059167	26.0	NaN	NaN

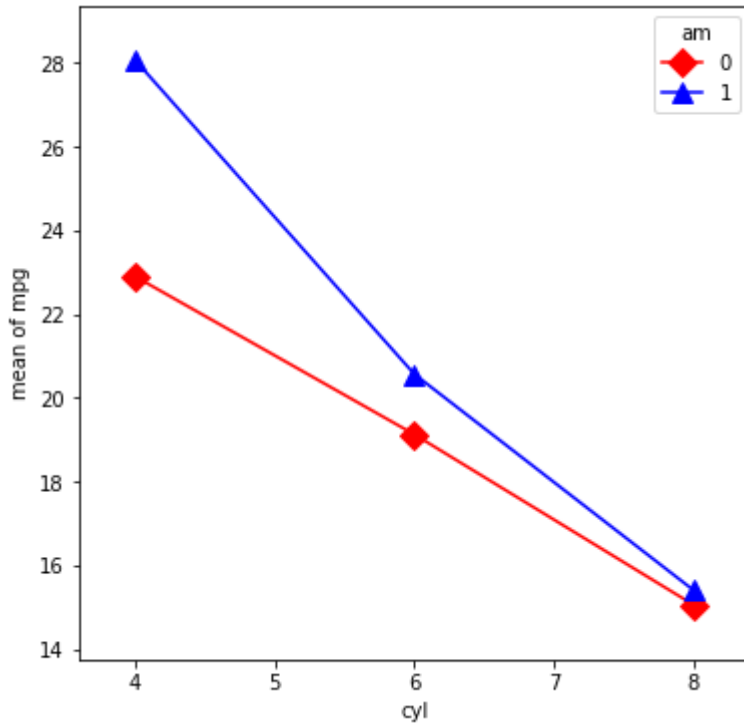
- cyl변수에 대한 p-value는 0.05보다 작으므로, 주효과 검정에서 1)귀무가설을 기각한다. 따라서 실린더 개수에 따라 주행거리 간 유의미한 차이는 존재한다고 해석할 수 있다.
- am변수에 대한 p-value는 0.0561로 0.05보다 크므로, 주효과 검정에서 '변속기 종류에 따른 주행거리 평균 간 차이는 존재하지 않는다'는 귀무가설을 기각하지 않는다.
- cyl변수와 am변수 간의 상호작용 효과에 대한 검정결과, p-value는 0.2686으로 0.05보다 크므로 귀무가설을 기각하지 않는다. 따라서 실린더 개수와 변속기 종류 간에는 교호작용이 존재하지 않는다는 것을 알 수 있다.

Q2. 실린더 개수와 변속기 종류 사이에 상호작용 효과가 있는지 없는지를 시각화 해주는 상호작용 그래프를 그린 후 이를 해석해보자.

In [11]:

```
from statsmodels.graphics.factorplots import interaction_plot
import matplotlib.pyplot as plt

fig, ax = plt.subplots(figsize=(6, 6))
fig = interaction_plot(cyl, am, mpg,
                      colors=['red', 'blue'], markers=['D', '^'], ms=10, ax=ax)
```



- 최우선으로 aov_table값으로 상호작용(교호작용)효과의 유의성을 판단한다.
- 일반적으로 상호작용 그래프에서 두 선이 서로 교차하고 있을 시에는 x축에 있는 독립변수와 그래프에서 시각화된 독립변수 간에는 상호작용이 존재한다고 해석할 수 있다.

4.1 교차분석

4.1.1 카이제곱 검정이란?

- 카이제곱(χ^2) 분포에 기초한 통계 방법
- 설명변수가 범주형 자료인 경우 사용
- 각 범주에 따른 결과변수의 분포를 설명하거나, 범주형 변수가 두 개 이상인 경우 두 변수가 상관이 있는지를 검정함
- 기본가정 : 각 범주의 기대빈도가 5 이상이어야 함

4.1.2 교차분석 개념

- 범주형 자료(명목/서열 수준)인 두 변수 간의 관계를 알아보기 위해 실시하는 분석 기법
- 적합성 검정, 독립성 검정, 동질성 검정에 사용되며, 카이제곱 검정통계량을 이용함
- 교차분석은 교차표에서 각 셀의 관찰빈도와 기대빈도 간의 차이를 검정함
 - 관찰빈도 : 자료로부터 얻은 빈도분포
 - 기대빈도 : 두 변수가 독립일 때 이론적으로 기대할 수 있는 빈도분포

4.1.3 카이제곱 검정의 세 가지 검정

- 적합성 검정 : 각 범주에 따른 데이터의 빈도분포가 이론적으로 기대하는 분포를 따르는지를 검정함
ex. 주사위를 굴렀을 때, 각 주사위의 값이 1/6의 확률로 등장하는 게 맞는지 검정
- 독립성 검정 : 두 개 이상의 범주형 변수에 대해서, 그 변수들의 관계에 따라 값들의 분포가 유의미한 차이를 보이는지 검정함
ex. 고객의 지역과 보험 가입 여부가 주어졌을 때 지역에 따른 보험 가입 비율의 차이가 존재하는지 검정
- 동질성 검정 : 부모집단의 관측값들이 정해진 범주 내에서 서로 비슷하게 나타나고 있는지를 검정
ex. 남학생의 TV 프로그램 선호도와 여학생의 TV프로그램 선호도가 동일한지 검정

4.1.3.1 동질성 검정과 독립성 검정의 차이

- 동질성 검정은 범주 A에 대해 범주B의 유형들이 같은 빈도로 나타나는지를 검정하는 것
- 독립성 검정은 독립적인지 (혹은 상관성이 있는지)를 검정하는 것

4.1.3.2 적합성 검정

- 실험에서 얻어진 관측값들이 예상한 이론과 일치하는지 아닌지를 검정하는 방법
- 관측값들이 어떠한 이론적 분포를 따르고 있는지 관찰할 수 있음
- 모집단 분포에 대한 가정이 옳게 됐는지 관측 자료와 비교하여 검정하는 것
- 검정통계량
 - 카이제곱 통계량 값이 큰 경우 : 관찰도수와 기대도수의 차이가 크고 적합도가 낮음 (일치한다고 볼 수 없음)

- 카이제곱 통계량 값이 작은 경우 : 관찰도수와 기대도수의 차이가 작고 적합도가 높음 (일치한다고 볼 수 있음)

```
scipy.stats.chisquare(f_obs, f_exp=None)
```

- `f_obs` : 각 범주에서 관찰된 빈도
- `f_exp` : 각 카테고리의 예상 빈도

[예제]

titanic데이터에서 sex 변수에 대한 분할표를 생성하고 아래의 가설에 대한 적합도 검정을 수행하세요.

- 귀무가설 : 전체 응답자 중 남자의 비율이 50%, 여자의 비율이 50%이다
- 대립가설 : 전체 응답자 중 남자의 비율이 50%, 여자의 비율이 50%이라고 할 수 없다

In [1]:

```
import pandas as pd
# 데이터 불러오기
df_t = pd.read_csv("../data/titanic.csv")
# titanic 데이터의 구조 확인
df_t.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 11 columns):
#   Column      Non-Null Count  Dtype
---  ---
0   survived    891 non-null    int64
1   pclass      891 non-null    int64
2   sex         891 non-null    object
3   age         714 non-null    float64
4   sibsp       891 non-null    int64
5   parch       891 non-null    int64
6   fare        891 non-null    float64
7   embarked    889 non-null    object
8   class       891 non-null    object
9   adult_male  891 non-null    bool
10  embark_town  889 non-null    object
dtypes: bool(1), float64(2), int64(4), object(4)
memory usage: 70.6+ KB
```

In [2]:

```
df_t
```

Out[2]:

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	adult_male	em
0	0	3	male	22.0	1	0	7.2500	S	Third	True	Sc
1	1	1	female	38.0	1	0	71.2833	C	First	False	
2	1	3	female	26.0	0	0	7.9250	S	Third	False	Sc
3	1	1	female	35.0	1	0	53.1000	S	First	False	Sc
4	0	3	male	35.0	0	0	8.0500	S	Third	True	Sc
...	...	...	...	...	...	...	...	...	...	...	
886	0	2	male	27.0	0	0	13.0000	S	Second	True	Sc
887	1	1	female	19.0	0	0	30.0000	S	First	False	Sc
888	0	3	female	NaN	1	2	23.4500	S	Third	False	Sc
889	1	1	male	26.0	0	0	30.0000	C	First	True	
890	0	3	male	32.0	0	0	7.7500	Q	Third	True	Q

891 rows × 11 columns



In [3]:

```
table= df_t[['sex']].value_counts()  
table
```

Out[3]:

```
sex  
male      577  
female    314  
dtype: int64
```

In [4]:

```
len(df_t['sex'])/2
```

Out[4]:

445.5

In [32]:

```
# 적합도 검정 수행
from scipy.stats import chisquare
chi = chisquare(table, f_exp=[445.5,445.5])
print('<적합도 검정>\n',chi)
print('p-value가 0.0으로 유의수준 0.05보다 작으므로 귀무가설을 기각하고 대립가설을 지지함')
```

<적합도 검정>

Power_divergenceResult(statistic=77.63075196408529, pvalue=1.2422095313910336e-18)
p-value가 0.0으로 유의수준 0.05보다 작으므로 귀무가설을 기각하고 대립가설을 지지함

4.1.3.3 독립성 검정

- 모집단이 두 개의 변수 A, B에 의해 범주화되었을 때, 이 두 변수들 사이의 관계가 독립인지 아닌지 검정하는 것
- 교차표를 활용함

```
scipy.stats.chi2_contingency(observed)
observed : 교차표 (각 범주에서의 발생 횟수 표)
```

[예제]

titanic 데이터에서 좌석등급(class)과 생존 여부(survived)가 서로 독립인지 확인하기 위해 분할표를 생성하고, 아래 가설에 대한 독립성 검정을 수행하라

- 귀무가설 : class와 survived는 독립이다
- 대립가설 : class와 survived는 독립이 아니다

In [5]:

```
# 변수의 분할표 생성

table = pd.crosstab(df_t['class'], df_t['survived'])
table
```

Out[5]:

survived	0	1
class		
First	80	136
Second	97	87
Third	372	119

In [7]:

```
# 카이제곱 검정을 통한 독립성 검정 수행
from scipy.stats import chi2_contingency

chi, p, df, expect = chi2_contingency(table)

print('Statistic:', chi)
print('p-value:', p)
print('df:', df)
print('expect: \n', expect)

print("\n p-value가 유의수준 0.05보다 작으므로 귀무가설을 기각")
print("\n 좌석 등급과 생존은 독립이 아니라고 할 수 있다. ")
```

```
Statistic: 102.88898875696056
p-value: 4.549251711298793e-23
df: 2
expect:
[[133.09090909  82.90909091]
 [113.37373737  70.62626263]
 [302.53535354 188.46464646]]
```

p-value가 유의수준 0.05보다 작으므로 귀무가설을 기각

좌석 등급과 생존은 독립이 아니라고 할 수 있다.

4.1.3.4 동질성 검정

- 모집단이 임의의 변수에 따라 R개의 속성으로 범주화되었을 때, R개의 부분 모집단에서 추출한 표본이 C개의 범주화된 집단의 분포가 서로 동일한지 검정
- 교차표를 활용하며, 계산법과 검증법은 모두 독립성 검정과 같은 방법으로 진행됨

- 귀무가설 : class의 분포는 survived에 관계없이 동일하다
- 대립가설 : class의 분포는 survived에 관계없이 동일하지 않다.

In [8]:

```
chi, p, df, expect = chi2_contingency(table)
result = "chi2 : {}, p-value : {}, df : {}".format(chi,p,df)
print(result)
#자유도(df, degree of freedom)는 (3-1)*(2-1)로 2 입니다
```

```
chi2 : 102.88898875696056, p-value : 4.549251711298793e-23, df : 2
```

In [11]:

```
print(expect)
```

```
[[133.09090909  82.90909091]
 [113.37373737  70.62626263]
 [302.53535354 188.46464646]]
```


In [12]:

```
table
```

Out [12]:

survived	0	1
class		
First	80	136
Second	97	87
Third	372	119

[검정결과]

- 동질성 검정 결과 p-value가 유의수준 0.05보다 작으므로, 귀무가설을 기각한다.
- 즉, 좌석등급의 분포는 생존과 동일하지 않다고 할 수 있다.

5 회귀분석

5.1 변수 유형별 분석 방법

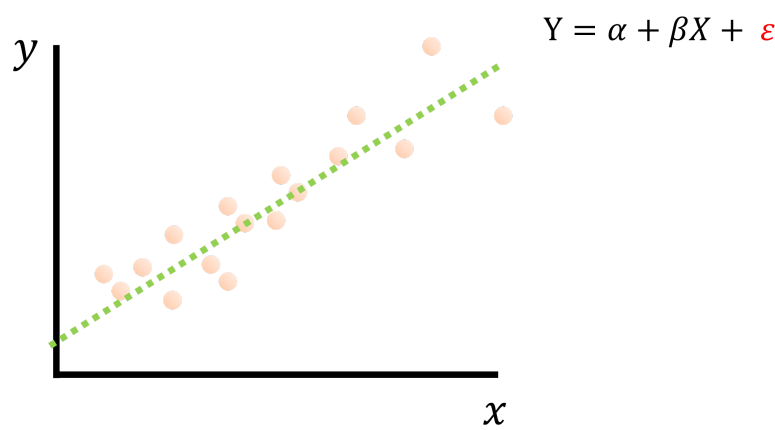
구분		독립변수 X (설명변수)	
		범주형	연속형
종속변수 y (반응변수)	범주형	카이제곱 검정	로지스틱회귀 분류 분석
	연속형	T검정, ANOVA검정	회귀분석

5.1.1 회귀분석 개념

- 하나 혹은 그 이상의 원인이 종속변수에 미치는 영향을 추적하여 식으로 표현하는 통계기법
- 변수들 사이의 인과관계를 밝히고 모형을 적합하여 관심있는 변수를 예측하거나 추론하기 위해 사용하는 분석 방법
- 독립변수의 개수가 하나인 경우 **단순선형회귀분석**, 독립변수의 개수가 두 개 이상이면 **다중선형회귀분석**으로 분석함

5.2 단순선형회귀분석

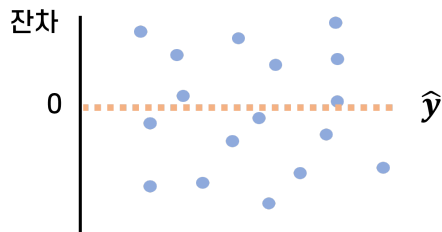
하나의 독립변수가 종속변수에 미치는 영향을 추정할 수 있는 통계기법



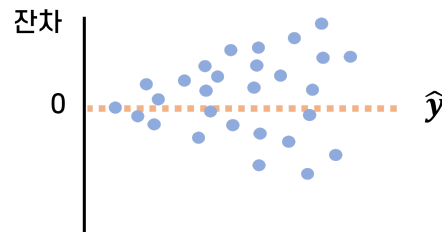
5.2.1 선형회귀분석의 가정

- 독립변수와 종속변수 간의 선형성 (선형회귀분석에서 가장 중요한 가정)
- 오차의 정규성 : 오차의 분포가 정규분포를 만족해야 함 **Q-Q plot**, **Shapiro-Wilk**검정 등을 활용하여 정규성 확인
- 오차의 등분산성 : 오차의 분산은 독립변수 값과 무관하게 일정해야 함

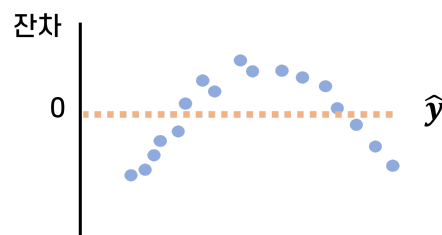
- 오차의 독립성 : 예측값의 변화에 따라 오차항이 특정한 패턴을 가지면 안 됨
Durbin-Watson 통계량이 2에 가까우면 독립성을 만족
Durbin-Watson 통계량이 0에 가까우면 오차항 간에 양의 상관관계가 존재
Durbin-Watson 통계량이 4에 가까우면 오차항 간에 음의 상관관계가 존재



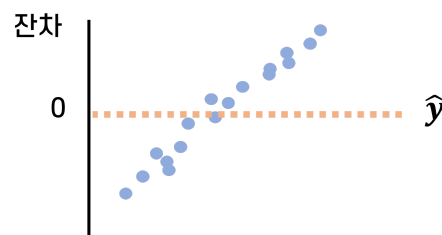
1) 랜덤하게 분포되어 있는 경우
- 등분산성을 만족



2) 등분산성이 벗어난 경우
- 분산이 증가하므로, 가중회귀 직선이 적합할 것



1) 선형성이 벗어나는 경우
- 이차항을 추가한 다항회귀가 적합



4) 잔차가 선형을 이루는 경우
- 새로운 독립변수를 추가하는 것이 적합

5.2.2 회귀분석 시 검토사항

F 통계량 : 모델 또는 모델 성분의 유의성을 검정하는 분산분석(ANOVA) 방식에 대한 검정 통계량

- 1) 모형이 데이터를 잘 적합하고 있는가 : 모형의 잔차를 그리고 **회귀진단**을 수행해 판단함
- 2) 회귀모형이 통계적으로 유의한가 :
 - 회귀분석의 결과로 산출되는 **F-통계량**의 p-value가 유의수준(α)보다 작으면 회귀식이 통계적으로 유의하다고 볼 수 있음
 - H_0 : 회귀모형은 유의하지 않다 ($\beta = 0$)
 - H_1 : 회귀모형은 유의하다 (β 는 0이 아니다)
- 3) 모형은 데이터를 얼마나 설명할 수 있는가 : **결정계수**(R-square)를 확인함
 - 결정계수는 0~1의 값을 가짐
 - 추정된 회귀식이 전체 데이터에서 설명할 수 있는 데이터의 비율을 의미함 (높을수록 높은 설명력)
 - 다변량 회귀분석의 경우 포함된 독립변수의 유의성에 관계없이 독립변수의 수가 많아지면 결정계수가 높아지는 경향을 보임
이를 보완하기 위해 수정된 결정계수(adjusted R square)를 활용하여 모형의 설명력을 판단함
- 4) 모형 내의 회귀계수가 유의한가 : 회귀계수에 대한 **t통계량**의 p-value가 0.05보다 작으면 회귀계수가 통계적으로 유의하다고 볼 수 있음

5.2.3 statsmodels를 이용한 방법

```
ols('y ~ x', data=df).fit()
```

- 모델.summary() : 적합 결과를 요약하여 제시함
- 모델.params : 변수의 회귀계수 (또는 y절편값)
- 모델.resid : 잔차
- 모델.predict(X) : 새로운 데이터에 대한 예측값

[예제]

kc_house_data 데이터의 sqft_living(주거공간의 평방 피트)를 독립변수, price를 종속변수로 설정하여 단순 선형 회귀분석을 실시한 후, 추정된 회귀모형에 대해 해석하라

In [1]:

```
import pandas as pd
import numpy as np
house = pd.read_csv('../data/kc_house_data.csv')
```

In [2]:

```
house = house[["price", "sqft_living"]]
```

In [3]:

```
## 독립변수와 종속변수의 선형 가정
house.corr()
```

Out[3]:

	price	sqft_living
price	1.000000	0.702035
sqft_living	0.702035	1.000000

In [4]:

```
y = house['price']
X = house[['sqft_living']]
```

In [5]:

```
#pip install statsmodels
# ols : ordinary least squares 사용
from statsmodels.formula.api import ols

# 단순선형회귀모형 적합
lr = ols('price ~ sqft_living', data=house).fit()
```

In [6]:

```
# 회귀 계수  
lr.params
```

Out[6]:

```
Intercept      -43580.743094  
sqft_living     280.623568  
dtype: float64
```

In [7]:

```
y_pred = lr.predict(X)  
y_pred
```

Out[7]:

```
0      287555.067025  
1      677621.826402  
2      172499.404187  
3      506441.449985  
4      427866.850973  
...  
21608   385773.315789  
21609   604659.698749  
21610   242655.296161  
21611   405416.965541  
21612   242655.296161  
Length: 21613, dtype: float64
```

In [8]:

```
resid = y - y_pred
```

In [9]:

```
#mse  
(resid**2).sum()/len(y)
```

Out[9]:

```
68351286833.039825
```

In [10]:

```
## rmse  
np.sqrt((resid**2).sum()/len(y))
```

Out[10]:

```
261440.79030067177
```

In [11]:

```
lr.summary()
```

Out[11]:

OLS Regression Results

Dep. Variable:	price	R-squared:	0.493
Model:	OLS	Adj. R-squared:	0.493
Method:	Least Squares	F-statistic:	2.100e+04
Date:	Sun, 06 Feb 2022	Prob (F-statistic):	0.00
Time:	20:17:36	Log-Likelihood:	-3.0027e+05
No. Observations:	21613	AIC:	6.005e+05
Df Residuals:	21611	BIC:	6.006e+05
Df Model:	1		
Covariance Type:	nonrobust		

	coef	std err	t	P> t	[0.025	0.975]
Intercept	-4.358e+04	4402.690	-9.899	0.000	-5.22e+04	-3.5e+04
sqft_living	280.6236	1.936	144.920	0.000	276.828	284.419

Omnibus:	14832.490	Durbin-Watson:	1.983
Prob(Omnibus):	0.000	Jarque-Bera (JB):	546444.713
Skew:	2.824	Prob(JB):	0.00
Kurtosis:	26.977	Cond. No.	5.63e+03

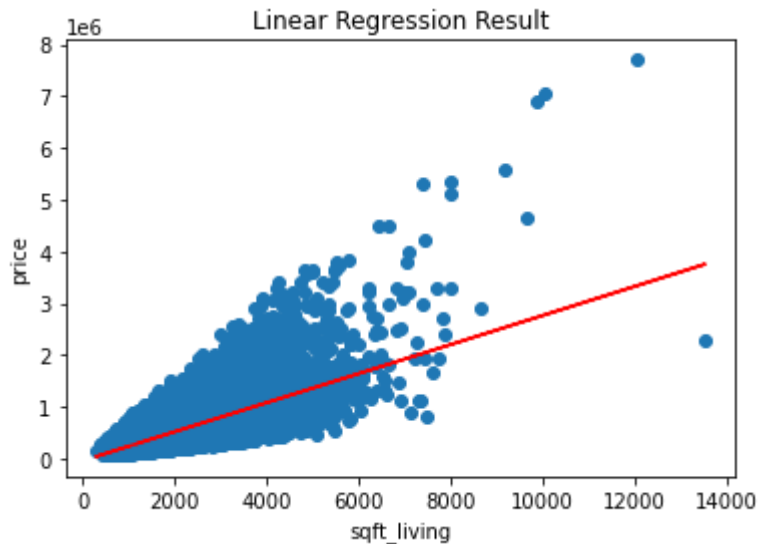
Warnings:

- [1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
- [2] The condition number is large, 5.63e+03. This might indicate that there are strong multicollinearity or other numerical problems.

In [12]:

```
# 회귀모형의 시각화
import matplotlib.pyplot as plt
plt.scatter(X, y) ## 원 데이터 산포도
plt.plot(X, y_pred, color='red') ## 회귀직선 추가

plt.xlabel('sqft_living', fontsize=10)
plt.ylabel('price', fontsize=10)
plt.title('Linear Regression Result')
plt.show()
```

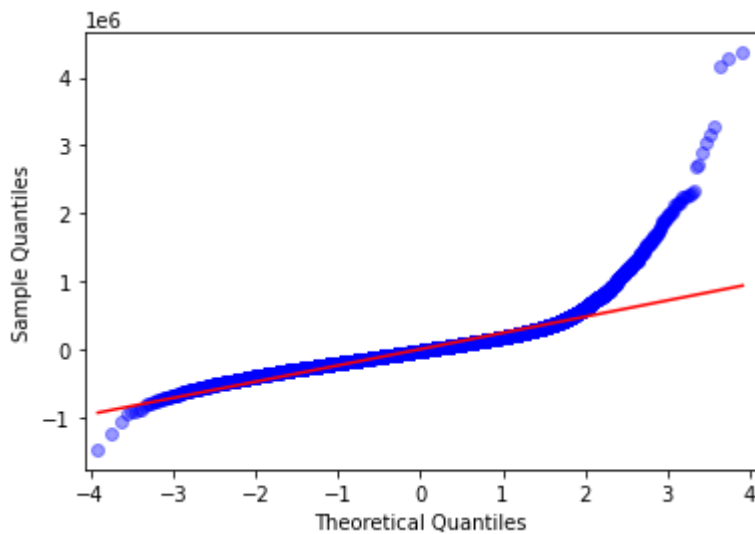


In [13]:

```
# Q-Q plot
# 잔차가 정규분포를 따르는지 확인

import statsmodels.api as sm

sm.qqplot(resid, line="r", alpha=0.4)
# line은 참조선을 그리는 것으로 '45'(45도 선), 's'(표준화된 라인), 'r'(회귀선), 'q'(사분위수를 통과
하는 선) 옵션이 있음
plt.show()
```



모델 설명

- 1) 독립변수는 `sqft_living`이고 종속변수는 `price`이다.
- 2) F-통계량은 $2.100e+04$ 이며, F-통계량의 p-value는 0.05보다 매우 작기 때문에 추정된 회귀 모형은 통계적으로 유의하다고 볼 수 있다
- 3) 결정계수는 0.493로, 이 회귀모형의 데이터에 대한 설명력은 49.3% 이다
- 4) 상수항과 독립변수의 회귀계수에 대한 p-value가 각각 0.05보다 작으므로 통계적으로 유의하다고 판단할 수 있다
- 5) 회귀식은 $\text{Price} = \text{sqft_living} * 280.6236 + -43580.743094$ 와 같이 도출할 수 있다

5.2.4 scikit-learn 머신러닝을 이용한 방법

```
sklearn.linear_model.LinearRegression().fit()
```

`LinearRegression()` : 선형회귀분석 모델을 생성하는 함수

`fit()` : 선형 회귀 모델에 필요한 두 가지 변수(x, y)를 전달

- 설명변수의 차원을 조정해야 함
- R의 `summary()`와 같은 기능이 없음
- 기울기와 절편을 따로 호출해서 얻을 수 있음
- 잔차를 자동으로 계산해주는 기능이 없음

In [14]:

```
# fit 함수가 x 값으로 2차원 array를 받으므로 dataframe으로 저장
X = house[['sqft_living']]
y = house['price']
```

In [15]:

```
#pip install sklearn
from sklearn.linear_model import LinearRegression
lr = LinearRegression() # 회귀모델 생성
lr.fit(X, y) # 회귀모델 학습
```

Out[15]:

```
LinearRegression()
```

In [16]:

```
# 기울기와 절편은 따로 호출하여 확인할 수 있다
print('기울기 :', lr.coef_)
print('절편 :', lr.intercept_)
```

```
기울기 : [280.6235679]
```

```
절편 : -43580.74309447361
```


In [17]:

```
# 추정된 값을 보기 위해서는 모델.predict(x)를 출력한다

y_pred= lr.predict(X)
y_pred
```

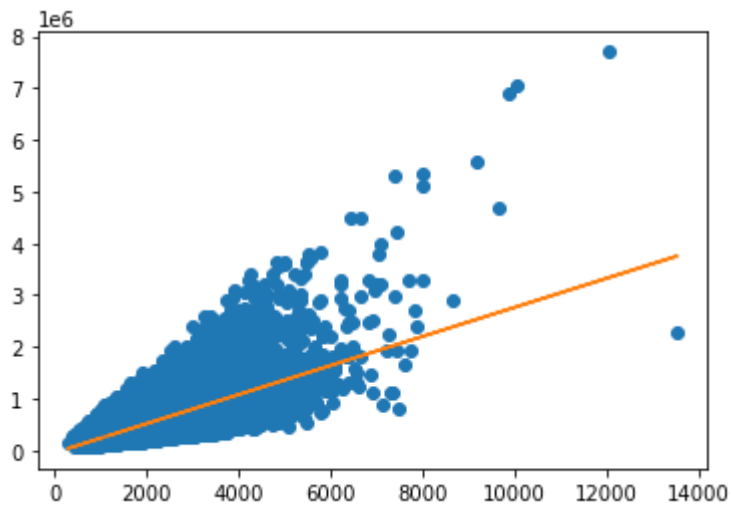
Out[17]:

```
array([287555.06702452, 677621.82640197, 172499.40418656, ...,
       242655.29616092, 405416.96554144, 242655.29616092])
```

In [18]:

```
# plot로 확인하는 방법
```

```
plt.plot(X, y, 'o')
plt.plot(X, y_pred)
plt.show()
```



In [19]:

```
# 잔차를 확인하는 방법 : y값과 추정값의 차를 계산
resid = y - y_pred
print('잔차 :Wn', resid.values)
```

잔차 :

```
[ -65655.06702452 -139621.82640197   7500.59581344 ...  159445.70383908
 -5416.96554144   82344.70383908]
```

In [20]:

```
## R2
lr.score(X,y)
```

Out[20]:

```
0.4928532179037931
```

In [21]:

```
#mse  
(resid**2).sum()/len(y)
```

Out[21]:

68351286833.039825

In [22]:

```
## rmse  
np.sqrt((resid**2).sum()/len(y))
```

Out[22]:

261440.79030067177

In [23]:

```
### 데이터셋이 모델을 잘 적합하는지 검증을 하고 싶다면  
  
from sklearn.model_selection import train_test_split  
  
X_train,X_test,y_train,y_test = train_test_split(X,y,test_size=0.3,random_state=2021)
```

In [24]:

```
lr.fit(X_train,y_train)
```

Out[24]:

LinearRegression()

In [25]:

```
y_pred = lr.predict(X_test)
```

In [26]:

```
resid = y_test - y_pred
```

In [27]:

```
## R2  
lr.score(X_test,y_test)
```

Out[27]:

0.49761443846972153

In [28]:

```
#mse  
(resid**2).sum()/len(y_test)
```

Out[28]:

66742318034.805626

In [29]:

```
## rmse  
np.sqrt((resid**2).sum()/len(y_test))
```

Out[29]:

258345.34645471288

5.3 다중선형회귀분석

5.3.1 다중선형회귀분석(다변량 회귀분석)

- 다중선형회귀분석은 두 개 이상의 독립변수가 종속변수에 미치는 영향을 추정하는 통계기법
- 회귀식은 일반적인 1차항으로 구성된 다항식

5.3.2 다중선형회귀분석 시 검토사항 .

가. 데이터가 전제하는 가정을 만족시키는가?

- 독립변수와 종속변수 간 선형성, 오차의 독립성 / 등분산성/ 정규성 등을 만족하고 있는지 확인해야함

나. 회귀모형은 통계적으로 유의한가?

- H_0 : 회귀모형은 유의하지 않다 ($\beta_1 = \beta_2 = \beta_3 = \dots = \beta_k = 0$)
- H_1 : 회귀모형은 유의하다 (적어도 하나의 β_i 는 0이 아니다)
- 회귀분석의 결과로 산출되는 F-통계량의 p-value값이 0.05보다 작으면 해당 회귀식은 통계적으로 유의하다고 볼 수 있다.

다. 모형은 데이터를 얼마나 설명할 수 있는가?

- 수정된 결정계수(adjusted R^2)를 확인한다.

라. 모형 내의 회귀계수가 유의한가?

- 단변량 회귀분석에서 회귀계수의 유의성 검토와 마찬가지로 회귀계수에 대한 t통계량의 p-value값이 0.05보다 작으면 해당 회귀 계수가 통계적으로 유의하다고 볼 수 있다.
단, 다중회귀분석을 할 때는 모든 회귀계수가 유의한지를 검정한 후 해당 회귀식을 해석해야 함

마. 모형이 데이터를 잘 적합하고 있는가?

- 모형의 잔차와 종속변수에 대한 산점도를 그리고, 회귀진단을 수행하여 판단한다.

5.3.2.1 더미변수화

가. 범주형 변수 변환

- 회귀분석은 연속형 변수를 다루는 기법이므로 범주형 데이터의 경우 형태를 변환해주어야 회귀분석을 수행할 수 있다.
- 더미변수란 0 or 1값만 가지며 어떤 특징에 해당 하는지의 여부를 표현하는 변수이다.

[예제]

kc_house_data 데이터에서 price를 종속변수로 설정하고, date와 id를 제거한 15개의 컬럼을 독립변수로 설정하여 다중선형 회귀분석을 실시한 후, 추정된 회귀모형에 대해 해석하라

In [1]:

```
import pandas as pd
import numpy as np
house = pd.read_csv('../data/kc_house_data.csv')
house.head()
```

Out[1]:

	id	date	price	bedrooms	bathrooms	sqft_living	sqft_lot	floors
0	7129300520	20141013T000000	221900.0	3	1.00	1180	5650	1.0
1	6414100192	20141209T000000	538000.0	3	2.25	2570	7242	2.0
2	5631500400	20150225T000000	180000.0	2	1.00	770	10000	1.0
3	2487200875	20141209T000000	604000.0	4	3.00	1960	5000	1.0
4	1954400510	20150218T000000	510000.0	3	2.00	1680	8080	1.0

In [2]:

```
house.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 21613 entries, 0 to 21612
Data columns (total 18 columns):
#   Column                Non-Null Count  Dtype
---  -
0   id                     21613 non-null  int64
1   date                   21613 non-null  object
2   price                  21613 non-null  float64
3   bedrooms               21613 non-null  int64
4   bathrooms              21613 non-null  float64
5   sqft_living            21613 non-null  int64
6   sqft_lot               21613 non-null  int64
7   floors                 21613 non-null  float64
8   waterfront             21613 non-null  object
9   view                   21613 non-null  int64
10  condition              21613 non-null  int64
11  grade                  21613 non-null  int64
12  sqft_above             21613 non-null  int64
13  sqft_basement          21613 non-null  int64
14  no_bathrooms            21613 non-null  int64
```

In [3]:

```
house = house.drop(['id', 'date'], axis=1)
house.head()
```

Out[3]:

	price	bedrooms	bathrooms	sqft_living	sqft_lot	floors	waterfront	view	condition	gr
0	221900.0	3	1.00	1180	5650	1.0	standard	0	3	
1	538000.0	3	2.25	2570	7242	2.0	standard	0	3	
2	180000.0	2	1.00	770	10000	1.0	standard	0	3	
3	604000.0	4	3.00	1960	5000	1.0	standard	0	5	
4	510000.0	3	2.00	1680	8080	1.0	standard	0	3	

In [4]:

```
pd.get_dummies(house, columns=['waterfront'])
```

Out[4]:

	price	bedrooms	bathrooms	sqft_living	sqft_lot	floors	view	condition	grade	sqft_above	sqft_below
0	221900.0	3	1.00	1180	5650	1.0	0	3	7	1180	0
1	538000.0	3	2.25	2570	7242	2.0	0	3	7	2170	0
2	180000.0	2	1.00	770	10000	1.0	0	3	6	770	0
3	604000.0	4	3.00	1960	5000	1.0	0	5	7	1050	0
4	510000.0	3	2.00	1680	8080	1.0	0	3	8	1680	0
...	...	...	...	...	...	...	...	...	...	...	...
21608	360000.0	3	2.50	1530	1131	3.0	0	3	8	1530	0
21609	400000.0	4	2.50	2310	5813	2.0	0	3	8	2310	0
21610	402101.0	2	0.75	1020	1350	2.0	0	3	7	1020	0

In [5]:

```
house = pd.get_dummies(house, columns=['waterfront'])
```

In [6]:

```
ols_str = 'price ~ '  
  
## 리뷰가 아니면 스탠다드 뷰 이므로 스탠다드 뷰도 삭제  
  
for i in house.columns.drop(['price', 'waterfront_standard']):  
    ols_str = ols_str + i + " + "  
  
print(ols_str)
```

```
price ~ bedrooms + bathrooms + sqft_living + sqft_lot + floors + view + condition +  
grade + sqft_above + sqft_basement + yr_built + yr_renovated + sqft_living15 + sqft_  
lot15 + waterfront_river_view +
```

In [7]:

```
ols_str[:-3]
```

Out[7]:

```
'price ~ bedrooms + bathrooms + sqft_living + sqft_lot + floors + view + condition  
+ grade + sqft_above + sqft_basement + yr_built + yr_renovated + sqft_living15 + sqf  
t_lot15 + waterfront_river_view'
```

In [8]:

```
ols_str = ols_str[:-3]
```

In [9]:

```
import numpy as np
import statsmodels.api as sm
import statsmodels.formula.api as smf

model = smf.ols(formula = ols_str, data = house)
result = model.fit()
result.summary()

## 범주형 변수가 추가되면 오류가 남
## 더미변수로 변환 필요
```

Out [9]:

OLS Regression Results

Dep. Variable:	price	R-squared:	0.654			
Model:	OLS	Adj. R-squared:	0.654			
Method:	Least Squares	F-statistic:	2913.			
Date:	Sun, 06 Feb 2022	Prob (F-statistic):	0.00			
Time:	21:20:34	Log-Likelihood:	-2.9614e+05			
No. Observations:	21613	AIC:	5.923e+05			
Df Residuals:	21598	BIC:	5.924e+05			
Df Model:	14					
Covariance Type:	nonrobust					
	coef	std err	t	P> t	[0.025	0.975]
Intercept	6.195e+06	1.38e+05	44.754	0.000	5.92e+06	6.47e+06
bedrooms	-3.931e+04	2026.602	-19.395	0.000	-4.33e+04	-3.53e+04
bathrooms	4.575e+04	3492.347	13.099	0.000	3.89e+04	5.26e+04
sqft_living	109.2685	2.435	44.877	0.000	104.496	114.041
sqft_lot	-0.0016	0.051	-0.031	0.975	-0.102	0.099
floors	2.688e+04	3783.810	7.104	0.000	1.95e+04	3.43e+04
view	4.324e+04	2273.166	19.020	0.000	3.88e+04	4.77e+04
condition	1.951e+04	2496.774	7.814	0.000	1.46e+04	2.44e+04
grade	1.197e+05	2249.149	53.230	0.000	1.15e+05	1.24e+05
sqft_above	51.5165	2.356	21.866	0.000	46.899	56.134
sqft_basement	57.7520	2.782	20.759	0.000	52.299	63.205
yr_built	-3570.1592	70.988	-50.292	0.000	-3709.302	-3431.017
yr_renovated	10.1596	3.914	2.596	0.009	2.488	17.831
sqft_living15	24.8733	3.600	6.909	0.000	17.816	31.930
sqft_lot15	-0.5505	0.078	-7.025	0.000	-0.704	-0.397
waterfront_river_view	5.791e+05	1.86e+04	31.072	0.000	5.43e+05	6.16e+05

Omnibus:	16329.452	Durbin-Watson:	1.979
Prob(Omnibus):	0.000	Jarque-Bera (JB):	1194601.451
Skew:	3.030	Prob(JB):	0.00
Kurtosis:	38.914	Cond. No.	1.90e+17

Warnings:

- [1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
 [2] The smallest eigenvalue is 1.53e-21. This might indicate that there are strong multicollinearity problems or that the design matrix is singular.

5.5 다중공선성이 존재하는가?

- 다중공선성은 회귀분석에서 독립변수들 간에 강한 상관관계가 나타나는 문제
- 다중공선성의 문제가 존재하면 정확한 회귀계수 추정이 어려움
- 독립변수들간의 상관관계를 파악한 후, 다중공선성이 발생하면 변수를 제거하거나 주성분, Ridge회귀 등을 적용하여 문제를 해결
- 다중공선성을 검사하는 방법

- 1) 독립변수들 간의 상관계수를 구하여 상관성을 직접 파악** 0.9 이상이면 다중공선성이 있다고 판단
- 2) 허용 오차**를 구했을 때 0.1이하면 다중공선성 문제가 심각하다고 할 수 있음
 허용오차 = $(1-R_i^2)$: 한 독립변수의 분산 중 다른 독립변수들에 의해서 설명되지 않는 부분을 의미함,
 즉 그 값이 작을 수록 공선성은 높다고 볼 수 있음.
- 3) 분산팽창요인(VIF)**은 허용오차의 역수로 그 값이 클수록 독립변수들 간의 상관성이 높다. 일반적으로 VIF가 10 이상이면 공선성의 문제가 심각하다고 본다.

In [10]:

```
from patsy import dmatrices
from statsmodels.stats.outliers_influence import variance_inflation_factor

Y,X = dmatrices(ols_str,
                 data = house,return_type = "dataframe")
```

In [11]:

```
X
```

Out[11]:

	Intercept	bedrooms	bathrooms	sqft_living	sqft_lot	floors	view	condition	grade	sc
0	1.0	3.0	1.00	1180.0	5650.0	1.0	0.0	3.0	7.0	
1	1.0	3.0	2.25	2570.0	7242.0	2.0	0.0	3.0	7.0	
2	1.0	2.0	1.00	770.0	10000.0	1.0	0.0	3.0	6.0	
3	1.0	4.0	3.00	1960.0	5000.0	1.0	0.0	5.0	7.0	
4	1.0	3.0	2.00	1680.0	8080.0	1.0	0.0	3.0	8.0	
...	...	...	...	...	...	...	...	...	...	...
21608	1.0	3.0	2.50	1530.0	1131.0	3.0	0.0	3.0	8.0	
21609	1.0	4.0	2.50	2310.0	5813.0	2.0	0.0	3.0	8.0	
21610	1.0	2.0	0.75	1020.0	1350.0	2.0	0.0	3.0	7.0	
21611	1.0	3.0	2.50	1600.0	2388.0	2.0	0.0	3.0	8.0	
21612	1.0	2.0	0.75	1020.0	1076.0	2.0	0.0	3.0	7.0	

21613 rows × 16 columns



In [12]:

```
# bedrooms
variance_inflation_factor(X.values, 1)
```

Out[12]:

1.6440999663822322

In [13]:

```
vif_list = []
for i in range(1, len(X.columns)):
    vif_list.append([variance_inflation_factor(X.values, i), X.columns[i]])
vif_list
```

C:\Users\Wjjang\Anaconda3\envs\WADP_Class\lib\site-packages\statsmodels\stats\outliers_influence.py:193: RuntimeWarning: divide by zero encountered in double_scalars
vif = 1. / (1. - r_squared_i)

Out[13]:

```
[[1.6440999663822322, 'bedrooms'],
 [3.3478653317649183, 'bathrooms'],
 [inf, 'sqft_living'],
 [2.0881209498906834, 'sqft_lot'],
 [1.931951201954005, 'floors'],
 [1.404260982606276, 'view'],
 [1.2216456708375973, 'condition'],
 [3.234590020681004, 'grade'],
 [inf, 'sqft_above'],
 [inf, 'sqft_basement'],
 [2.012113585849979, 'yr_built'],
 [1.1437500689366467, 'yr_renovated'],
 [2.8179992457020515, 'sqft_living15'],
 [2.1184753736433577, 'sqft_lot15'],
 [1.2030902447466258, 'waterfront_river_view']]
```

In [14]:

```
pd.DataFrame(vif_list, columns=['vif', 'variable'])
```

Out[14]:

	vif	variable
0	1.644100	bedrooms
1	3.347865	bathrooms
2	inf	sqft_living
3	2.088121	sqft_lot
4	1.931951	floors
5	1.404261	view
6	1.221646	condition
7	3.234590	grade
8	inf	sqft_above
9	inf	sqft_basement
10	2.012114	yr_built
11	1.143750	yr_renovated
12	2.817999	sqft_living15
13	2.118475	sqft_lot15
14	1.203090	waterfront_river_view

[다중공선성 검정 결과]

- sqft_living, sqft_above, sqft_basement 이 inf로 심각한 다중공선성이 존재함을 알 수 있다.
- 변수선택법을 통해 다중공선성을 제거해 보아야 한다.

5.6 변수선택법

최적회귀방정식의 선택

- 모형 내 설명변수의 수가 증가할수록 데이터 관리에는 많은 노력이 요구된다. 따라서 상황에 따라 종속변수에 영향을 미치는 유의미한 독립변수들을 선택하여 최적의 회귀방정식을 도출하는 과정이 필요하다.
- 변수를 선택할 때는 F-통계량이나 AIC와 같은 특정 기준을 근거로 변수를 제거하거나 선택한다.
- t-통계량의 유의확률이 유의수준보다 큰 변수는 통계적으로 유의하지 않으므로 제거해야하고, AIC와 같은 벌점화 기준을 가장 낮게 만드는 변수 조합을 선택해야 한다.

$$AIC = -2 \ln(L) + 2k$$

여기서 $-2\ln(L)$ 은 모형의 적합도를 의미하며, k 는 모형의 추정된 파라미터의 개수이다. $-2\ln(L)$ 에서 L 은 Likelihood function 을 의미하며, AIC 값이 낮다는 것은 즉 모형의 적합도가 높은 것을 의미한다.

(모형의 적합도란 실제 자료와 연구자의 연구 모형이 얼마나 부합하는지 평가하는 것)

여기서 $2k$ 는 모형의 추정된 파라미터의 개수를 의미하며, 해당 모형에 패널티를 주기 위해 사용한다.

실제로 어떤 모형이 $2\ln(L)$ 즉 적합도를 높이기 위해 여러 불필요한 파라미터를 사용할 수도 있다. 실제 모형 비교 시 독립변수가 많은 모형이 적합도 면에서 유리하게 되는데, 이는 즉 독립변수에 따라서 모형의 적합도에 차이가 난다는 의미이다. 따라서 이를 상쇄시키기 위하여 불필요한 파라미터, 즉 독립변수의 수가 증가할수록 $2k$ 를 증가시켜 패널티를 부여하여 모델의 품질을 평가한다.

In [15]:

```
import time
import itertools
def processSubset(X,y, feature_set):
    model = sm.OLS(y,X[list(feature_set)]) # Modeling
    regr = model.fit() # 모델 학습
    AIC = regr.aic # 모델의 AIC
    return {"model":regr, "AIC":AIC}

'''
전진선택법
'''
def forward(X, y, predictors):

    # 데이터 변수들이 미리정의된 predictors에 있는지 없는지 확인 및 분류
    remaining_predictors = [p for p in X.columns.difference(['Intercept']) if p not in
predictors]
    results = []
    for p in remaining_predictors:
        results.append(processSubset(X=X, y= y, feature_set=predictors+[p]+['Intercept']))

    # 데이터프레임으로 변환
    models = pd.DataFrame(results)

    # AIC가 가장 낮은 것을 선택
    best_model = models.loc[models['AIC'].argmin()] # index

    print("Processed ", models.shape[0], "models on", len(predictors)+1, "predictors in")
    print('Selected predictors:',best_model['model'].model.exog_names, ' AIC:',best_model[0] )
    return best_model

'''
후진소거법
'''
def backward(X,y,predictors):
    tic = time.time()
    results = []

    # 데이터 변수들이 미리정의된 predictors 조합 확인
    for combo in itertools.combinations(predictors, len(predictors) - 1):
        results.append(processSubset(X=X, y= y, feature_set=list(combo)+['Intercept']))
    models = pd.DataFrame(results)

    # 가장 낮은 AIC를 가진 모델을 선택
    best_model = models.loc[models['AIC'].argmin()]
    toc = time.time()
    print("Processed ", models.shape[0], "models on", len(predictors) - 1, "predictors in",
        (toc - tic))
    print('Selected predictors:',best_model['model'].model.exog_names, ' AIC:',best_model[0] )
    return best_model

'''
단계적 선택법
'''

def Stepwise_model(X,y):
    Stepmodels = pd.DataFrame(columns=["AIC", "model"])
    tic = time.time()
```

```

predictors = []
Smodel_before = processSubset(X,y,predictors+['Intercept'])['AIC']
# 변수 1~10개 : 0~9 -> 1~10
for i in range(1, len(X.columns.difference(['Intercept'])) + 1):
    Forward_result = forward(X=X, y=y, predictors=predictors) # constant added
    print('forward')
    Stepmodels.loc[i] = Forward_result
    predictors = Stepmodels.loc[i]["model"].model.exog_names
    predictors = [ k for k in predictors if k != 'Intercept']
    Backward_result = backward(X=X, y=y, predictors=predictors)
    if Backward_result['AIC'] < Forward_result['AIC']:
        Stepmodels.loc[i] = Backward_result
        predictors = Stepmodels.loc[i]["model"].model.exog_names
        Smodel_before = Stepmodels.loc[i]["AIC"]
        predictors = [ k for k in predictors if k != 'Intercept']
        print('backward')
    if Stepmodels.loc[i]['AIC'] > Smodel_before:
        break
    else:
        Smodel_before = Stepmodels.loc[i]["AIC"]
toc = time.time()
print("Total elapsed time:", (toc - tic), "seconds.")
return (Stepmodels['model'][len(Stepmodels['model'])])

```

후진 제거법 사용

In [16]:

```

backward_model = backward(X,Y,X.columns)
backward_model[0].summary()

```

```

Processed 16 models on 15 predictors in 0.1753528118133545
Selected predictors: ['Intercept', 'bedrooms', 'bathrooms', 'sqft_living', 'floors', 'view', 'condition', 'grade', 'sqft_above', 'sqft_basement', 'yr_built', 'yr_renovated', 'sqft_living15', 'sqft_lot15', 'waterfront_river_view', 'Intercept']
AIC: <statsmodels.regression.linear_model.RegressionResultsWrapper object at 0x000001B48E0B4CC8>

```

전진 선택법 사용

In [17]:

```
forward_model = forward(X,Y,[])
forward_model[0].summary()
```

Processed 15 models on 1 predictors in
Selected predictors: ['sqft_living', 'Intercept'] AIC: <statsmodels.regression.linear_model.RegressionResultsWrapper object at 0x000001B48E3B9108>

Out[17]:

OLS Regression Results

Dep. Variable:	price	R-squared:	0.493
Model:	OLS	Adj. R-squared:	0.493
Method:	Least Squares	F-statistic:	2.100e+04
Date:	Sun, 06 Feb 2022	Prob (F-statistic):	0.00
Time:	21:20:34	Log-Likelihood:	-3.0027e+05
No. Observations:	21613	AIC:	6.005e+05
Df Residuals:	21611	BIC:	6.006e+05
Df Model:	1		

단계적 선택법 사용

In [18]:

```
Stepwise_best_model = Stepwise_model(X=X, y=Y)
```

Processed 15 models on 1 predictors in
Selected predictors: ['sqft_living', 'Intercept'] AIC: <statsmodels.regression.linear_model.RegressionResultsWrapper object at 0x000001B48E3C6448>
forward
Processed 1 models on 0 predictors in 0.0029897689819335938
Selected predictors: ['Intercept'] AIC: <statsmodels.regression.linear_model.RegressionResultsWrapper object at 0x000001B48E3C69C8>
Processed 14 models on 2 predictors in
Selected predictors: ['sqft_living', 'view', 'Intercept'] AIC: <statsmodels.regression.linear_model.RegressionResultsWrapper object at 0x000001B48E3CF6C8>
forward
Processed 2 models on 1 predictors in 0.007512569427490234
Selected predictors: ['sqft_living', 'Intercept'] AIC: <statsmodels.regression.linear_model.RegressionResultsWrapper object at 0x000001B48E3CF8C8>
Processed 13 models on 3 predictors in
Selected predictors: ['sqft_living', 'view', 'grade', 'Intercept'] AIC: <statsmodels.regression.linear_model.RegressionResultsWrapper object at 0x000001B48E397808>
forward
Processed 2 models on 2 predictors in 0.012640462652564452

In [19]:

```
Stepwise_best_model.summary()
```

Out[19]:

OLS Regression Results

Dep. Variable:	price	R-squared:	0.654			
Model:	OLS	Adj. R-squared:	0.654			
Method:	Least Squares	F-statistic:	3398.			
Date:	Sun, 06 Feb 2022	Prob (F-statistic):	0.00			
Time:	21:20:36	Log-Likelihood:	-2.9614e+05			
No. Observations:	21613	AIC:	5.923e+05			
Df Residuals:	21600	BIC:	5.924e+05			
Df Model:	12					
Covariance Type:	nonrobust					
	coef	std err	t	P> t 	[0.025	0.975]
sqft_living	162.8899	3.574	45.575	0.000	155.884	169.895
view	4.38e+04	2234.296	19.604	0.000	3.94e+04	4.82e+04
grade	1.194e+05	2238.596	53.347	0.000	1.15e+05	1.24e+05
yr_built	-3579.2169	70.675	-50.644	0.000	-3717.744	-3440.689
waterfront_river_view	5.782e+05	1.86e+04	31.048	0.000	5.42e+05	6.15e+05
bedrooms	-3.93e+04	2026.008	-19.399	0.000	-4.33e+04	-3.53e+04
bathrooms	4.653e+04	3444.688	13.509	0.000	3.98e+04	5.33e+04
sqft_lot15	-0.5603	0.056	-10.086	0.000	-0.669	-0.451
condition	1.972e+04	2492.191	7.911	0.000	1.48e+04	2.46e+04
floors	2.471e+04	3435.357	7.193	0.000	1.8e+04	3.14e+04
sqft_living15	23.9194	3.527	6.782	0.000	17.006	30.833
yr_renovated	10.1141	3.914	2.584	0.010	2.443	17.785
Intercept	6.216e+06	1.38e+05	45.165	0.000	5.95e+06	6.49e+06
Omnibus:	16293.531	Durbin-Watson:	1.980			
Prob(Omnibus):	0.000	Jarque-Bera (JB):	1184507.441			
Skew:	3.021	Prob(JB):	0.00			
Kurtosis:	38.761	Cond. No.	2.83e+06			

Warnings:

- [1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
- [2] The condition number is large, 2.83e+06. This might indicate that there are strong multicollinearity or other numerical problems.

In [20]:

```
### 세 방법의 AIC 비교
```

```
print("단계적 선택법 : " , Stepwise_best_model.aic)
print("후진 제거법 : " , backward_model[0].aic)
print("전진 선택법 : " , forward_model[0].aic)
```

단계적 선택법 : 592315.223993904

후진 제거법 : 592315.3302147395

전진 선택법 : 600538.5665815409

단계적 선택법의 AIC가 가장 낮으므로 단계적 선택법을 선택하여 모델을 구성한다.

- 아래에서 최적의 모델에서 다중공선성이 제거되었는지 다시 한 번 확인해본다.

In [21]:

```
Stepwise_best_model.params.index
```

Out[21]:

```
Index(['sqft_living', 'view', 'grade', 'yr_built', 'waterfront_river_view',
      'bedrooms', 'bathrooms', 'sqft_lot15', 'condition', 'floors',
      'sqft_living15', 'yr_renovated', 'Intercept'],
      dtype='object')
```

In [22]:

```
X[Stepwise_best_model.params.index]
```

Out[22]:

	sqft_living	view	grade	yr_built	waterfront_river_view	bedrooms	bathrooms	sqft_lot1
0	1180.0	0.0	7.0	1955.0	0.0	3.0	1.00	5650.
1	2570.0	0.0	7.0	1951.0	0.0	3.0	2.25	7639.
2	770.0	0.0	6.0	1933.0	0.0	2.0	1.00	8062.
3	1960.0	0.0	7.0	1965.0	0.0	4.0	3.00	5000.
4	1680.0	0.0	8.0	1987.0	0.0	3.0	2.00	7503.
...	...	...	...	...	...	...	...	.
21608	1530.0	0.0	8.0	2009.0	0.0	3.0	2.50	1509.
21609	2310.0	0.0	8.0	2014.0	0.0	4.0	2.50	7200.
21610	1020.0	0.0	7.0	2009.0	0.0	2.0	0.75	2007.
21611	1600.0	0.0	8.0	2004.0	0.0	3.0	2.50	1287.
21612	1020.0	0.0	7.0	2008.0	0.0	2.0	0.75	1357.

21613 rows × 13 columns

In [23]:

```
step_X = X[Stepwise_best_model.params.index]
```

In [24]:

```
vif_list = []
for i in range(1, len(step_X.columns)):
    vif_list.append([variance_inflation_factor(step_X.values, i), step_X.columns[i]])
vif_list
```

Out[24]:

```
[[1.356653736814582, 'view'],
 [3.2043234837000343, 'grade'],
 [1.994368773317828, 'yr_built'],
 [1.2011668141150398, 'waterfront_river_view'],
 [1.6431458409341677, 'bedrooms'],
 [3.257131749275788, 'bathrooms'],
 [1.0649128091664466, 'sqft_lot15'],
 [1.2171711810620078, 'condition'],
 [1.5925144882121425, 'floors'],
 [2.7044331233908974, 'sqft_living15'],
 [1.1435892775205947, 'yr_renovated'],
 [8764.929719480495, 'Intercept']]
```

In [25]:

```
pd.DataFrame(vif_list, columns=['vif', 'variable'])
```

Out[25]:

	vif	variable
0	1.356654	view
1	3.204323	grade
2	1.994369	yr_built
3	1.201167	waterfront_river_view
4	1.643146	bedrooms
5	3.257132	bathrooms
6	1.064913	sqft_lot15
7	1.217171	condition
8	1.592514	floors
9	2.704433	sqft_living15
10	1.143589	yr_renovated
11	8764.929719	Intercept

변수별 심각한 다중공선성이 제거되었음을 확인할 수 있다.

(intercept는 절편이므로 무시한다.)

6. 로지스틱 회귀분석

6.1 로지스틱 회귀분석 개념

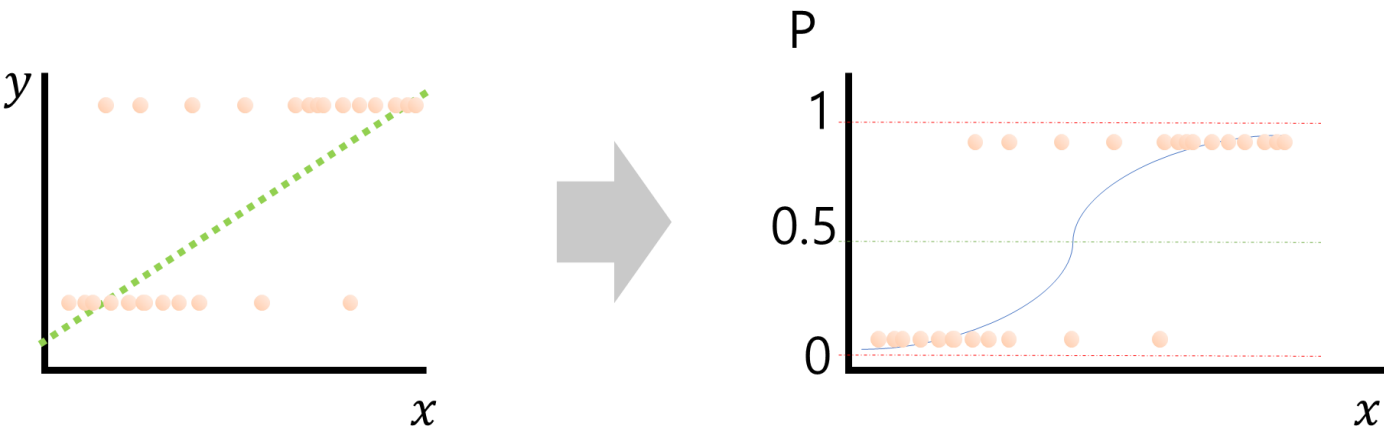
구분		독립변수 X (설명변수)	
		범주형	연속형
종속변수 y (반응변수)	범주형	카이제곱 검정	로지스틱회귀 (분류 분석)
	연속형	T검정, ANOVA검정	회귀분석

개념

- 확률 모델로서 종속변수와 독립변수의 선형 결합을 이용하여 사건의 발생 가능성을 예측하는 통계기법

특징

- 종속변수가 범주형 데이터를 대상으로 하며, 입력 데이터가 주어졌을 때 해당 데이터의 결과가 특정 분류로 나뉘기 때문에 일종의 분류(classification)기법
- 흔히 로지스틱 회귀는 종속변수가 이산형 변수(범주의 개수가 두개인 경우)일 때 사용
- 종속변수의 level이 2개 이상인 경우(두 개 이상의 범주를 가지는 경우)인 경우엔 다항 로지스틱 회귀



오즈(Odds)

$$odds = \frac{p}{1-p}$$

참일 확률 / 거짓일 확률
참일 확률이 거짓일
확률보다 얼마나 큰가?

로짓(logit)변환

$$logit(p) = \ln\left(\frac{p}{1-p}\right)$$

오즈에
자연로그를 취한다.

로지스틱(logistic)

$$\ln\left(\frac{p}{1-p}\right) = a + bX$$

$$p = f(x) = \frac{e^{a+bX}}{1 + e^{a+bX}}$$

앞에서 구한 로짓 변환한 식을
직선의 방정식과 같다고 놓고
방정식을 재정리 하면 나오는 함수

[예제 1-1]

titanic 데이터를 분할하고, train 데이터로 로지스틱 회귀모델을 만들어보자

In [1]:

```
import pandas as pd
import numpy as np

# 데이터 불러오기
df_t = pd.read_csv('../data/titanic.csv')
df_t.head()
```

Out[1]:

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	adult_male	embark
0	0	3	male	22.0	1	0	7.2500	S	Third	True	Southa
1	1	1	female	38.0	1	0	71.2833	C	First	False	Che
2	1	3	female	26.0	0	0	7.9250	S	Third	False	Southa
3	1	1	female	35.0	1	0	53.1000	S	First	False	Southa
4	0	3	male	35.0	0	0	8.0500	S	Third	True	Southa

In [2]:

```
df_t.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 11 columns):
 #   Column        Non-Null Count  Dtype  
---  --
 0   survived      891 non-null    int64  
 1   pclass        891 non-null    int64  
 2   sex           891 non-null    object  
 3   age           714 non-null    float64 
 4   sibsp         891 non-null    int64  
 5   parch         891 non-null    int64  
 6   fare          891 non-null    float64 
 7   embarked      889 non-null    object  
 8   class         891 non-null    object  
 9   adult_male    891 non-null    bool    
10  embark_town    889 non-null    object  
dtypes: bool(1), float64(2), int64(4), object(4)
memory usage: 70.6+ KB
```

In [3]:

```
## 전처리
# 결측치 제거
df_t['age'] = df_t['age'].fillna(df_t['age'].median())
```

In [4]:

```
df_t['embark_town'].value_counts().index[0]
```

Out[4]:

'Southampton'

In [5]:

```
df_t['embark_town'] = df_t['embark_town'].fillna(df_t['embark_town'].value_counts().index[0])
```

In [6]:

```
df_t.info()
```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 11 columns):
Column Non-Null Count Dtype
--- ---
0 survived 891 non-null int64
1 pclass 891 non-null int64
2 sex 891 non-null object
3 age 891 non-null float64
4 sibsp 891 non-null int64
5 parch 891 non-null int64
6 fare 891 non-null float64
7 embarked 889 non-null object
8 class 891 non-null object
9 adult_male 891 non-null bool
10 embark_town 891 non-null object
dtypes: bool(1), float64(2), int64(4), object(4)
memory usage: 70.6+ KB

In [7]:

```
# 더미변수화 및 중복 컬럼 제거  
df_t = df_t.drop(['pclass', 'embarked'], axis=1)  
  
df_t = pd.get_dummies(df_t, columns = ['sex', 'class', 'embark_town'])  
  
df_t
```

Out[7]:

	survived	age	sibsp	parch	fare	adult_male	sex_female	sex_male	class_First	class_Second
0	0	22.0	1	0	7.2500	True	0	1	0	1
1	1	38.0	1	0	71.2833	False	1	0	1	0
2	1	26.0	0	0	7.9250	False	1	0	0	0
3	1	35.0	1	0	53.1000	False	1	0	1	0
4	0	35.0	0	0	8.0500	True	0	1	0	0
...	...	...	...	...	...	...	...	...	...	...
886	0	27.0	0	0	13.0000	True	0	1	0	0
887	1	19.0	0	0	30.0000	False	1	0	1	0
888	0	28.0	1	2	23.4500	False	1	0	0	0
889	1	26.0	0	0	30.0000	True	0	1	1	0
890	0	32.0	0	0	7.7500	True	0	1	0	0

891 rows × 14 columns



In [8]:

```
## import statsmodels.api as sm 특성 상
df_t = df_t.astype(float)
df_t
```

Out[8]:

	survived	age	sibsp	parch	fare	adult_male	sex_female	sex_male	class_First	class_Second	cl
0	0.0	22.0	1.0	0.0	7.2500	1.0	0.0	1.0	0.0	0.0	
1	1.0	38.0	1.0	0.0	71.2833	0.0	1.0	0.0	1.0	0.0	
2	1.0	26.0	0.0	0.0	7.9250	0.0	1.0	0.0	0.0	0.0	
3	1.0	35.0	1.0	0.0	53.1000	0.0	1.0	0.0	1.0	0.0	
4	0.0	35.0	0.0	0.0	8.0500	1.0	0.0	1.0	0.0	0.0	
...	...	...	...	...	...	...	...	...	...	...	
886	0.0	27.0	0.0	0.0	13.0000	1.0	0.0	1.0	0.0	1.0	
887	1.0	19.0	0.0	0.0	30.0000	0.0	1.0	0.0	1.0	0.0	
888	0.0	28.0	1.0	2.0	23.4500	0.0	1.0	0.0	0.0	0.0	

In [9]:

```
# 설명변수(X), 타겟변수 (y) 분리
X = df_t.drop(['survived'],axis=1)
y = df_t['survived']
```

In [10]:

```
y.value_counts()
```

Out[10]:

```
0.0    549
1.0    342
Name: survived, dtype: int64
```

In [11]:

```
## 홀드아웃 기법

from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X,y,stratify=y, test_size=0.2,
random_state=2020)
```

In [12]:

```
# 로지스틱회귀모형 모델링
import statsmodels.api as sm
model = sm.Logit(y_train, X_train)
result = model.fit()
```

```
Optimization terminated successfully.  
Current function value: 0.409459  
Iterations 6
```

In [13]:

```
print(result.summary())
```

Logit Regression Results

Dep. Variable:	survived	No. Observations:	712
Model:	Logit	Df Residuals:	701
Method:	MLE	Df Model:	10
Date:	Tue, 07 Dec 2021	Pseudo R-squ.:	0.3849
Time:	18:44:01	Log-Likelihood:	-291.53
converged:	True	LL-Null:	-473.99
Covariance Type:	nonrobust	LLR p-value:	2.720e-72

	coef	std err	z	P> z	[0.025	0.975]
age	-0.0184	0.009	-1.960	0.050	-0.037	-0.001
sibsp	-0.5172	0.133	-3.875	0.000	-0.779	-0.256
parch	-0.3479	0.146	-2.386	0.017	-0.634	-0.062
fare	0.0049	0.003	1.606	0.108	-0.001	0.011
adult_male	-3.3484	0.575	-5.828	0.000	-4.474	-2.222
sex_female	1.0537	nan	nan	nan	nan	nan
sex_male	1.1034	nan	nan	nan	nan	nan
class_First	1.5521	nan	nan	nan	nan	nan
class_Second	0.8715	nan	nan	nan	nan	nan
class_Third	-0.2666	nan	nan	nan	nan	nan
embark_town_Ch	1.0016	2.5e+06	4.01e-07	1.000	-4.89e+06	4.89e+06
embark_town_Q	0.6284	2.5e+06	2.52e-07	1.000	-4.89e+06	4.89e+06
embark_town_S	0.5270	2.5e+06	2.11e-07	1.000	-4.89e+06	4.89e+06

In [14]:

```
# 성능 척도 AIC  
print("model AIC: ", "{:.5f}".format(result.aic))
```

model AIC: 605.06963

[예제 1-2]

회귀계수의 p-value가 유의수준 0.05보다 높게 나타나는 변수가 많으므로,
단계적 선택법을 이용하여 로지스틱 회귀분석을 다시 실시하자

In [15]:

```
import time
import itertools
def processSubset(X,y, feature_set):
    model = sm.Logit(y,X[list(feature_set)]) # Modeling
    regr = model.fit() # 모델 학습
    AIC = regr.aic # 모델의 AIC
    return {"model":regr, "AIC":AIC}

'''
전진선택법
'''
def forward(X, y, predictors):

    # 데이터 변수들이 미리정의된 predictors에 있는지 없는지 확인 및 분류
    remaining_predictors = [p for p in X.columns if p not in predictors]
    results = []
    for p in remaining_predictors:
        results.append(processSubset(X=X, y= y, feature_set=predictors+[p]))

    # 데이터프레임으로 변환
    models = pd.DataFrame(results)

    # AIC가 가장 낮은 것을 선택
    best_model = models.loc[models['AIC'].argmin()] # index

    print("Processed ", models.shape[0], "models on", len(predictors)+1, "predictors in")
    print('Selected predictors:',best_model['model'].model.exog_names, ' AIC:',best_model[0] )
    return best_model

'''
후진소거법
'''
def backward(X,y,predictors):
    tic = time.time()
    results = []

    # 데이터 변수들이 미리정의된 predictors 조합 확인
    for combo in itertools.combinations(predictors, len(predictors) - 1):
        results.append(processSubset(X=X, y= y, feature_set=list(combo)))
    models = pd.DataFrame(results)

    # 가장 낮은 AIC를 가진 모델을 선택
    best_model = models.loc[models['AIC'].argmin()]
    toc = time.time()
    print("Processed ", models.shape[0], "models on", len(predictors) - 1, "predictors in",
          (toc - tic))
    print('Selected predictors:',best_model['model'].model.exog_names, ' AIC:',best_model[0] )
    return best_model

'''
단계적 선택법
'''
def Stepwise_model(X,y):
    Stepmodels = pd.DataFrame(columns=["AIC", "model"])
    tic = time.time()
    predictors = [X.columns[0]]
```

```

Smodel_before = processSubset(X,y,predictors)['AIC']
# 변수 1~10개 : 0~9 -> 1~10
for i in range(1, len(X.columns) + 1):
    Forward_result = forward(X=X, y=y, predictors=predictors) # constant added
    print('forward')
    Stepmodels.loc[i] = Forward_result
    predictors = Stepmodels.loc[i]["model"].model.exog_names
    predictors = [ k for k in predictors]
    Backward_result = backward(X=X, y=y, predictors=predictors)
    if Backward_result['AIC'] < Forward_result['AIC']:
        Stepmodels.loc[i] = Backward_result
        predictors = Stepmodels.loc[i]["model"].model.exog_names
        Smodel_before = Stepmodels.loc[i]["AIC"]
        predictors = [ k for k in predictors]
        print('backward')
    if Stepmodels.loc[i]['AIC'] > Smodel_before:
        break
    else:
        Smodel_before = Stepmodels.loc[i]["AIC"]
toc = time.time()
print("Total elapsed time:", (toc - tic), "seconds.")
return (Stepmodels['model'][len(Stepmodels['model'])])

```

In [16]:

```
Stepwise_best_model = Stepwise_model(X_train,y_train)
```

```

Optimization terminated successfully.
    Current function value: 0.664126
    Iterations 4
Optimization terminated successfully.
    Current function value: 0.661913
    Iterations 5
Optimization terminated successfully.
    Current function value: 0.662992
    Iterations 4
Optimization terminated successfully.
    Current function value: 0.621623
    Iterations 6
Optimization terminated successfully.
    Current function value: 0.502203
    Iterations 6
Optimization terminated successfully.
    Current function value: 0.529373
    Iterations 6
Optimization terminated successfully.
    Current function value: 0.500107

```

In [17]:

```
Stepwise_best_model.aic
```

Out[17]:

```
601.1467578209549
```

In [18]:

```
Stepwise_best_model.summary()
```

Out[18]:

Logit Regression Results

Dep. Variable:	survived	No. Observations:	712
Model:	Logit	Df Residuals:	703
Method:	MLE	Df Model:	8
Date:	Tue, 07 Dec 2021	Pseudo R-squ.:	0.3849
Time:	18:44:03	Log-Likelihood:	-291.57
converged:	True	LL-Null:	-473.99
Covariance Type:	nonrobust	LLR p-value:	6.159e-74

	coef	std err	z	P> z	[0.025	0.975]
adult_male	-3.3122	0.256	-12.929	0.000	-3.814	-2.810
class_First	3.1538	0.501	6.296	0.000	2.172	4.136
class_Second	2.4695	0.391	6.324	0.000	1.704	3.235
sibsp	-0.5143	0.124	-4.136	0.000	-0.758	-0.271
class_Third	1.3493	0.326	4.134	0.000	0.710	1.989
age	-0.0186	0.009	-2.090	0.037	-0.036	-0.001
parch	-0.3526	0.144	-2.451	0.014	-0.635	-0.071
fare	0.0049	0.003	1.615	0.106	-0.001	0.011
embark_town_Chernbourg	0.4644	0.280	1.659	0.097	-0.084	1.013

In [19]:

```
# 로지스틱 회귀모델로 적합한 회귀계수 확인
```

```
##단계적 선택법
```

```
print(Stepwise_best_model.params)
```

```
adult_male      -3.312166
class_First      3.153808
class_Second     2.469512
sibsp            -0.514293
class_Third      1.349336
age             -0.018594
parch           -0.352575
fare             0.004931
embark_town_Chernbourg 0.464423
dtype: float64
```

In [20]:

```
# 적합한 회귀계수로 오즈비 확인

# adult_male 일때, adult_male이 아닌 사람에 비해 생존확률이 99 % 감소한다
# class_First일때, class_Second일 때보다 생존확률은 약 2배 증가 , class_Third일 때보다 약 6배 증가
# sibsp가 1단위 증가할 때 생존확률 40% 감소
# fare가 1단위 증가할 때, 생존확률 0.004% 증가

np.exp(Stepwise_best_model.params)
```

Out[20]:

```
adult_male          0.036437
class_First         23.425090
class_Second        11.816675
sibsp               0.597923
class_Third         3.854864
age                 0.981578
parch               0.702876
fare                1.004943
embark_town_Chерbourg 1.591095
dtype: float64
```

6.2 로지스틱 회귀분석 : 분류분석으로 활용하기

In [21]:

```
Stepwise_best_model.params.index
```

Out[21]:

```
Index(['adult_male', 'class_First', 'class_Second', 'sibsp', 'class_Third',
      'age', 'parch', 'fare', 'embark_town_Chерbourg'],
      dtype='object')
```

In [22]:

```
X_test = X_test[Stepwise_best_model.params.index]
```

In [23]:

```
y_test
```

Out[23]:

```
560    0.0
130    0.0
551    0.0
587    1.0
2      1.0
...
818    0.0
113    0.0
605    0.0
642    0.0
206    0.0
Name: survived, Length: 179, dtype: float64
```

In [24]:

```
# 예측하기
y_pred = Stepwise_best_model.predict(X_test)
y_pred
```

Out[24]:

```
560    0.079787
130    0.111742
551    0.228554
587    0.216542
2      0.711970
...
818    0.061194
113    0.625184
605    0.044369
642    0.310386
206    0.047699
Length: 179, dtype: float64
```

In [25]:

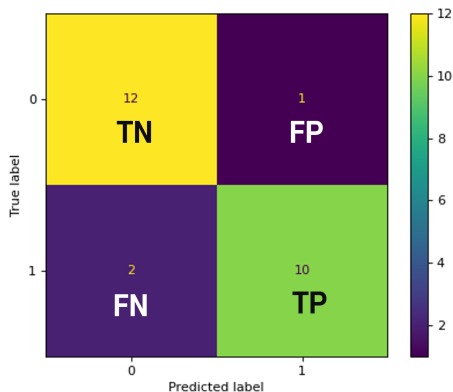
```
# 임계값을 설정해서 예측값을 이진분류로 반환하기

def cut_off(y, threshold):
    Y = y.copy()
    Y[Y>threshold] = 1
    Y[Y<=threshold] = 0
    return(Y.astype(int))

y_pred = cut_off(y_pred, 0.5) #임계값을 0.5로 설정
y_pred
```

Out[25]:

```
560    0
130    0
551    0
587    0
2      1
..
818    0
113    1
605    0
642    0
206    0
Length: 179, dtype: int32
```



- Accuracy(정확도) = $(TP + TN) / \text{전체}$
- Sensitivity(민감도, 재현도) : $TP / (TP + FN)$
- Precision(정밀도) : $TP / (TP + FP)$
- Specificity(특이도) : $TN / (FP + TN)$
- F1-score : $2 * \text{민감도} * \text{정밀도} / (\text{민감도} + \text{정밀도})$

불균형 데이터에서의 주요 척도

FPR(거짓양성계수) : 1- 특이도

- 0인 케이스에 대해 1로 잘못 예측한 비율
- ex) 암환자가 아닌데 암이라고 진단 함

In [26]:

```
from sklearn.metrics import confusion_matrix, accuracy_score, recall_score, precision_score, f1_score
```

In [27]:

```
# 혼동행렬
cm = confusion_matrix(y_test, y_pred)
print(cm)
```

```
[[92 18]
 [21 48]]
```

In [28]:

```
print("Accuracy : ", accuracy_score(y_test, y_pred))
print("Sensitivity(recall) : ", recall_score(y_test, y_pred))
print("precision_score : ", precision_score(y_test, y_pred))
print("f1_score : ", f1_score(y_test, y_pred))
```

```
Accuracy : 0.7821229050279329
Sensitivity(recall) : 0.6956521739130435
precision_score : 0.7272727272727273
f1_score : 0.7111111111111111
```

6.3 분류분석 평가지표

ROC 곡선

- x축은 FPR(1-특이도), y축은 재현율을 가지는 그래프
- TPR(민감도)은 1 이고, FPR(1-특이도)는 0인 경우 가장 이상적
- 거짓 양성 비율(False Positive Rate;FPR) 의미

이름	의미
민감도(재현율)	실제 True일 때 True라고 예측하는 비율
특이도	실제 False일 때 False라고 예측하는 비율
FPR	FP/TN+FP는 실제로는 음성인 것과 양성으로 잘못 예측한 것의 비율

AUC

- ROC 곡선의 아래 면적을 계산한 것으로 값이 클수록 모델의 성능이 좋다고 판단 가능

roc_auc_score

- binary 분류일 때 : roc_auc_score(Y, clf.predict_proba(X)[:, 1])
- 다중 클래스 분류일 때 : roc_auc_score(y, clf.predict_proba(X), multi_class='ovr')
- 여러 개의 라벨을 분류할 때 : roc_auc_score(y, y_pred, average=None)

In [29]:

```
# 확률로 예측하기
y_pred_proba = Stepwise_best_model.predict(X_test)
y_pred_proba
```

Out[29]:

```
560    0.079787
130    0.111742
551    0.228554
587    0.216542
2      0.711970
...
818    0.061194
113    0.625184
605    0.044369
642    0.310386
206    0.047699
Length: 179, dtype: float64
```

[예제 1-3]

ROC 커브 그리기 및 ROC_AUC 산출하기

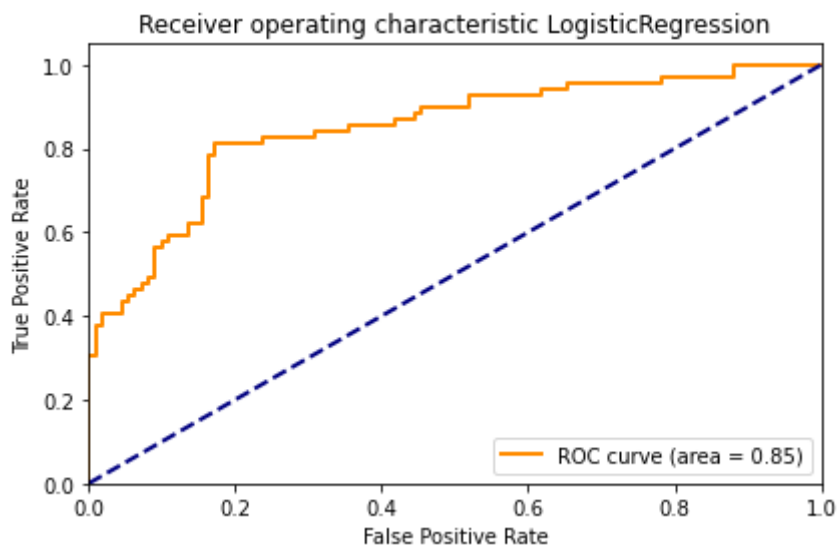
In [30]:

```
from sklearn.metrics import roc_curve, auc
import matplotlib.pyplot as plt
# ROC 커브 시각화

def draw_roc(test_y, pred_y, model_name):
    fpr, tpr, thr = roc_curve(test_y, pred_y)
    roc_auc = auc(fpr, tpr)
    plt.figure()
    lw = 2
    plt.plot(fpr, tpr, color = 'darkorange',
             lw = lw, label = "ROC curve (area = %0.2f)" % roc_auc)
    plt.plot([0, 1], [0, 1], color='navy', lw=lw, linestyle='--')
    plt.xlim([0.0, 1.0])
    plt.ylim([0.0, 1.05])
    plt.xlabel("False Positive Rate")
    plt.ylabel("True Positive Rate")
    plt.title("Receiver operating characteristic {}".format(model_name))
    plt.legend(loc='lower right')
    plt.ion()
    plt.tight_layout()
    plt.show()
```

In [31]:

```
draw_roc(y_test, y_pred_proba, "LogisticRegression")
```



In [33]:

```
from sklearn.metrics import roc_auc_score

roc_auc_score(y_test, y_pred_proba)
```

Out[33]:

0.8469038208168642

6.4 다항 로지스틱 회귀모형 실습

[예제]

iris 데이터의 Species 를 분류하는 다항 로지스틱 회귀분석을 실시하고 오분류표를 만들어라

In [1]:

```
import pandas as pd
import warnings

warnings.filterwarnings(action='ignore')
#warnings.filterwarnings(action='default')

iris = pd.read_csv('../data/iris.csv')

X = iris.drop(['target'],axis=1)
y = iris.target
```

In [2]:

```
y.value_counts()
```

Out[2]:

```
Iris-setosa      50
Iris-virginica   50
Iris-versicolor  50
Name: target, dtype: int64
```

In [3]:

```
# 훈련셋 평가셋 분리하기
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, stratify=y, train_size=0.7,
                                                    test_size = 0.3, random_state=123)
print(X_train.shape, X_test.shape, y_train.shape, y_test.shape)
```

```
(105, 4) (45, 4) (105,) (45,)
```

In [4]:

```
y_test.value_counts()
```

Out[4]:

```
Iris-versicolor  15
Iris-virginica   15
Iris-setosa      15
Name: target, dtype: int64
```

6.4.1 sklearn 모델 생성

- sklearn 모델은 l2 패널티를 이용하여 전통적 통계모델에서 다중공선성의 문제를 내부적으로 해결해준다.
- 독립변수 간의 상관성이 높은 변수라면, l2패널티를 0에 가깝게하여 변수를 삭제하는 것과 같은 효과를 보인다.

In [5]:

```
# 모델 적합하기
from sklearn.linear_model import LogisticRegression

model = LogisticRegression()
model.fit(X_train, y_train)
print(model)
```

LogisticRegression()

6.4.2 모델 평가

In [6]:

```
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score,
roc_auc_score
# 테스트셋 예측
predicted = model.predict(X_test)

# 오분류표 생성
cm = confusion_matrix(y_test, predicted)
cmtb = pd.DataFrame(cm, columns=['predicted_setosa', 'predicted_versicolor',
                                'predicted_virginica'],
                    index = ['setosa', 'versicolor', 'virginica'])

cmtb
```

Out[6]:

	predicted_setosa	predicted_versicolor	predicted_virginica
setosa	15	0	0
versicolor	0	14	1
virginica	0	0	15

In [7]:

```
model.predict(X_test)
```

Out[7]:

```
array(['Iris-versicolor', 'Iris-virginica', 'Iris-versicolor',
      'Iris-versicolor', 'Iris-virginica', 'Iris-setosa', 'Iris-setosa',
      'Iris-versicolor', 'Iris-setosa', 'Iris-setosa', 'Iris-virginica',
      'Iris-setosa', 'Iris-virginica', 'Iris-setosa', 'Iris-versicolor',
      'Iris-setosa', 'Iris-virginica', 'Iris-virginica', 'Iris-setosa',
      'Iris-setosa', 'Iris-setosa', 'Iris-setosa', 'Iris-setosa',
      'Iris-versicolor', 'Iris-setosa', 'Iris-versicolor',
      'Iris-virginica', 'Iris-setosa', 'Iris-versicolor',
      'Iris-virginica', 'Iris-virginica', 'Iris-virginica',
      'Iris-virginica', 'Iris-versicolor', 'Iris-versicolor',
      'Iris-versicolor', 'Iris-virginica', 'Iris-virginica',
      'Iris-virginica', 'Iris-setosa', 'Iris-virginica',
      'Iris-versicolor', 'Iris-versicolor', 'Iris-virginica',
      'Iris-versicolor'], dtype=object)
```

In [8]:

```
# 정확도
print('Accuracy Score: ', accuracy_score(y_test, predicted))

print('\n')
# 분류 리포트 생성하기
class_report = classification_report(y_test, predicted)
print(class_report)
```

Accuracy Score: 0.9777777777777777

	precision	recall	f1-score	support
Iris-setosa	1.00	1.00	1.00	15
Iris-versicolor	1.00	0.93	0.97	15
Iris-virginica	0.94	1.00	0.97	15
accuracy			0.98	45
macro avg	0.98	0.98	0.98	45
weighted avg	0.98	0.98	0.98	45

In [9]:

```
roc_auc_score(y_test, model.predict_proba(X_test), multi_class='ovr')
```

Out[9]:

0.9985185185185186

6.4.3 다항 로지스틱 회귀 계수 해석

In [11]:

```
# 회귀계수 확인하기
print('Intercept: \n', model.intercept_)
print('Coefficient: \n', model.coef_)
```

```
Intercept:
[ 9.42940015  2.10066833 -11.53006848]
Coefficient:
[[-0.45747705  0.87262687 -2.30840796 -0.96053751]
 [ 0.37578083 -0.19466078 -0.16297032 -0.75289644]
 [ 0.08169622 -0.6779661  2.47137828  1.71343395]]
```

In [12]:

```
# 오즈비 계산하기
import numpy as np
np.exp(model.coef_)
```

Out[12]:

```
array([[ 0.63287835,  2.39318921,  0.09941941,  0.38268713],
       [ 1.45612796,  0.82311383,  0.8496164 ,  0.47100035],
       [ 1.08512612,  0.50764845, 11.83875275,  5.54798029]])
```

In [13]:

```
pd.DataFrame(np.exp(model.coef_), columns=X_train.columns, index = model.classes_)
```

Out[13]:

	sepal length	sepal width	petal length	petal width
Iris-setosa	0.632878	2.393189	0.099419	0.382687
Iris-versicolor	1.456128	0.823114	0.849616	0.471000
Iris-virginica	1.085126	0.507648	11.838753	5.547980

* 다른 변수가 일정할 때, sepal width가 1단위 증가하면 Iris-setosa로 분류될 확률이 2.393189배 증가한다.

7. 계층적 군집분석

7.1 계층적 군집분석 개념

n개의 군집으로 시작해 점차 군집의 개수를 줄여나가는 방법

가. 종류

- 최단연결법
 - $n \times n$ 거리행렬에서 거리가 가장 가까운 데이터를 묶어서 군집을 형성함
- 최장연결법
 - 데이터와의 거리를 계산할 때 최장거리를 거리로 계산함
- 평균연결법
 - 데이터와의 거리를 계산할 때 평균을 거리로 계산함
 - 와드연결법
 - 군집 내 편차들의 제곱합을 고려한 방법
 - 군집 간 정보의 손실을 최소화하기 위해 군집화를 진행함

나. 군집화

- 거리행렬을 통해 가장 가까운 거리의 객체들 관계를 규명함
- 덴드로그램 그리기
- 덴드로그램의 최상단부터 세로축의 개수에 따라 가로선을 그어 군집 개수를 선택함
- 각 객체들의 구성을 고려해 적절한 군집수 생성함

함수1. linkage 함수

`linkage(y, method='single', metric='euclidean')`

- y: 거리행렬(데이터를 넣으면 됨)
 - 모든 요소가 유한해야함(NaNs, infs가 없어야함)
- method:
 - single : 최단연결법
 - complete : 최장연결법
 - average : 평균연결법
 - centroid : 중심연결법
 - ward : 와드연결법
- metric:
 - euclidean : 유클리디언 거리
 - seuclidean : 표준화거리
 - mahalanobis : 마할라노비스 거리
 - chebyshev : 체비셰프 거리
 - cityblock : 맨하탄거리
 - canberra : 캔버라 거리
 - minkowski : 민코우스키 거리
 - jaccard : 자카드 거리
 - cosine : 코사인 유사도

함수2. dendrogram 함수

```
dendrogram(Z, orientation='top', labels=None, color_threshold=None, get_leaves=True, ,
* Z : linkage matrix. linkage 함수로 생성함
* orientation
    - top: 플롯이 위에서 아래방향으로 그려짐
    - bottom: 플롯이 아래에서 위방향으로 그려짐
    - left: 플롯이 오른쪽에서 왼쪽방향으로 그려짐
    - right: 플롯이 왼쪽에서 오른쪽방향으로 그려짐

scipy.cluster.hierarchy.dendrogram(Z, p=30, truncate_mode=None, color_threshold=None,
get_leaves=True, orientation='top', labels=None, count_sort=False, distance_sort=False,
show_leaf_counts=True, no_plot=False, no_labels=False, leaf_font_size=None, leaf_rotation=None,
leaf_label_func=None, show_contracted=False, link_color_func=None, ax=None,
above_threshold_color='C0')
```

7.2 계층적 군집분석 실습

[예제]

USArrests 데이터의 정보로 거리를 구하고 최단, 최장, 평균연결법을 실시해보아라

이 데이터 세트에는 1973년에 미국 50개 주 각각에서 폭행, 살인 및 강간으로 체포된 인구 100,000명당 통계가 포함되어 있습니다. 또한 도시 지역에 거주하는 인구 비율도 제공됩니다.

4개의 변수에 대한 50개의 관측값이 있는 데이터 프레임.

- Murder: 살인 체포(100,000명당)
- Assault: 폭행 체포(100,000명당)
- UrbanPop : 도시 인구 비율
- Rape: 강간 체포(100,000명당)

In [1]:

```
import pandas as pd
import numpy as np
from scipy.cluster.hierarchy import dendrogram, linkage, fcluster
from matplotlib import pyplot as plt
```

In [2]:

```
US = pd.read_csv('../data/USArrests.csv')
US.head()
```

Out[2]:

	Unnamed: 0	Murder	Assault	UrbanPop	Rape
0	Alabama	13.2	236	58	21.2
1	Alaska	10.0	263	48	44.5
2	Arizona	8.1	294	80	31.0
3	Arkansas	8.8	190	50	19.5
4	California	9.0	276	91	40.6

In [3]:

```
US.columns = ['State', 'Murder', 'Assault', 'UrbanPop', 'Rape']  
labelList=US.State.tolist()  
US.head()
```

Out[3]:

	State	Murder	Assault	UrbanPop	Rape
0	Alabama	13.2	236	58	21.2
1	Alaska	10.0	263	48	44.5
2	Arizona	8.1	294	80	31.0
3	Arkansas	8.8	190	50	19.5
4	California	9.0	276	91	40.6

In [4]:

```
# 최단연결법
```

```
single = linkage(US.iloc[:, 1::], metric = 'euclidean', method='single')
```

```
# 덴드로그램 그리기
```

```
plt.figure(figsize=(10, 7))
```

```
dendrogram(single,
```

```
    orientation='top',
```

```
    labels=labelList,
```

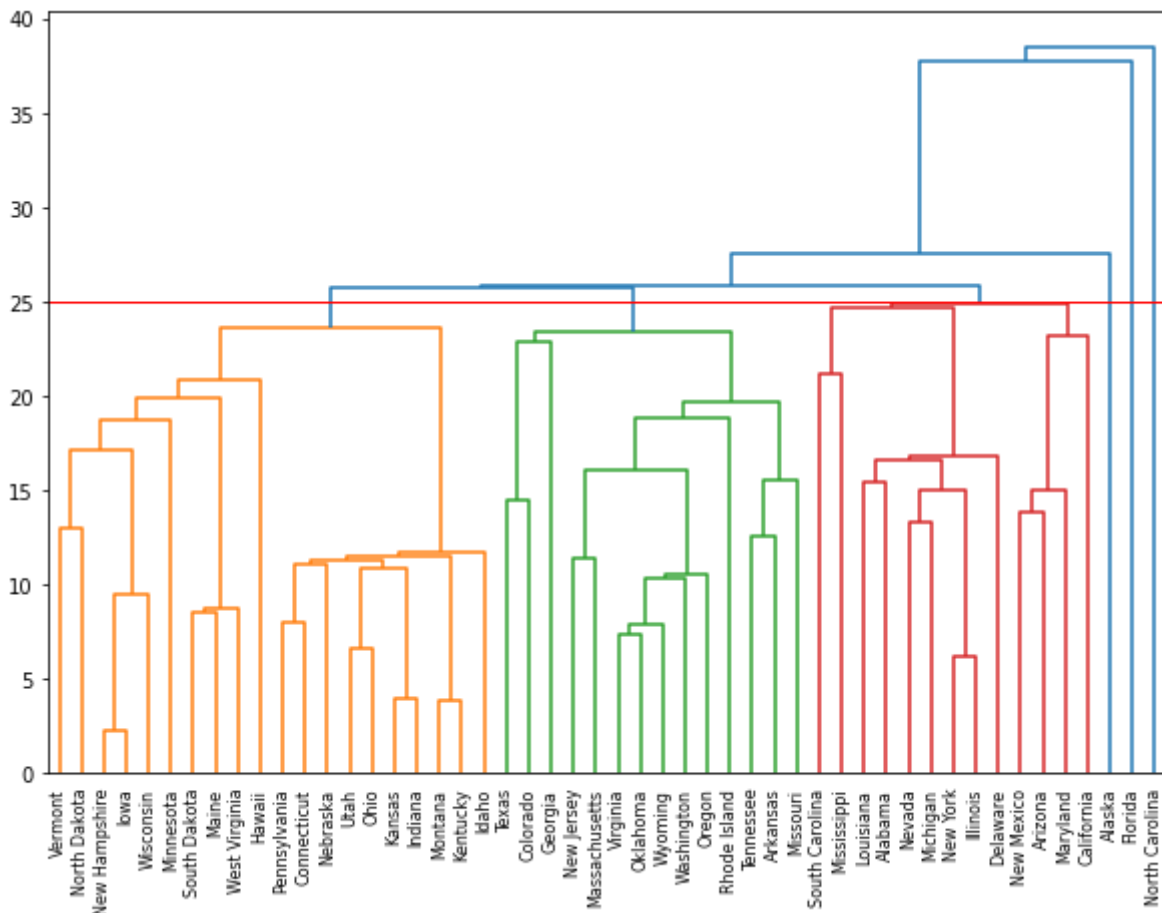
```
    distance_sort='descending',
```

```
    color_threshold=25, #군집의 수를 설정하는 Height 값 설정
```

```
    show_leaf_counts=True)
```

```
plt.axhline(y=25, color='r', linewidth=1) #Height 값에 따라 선을 그어 적절한 군집 수 설정
```

```
plt.show()
```



In [5]:

```
single
```

Out[5]:

```
array([[14.      , 28.      , 2.29128785, 2.      ],
       [16.      , 25.      , 3.8340579 , 2.      ],
       [13.      , 15.      , 3.92937654, 2.      ],
       [12.      , 31.      , 6.23698645, 2.      ],
       [34.      , 43.      , 6.63777071, 2.      ],
       [35.      , 45.      , 7.35527022, 2.      ],
       [49.      , 55.      , 7.93095202, 3.      ],
       [ 6.      , 37.      , 8.0274529 , 2.      ],
       [18.      , 40.      , 8.53756406, 2.      ],
       [47.      , 58.      , 8.76698352, 3.      ],
       [48.      , 50.      , 9.50841732, 3.      ],
       [46.      , 56.      , 10.30533842, 4.      ],
       [36.      , 61.      , 10.57922492, 5.      ],
       [52.      , 54.      , 10.92016483, 4.      ],
       [26.      , 57.      , 11.070682  , 3.      ],
       [63.      , 64.      , 11.25610945, 7.      ],
       [20.      , 29.      , 11.45643924, 2.      ],
       [51.      , 65.      , 11.52822623, 9.      ]]
```

```
fcluster(Z, t, criterion='distance')
```

- Z: linkage 함수로 반환되는 matrix로 구성된 계층군집
- t: 상수. flat clusters 형성할 때 적용하는 임계값. 덴드로그램의 color_threshold랑 일치하게 설정하면 그 값에 따른 군집결과 확인 가능
- criterion: flat cluster 형성하는 기준
 - distance: 각 군집의 원래 관측치가 t보다 큰 동위원소 거리를 갖지 않도록 군집을 형성

In [6]:

```
# state마다 어떤 군집에 설정되었는지 확인
assignments = fcluster(single, 25, 'distance')
assignments
```

Out[6]:

```
array([1, 4, 1, 2, 1, 2, 3, 1, 5, 2, 3, 3, 1, 3, 3, 3, 3, 1, 3, 1, 2, 1,
       3, 1, 2, 3, 3, 1, 3, 2, 1, 1, 6, 3, 3, 2, 2, 3, 2, 1, 3, 2, 2, 3,
       3, 2, 2, 3, 3, 2], dtype=int32)
```

In [7]:

```
US['cluster'] = assignments
```

In [8]:

```
US.head()
```

Out[8]:

	State	Murder	Assault	UrbanPop	Rape	cluster
0	Alabama	13.2	236	58	21.2	1
1	Alaska	10.0	263	48	44.5	4
2	Arizona	8.1	294	80	31.0	1
3	Arkansas	8.8	190	50	19.5	2
4	California	9.0	276	91	40.6	1

In [9]:

```
# 최장연결법
```

```
complete = linkage(US.iloc[:, 1::], metric = 'euclidean', method='complete')
```

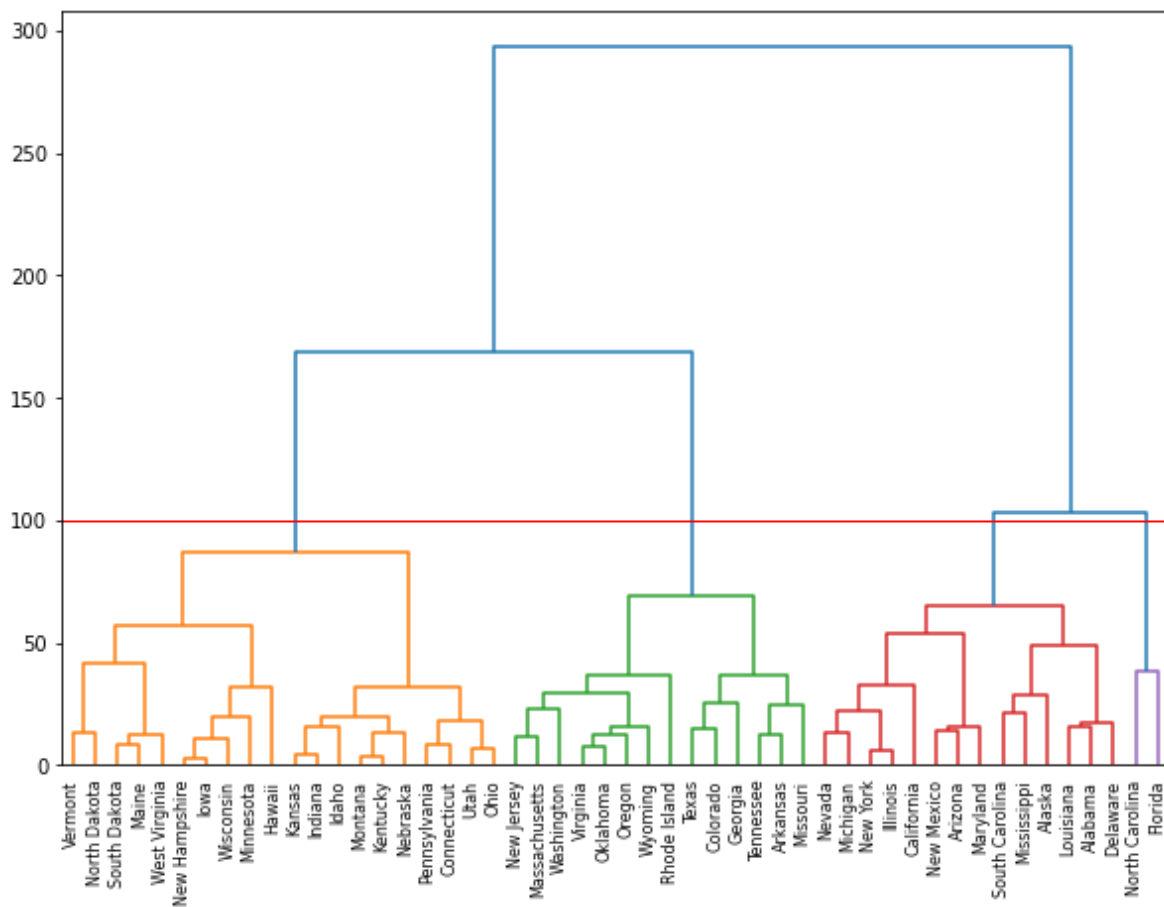
```
# 덴드로그램 그리기
```

```
plt.figure(figsize=(10, 7))
```

```
dendrogram(complete,  
            orientation='top',  
            labels=labelList,  
            distance_sort='descending',  
            color_threshold=100,  
            show_leaf_counts=True)
```

```
plt.axhline(y=100, color='r', linewidth=1) #Height 값에 따라 선을 그어 적절한 군집 수 설정
```

```
plt.show()
```



In [10]:

```
# 평균연결법
```

```
average = linkage(US.iloc[:, 1::], metric = 'euclidean', method='average')
```

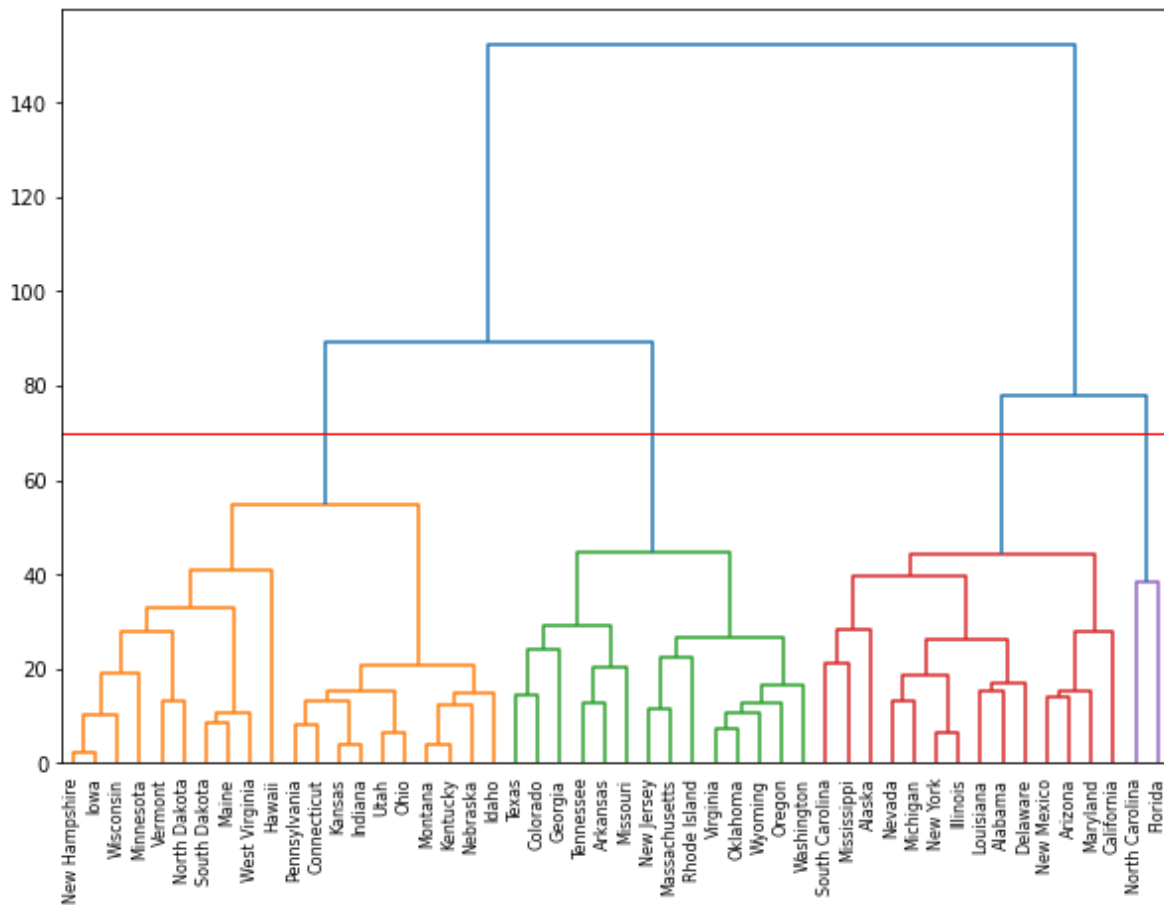
```
# 덴드로그램 그리기
```

```
plt.figure(figsize=(10, 7))
```

```
dendrogram(average,  
            orientation='top',  
            labels=labelList,  
            distance_sort='descending',  
            color_threshold=70,  
            show_leaf_counts=True)
```

```
plt.axhline(y=70, color='r', linewidth=1) #Height 값에 따라 선을 그어 적절한 군집 수 설정
```

```
plt.show()
```



In [11]:

```
# 와드연결법
```

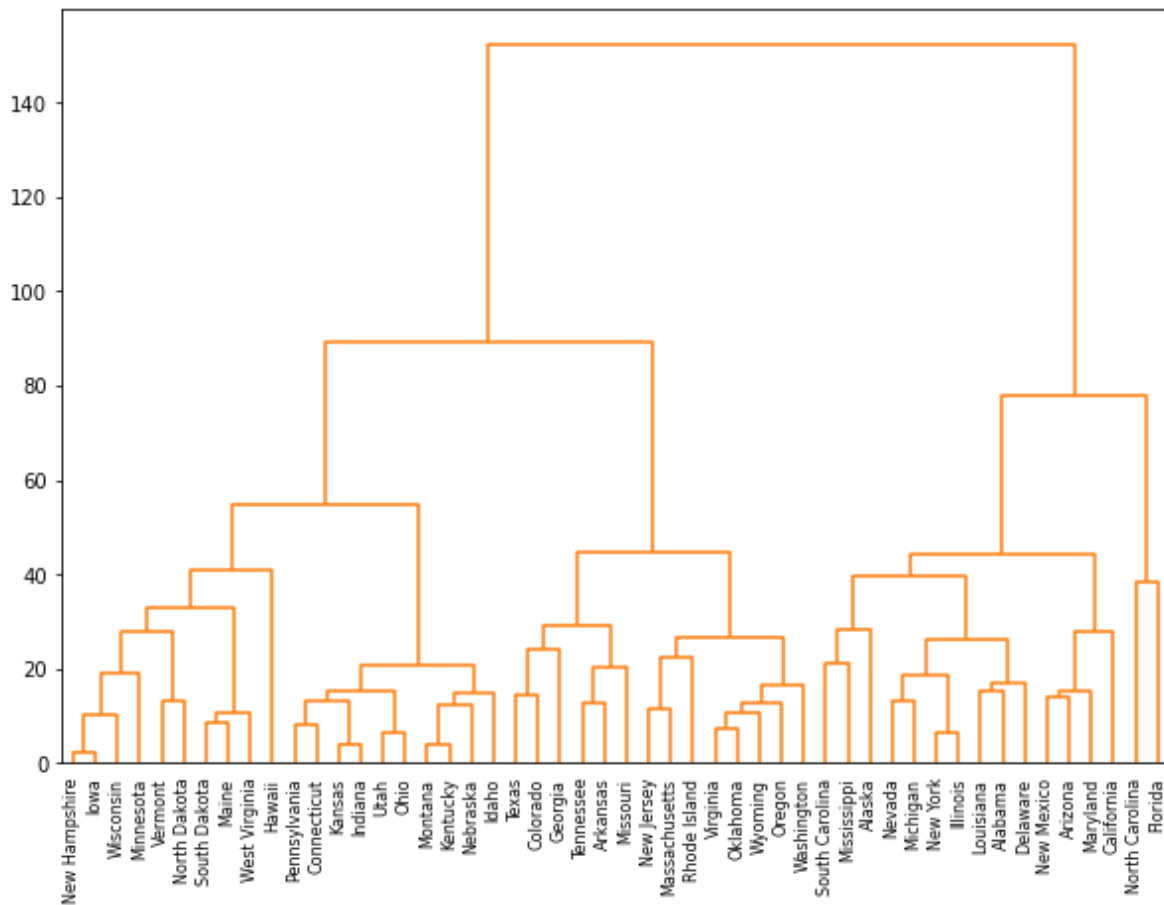
```
ward = linkage(US.iloc[:, 1::], metric = 'euclidean', method='ward')
```

```
# 덴드로그램 그리기
```

```
plt.figure(figsize=(10, 7))
```

```
dendrogram(average,  
            orientation='top',  
            labels=labelList,  
            distance_sort='descending',  
            color_threshold=250,  
            show_leaf_counts=True)
```

```
plt.axhline(y=250, color='r', linewidth=1) #Height 값에 따라 선을 그어 적절한 군집 수 설정  
plt.show()
```



In [12]:

```
# state마다 어떤 군집에 설정되었는지 확인
```

```
assignments = fcluster(ward, 250, 'distance')
```

```
assignments
```

Out[12]:

```
array([1, 1, 1, 2, 1, 2, 3, 1, 1, 2, 3, 3, 1, 3, 3, 3, 3, 1, 3, 1, 2, 1,  
       3, 1, 2, 3, 3, 1, 3, 2, 1, 1, 3, 3, 2, 2, 3, 2, 1, 3, 2, 2, 3,  
       3, 2, 2, 3, 3, 2], dtype=int32)
```

In [13]:

```
US['cluster'] = assignments
```

In [14]:

```
US.groupby('cluster').mean()
```

Out[14]:

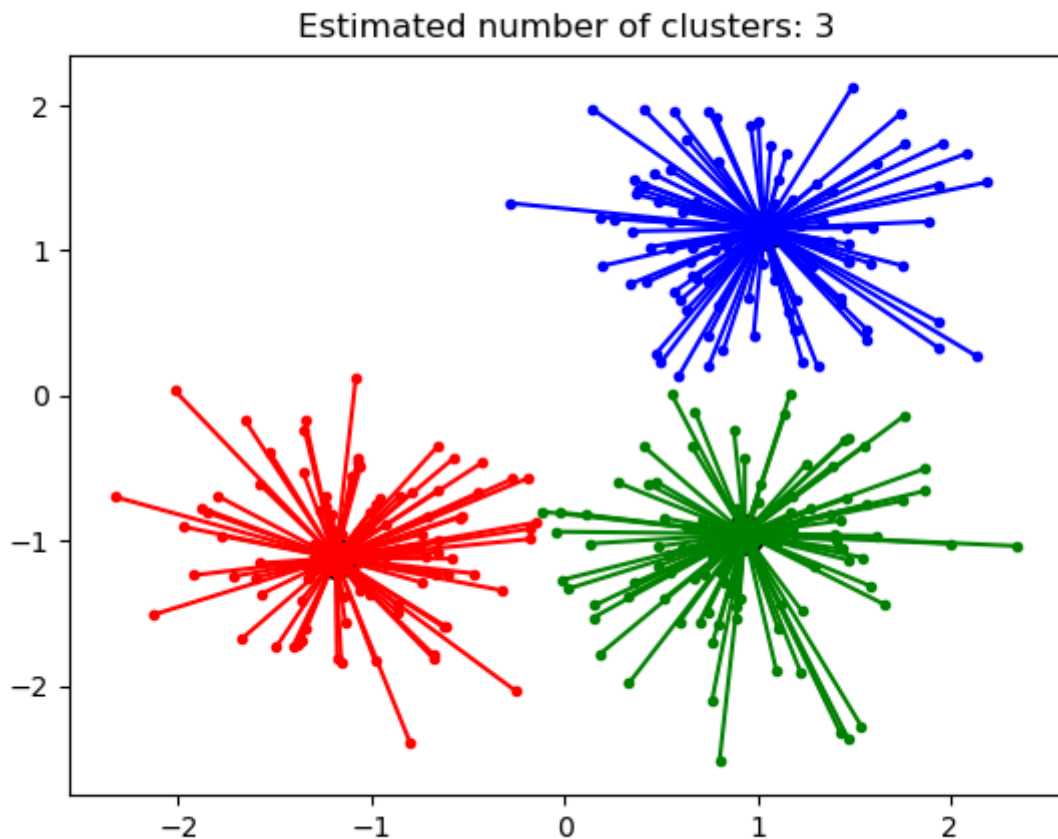
	Murder	Assault	UrbanPop	Rape
cluster				
1	11.812500	272.562500	68.312500	28.375000
2	8.214286	173.285714	70.642857	22.842857
3	4.270000	87.550000	59.750000	14.390000

7.3 비계층적 군집분석

7.3.1 K-means

가. 개념

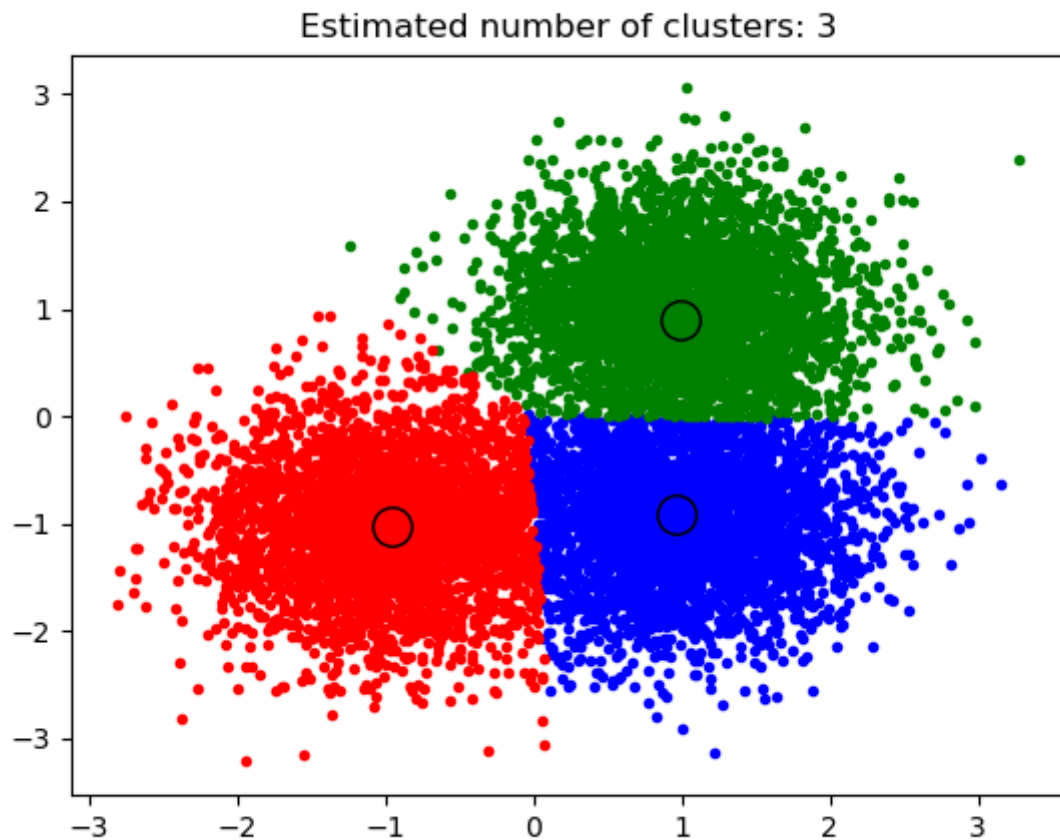
- 주어진 데이터를 k개의 클러스터로 묶는 알고리즘
- 각 클러스터와 거리 차이의 분산을 최소화 하는 방식으로 작동
 - n개의 객체들을 각 집합 내 객체간 응집도를 최대로 하는 k개의 집합 S를 찾는 것이 목표
 - 응집도 : 각 집합별 중심점 ~ 집합 내 객체들 간 거리의 제곱합



- 출처 : <https://scikit-learn.org/stable/modules/clustering.html#overview-of-clustering-methods>
(<https://scikit-learn.org/stable/modules/clustering.html#overview-of-clustering-methods>)

나. 표준 알고리즘

- (1) i번째 클러스터의 중심을 μ_i , 클러스터에 속하는 점의 집합을 S_i 라고 할 때, 전체 분산을 최소화하는 S를 찾는 것이 목표
- (2) 초기의 μ_i 를 설정
- (3) 클러스터 설정: 각 데이터로부터 각 클러스터들의 μ_i 까지의 유클리드 거리를 계산해, 해당 데이터에서 가장 가까운 클러스터를 찾아 데이터를 배당함
- (4) 클러스터 중심 재조정: μ_i 를 각 클러스터에 있는 데이터들의 무게중심 값으로 재설정
- 알고리즘을 실행하는 동안 여러 개의 최근접 이웃 검색이 필요하기 때문에 이 알고리즘은 확장성이 높지 않습니다. 알고리즘은 중심 변화가 작을 때 알고리즘은 반복을 중지합니다.



최적의 군집개수를 결정하는 방법 세 가지

- 콜린스키 하라바츠 스코어
- 엘보우 방법
- 실루엣 방법

`KMeans(n_clusters=8, random_state=None)`

- `n_clusters`: 클러스터의 개수

[예제]

iris 데이터로 kmeans 군집분석을 해보자

In [1]:

```
# 필요한 모듈
import pandas as pd
from sklearn.cluster import KMeans

# 데이터 로드
iris = pd.read_csv('../data/iris.csv')
X = iris.drop('target',axis=1)
X
```

Out[1]:

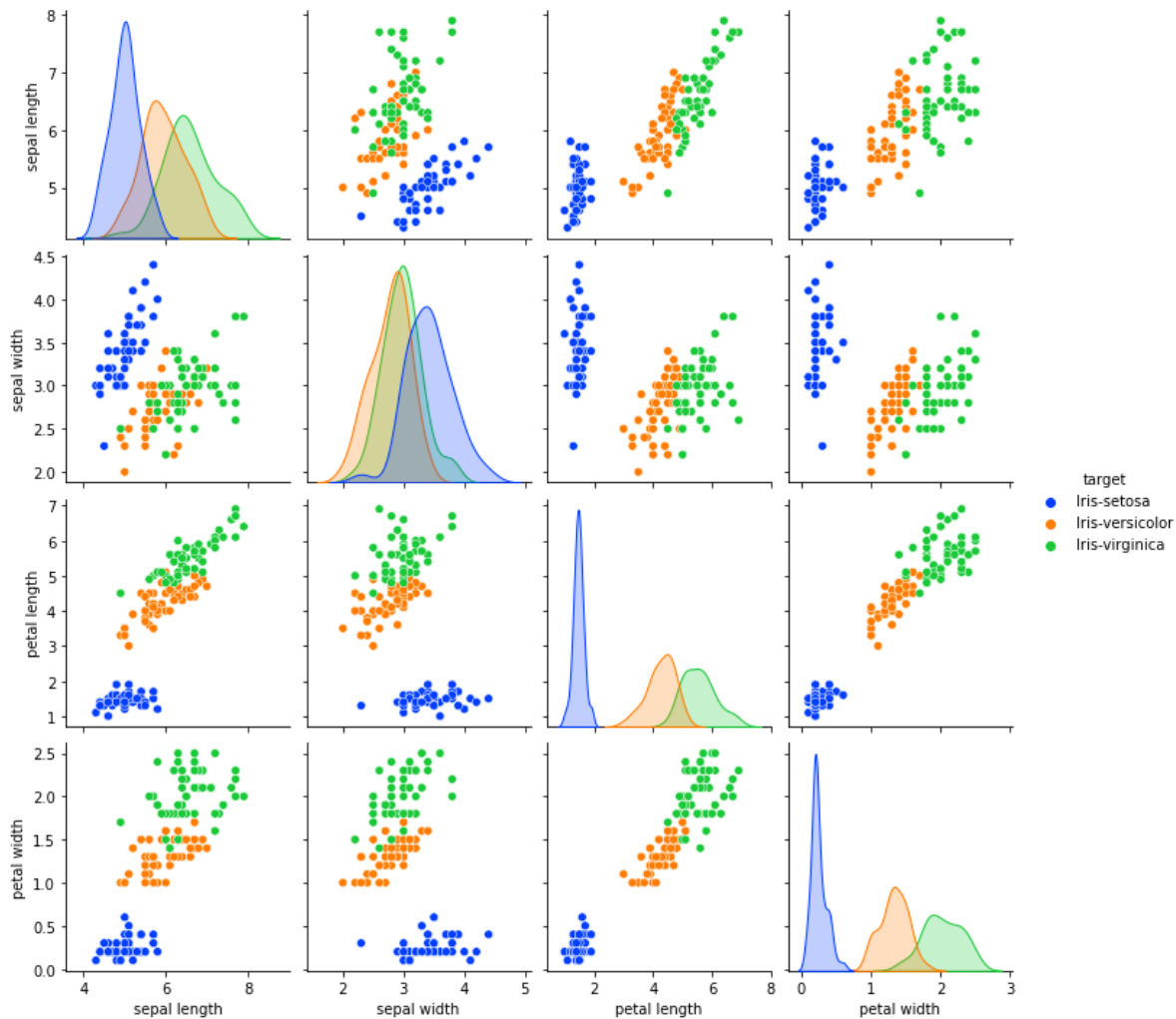
	sepal length	sepal width	petal length	petal width
0	5.1	3.5	1.4	0.2
1	4.9	3.0	1.4	0.2
2	4.7	3.2	1.3	0.2
3	4.6	3.1	1.5	0.2
4	5.0	3.6	1.4	0.2
...	...	...	...	...
145	6.7	3.0	5.2	2.3
146	6.3	2.5	5.0	1.9
147	6.5	3.0	5.2	2.0
148	6.2	3.4	5.4	2.3
149	5.9	3.0	5.1	1.8

150 rows × 4 columns

In [2]:

```
# 원본 데이터 iris target 시각화
import seaborn as sns
import matplotlib.pyplot as plt

sns.pairplot(iris,
             diag_kind='kde',
             hue="target",
#             corner = True, #직각삼각형 모양으로 산점도 행렬 만드려면 주석해제
             palette='bright')
plt.show()
```



In [3]:

```
# K-Means 군집화 실행
km = KMeans(n_clusters = 2, random_state=1)
km.fit(X)
```

Out[3]:

```
KMeans(n_clusters=2, random_state=1)
```

In [4]:

```
km.labels_
```

Out[4]:

```
array([0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 1, 1,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
       1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1])
```

최적의 클러스터 개수를 결정하는데 사용되는 방법 (1)

- Calinski-Harabasz (콜린스키 하라바츠 지수)
 - 모든 클러스터에 대한 클러스터 간 분산과 클러스터 내 분산의 합의 비율
 - 점수가 높을수록 성능이 좋음
 - 장점
 - 계산이 빠름
 - 단점
 - 밀도 기반 클러스터에서 점수가 더 높게 나타나는 경향이 있음

In [5]:

```
# 군집분석 평가척도: 콜린스키 하라바츠
from sklearn.metrics import calinski_harabasz_score
calinski_harabasz_score(X, km.labels_)
```

Out[5]:

```
513.3038433517568
```

In [6]:

```
# 클러스터별 콜린스키 하라바츠 결과 비교
for k in range(2, 10):
    kmeans_model = KMeans(n_clusters=k, random_state=1).fit(X)
    labels = kmeans_model.labels_
    print(calinski_harabasz_score(X, labels))
```

```
513.3038433517568
560.3999242466402
529.1207190840455
494.0943819140986
474.51742958716295
448.8714494416451
436.61302488142496
414.5303174051382
```

- iris 데이터는 군집이 3일 때, 가장 점수가 높음

최적의 클러스터 개수를 결정하는데 사용되는 방법 (2)

- 엘보우 기법
 - 클러스터 내 오차제곱합(SSE)을 클러스터 개수마다 비교
 - 클러스터 개수를 늘려나가면서 계산한 SSE를 비교
 - 어느 순간 줄어드는 비율이 급격하게 작아지는 부분과 기울기가 소실되는 구간이 생김 (=팔꿈치, elbow)
 - 그 부분이 최적의 클러스터 개수임

In [7]:

```
import matplotlib.pyplot as plt
def elbow(X):
    sse=[]
    for i in range(1, 11):
        km=KMeans(n_clusters=i, random_state=1)
        km.fit(X)
        sse.append(km.inertia_)

    plt.plot(range(1,11), sse, marker='o')
    plt.xlabel('The Number of Clusters')
    plt.ylabel('SSE')
    plt.show()
    print(sse)
```

In [8]:

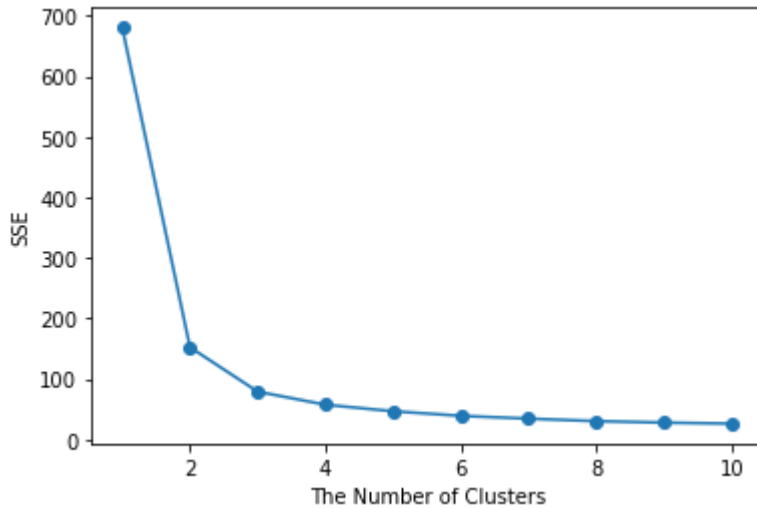
```
km.inertia_
```

Out[8]:

```
152.36870647733915
```

In [9]:

```
elbow(X)
```



[680.8243999999996, 152.36870647733915, 78.94084142614601, 57.34540931571815, 46.535582051282034, 38.95701115711987, 34.32652991452991, 30.227724598930493, 27.766706937799047, 26.07225182334007]

<결과해석>

3에서 4로 갈때, 기울기가 소실됨, 그러므로 군집의 수는 2~3이 적당함

In [10]:

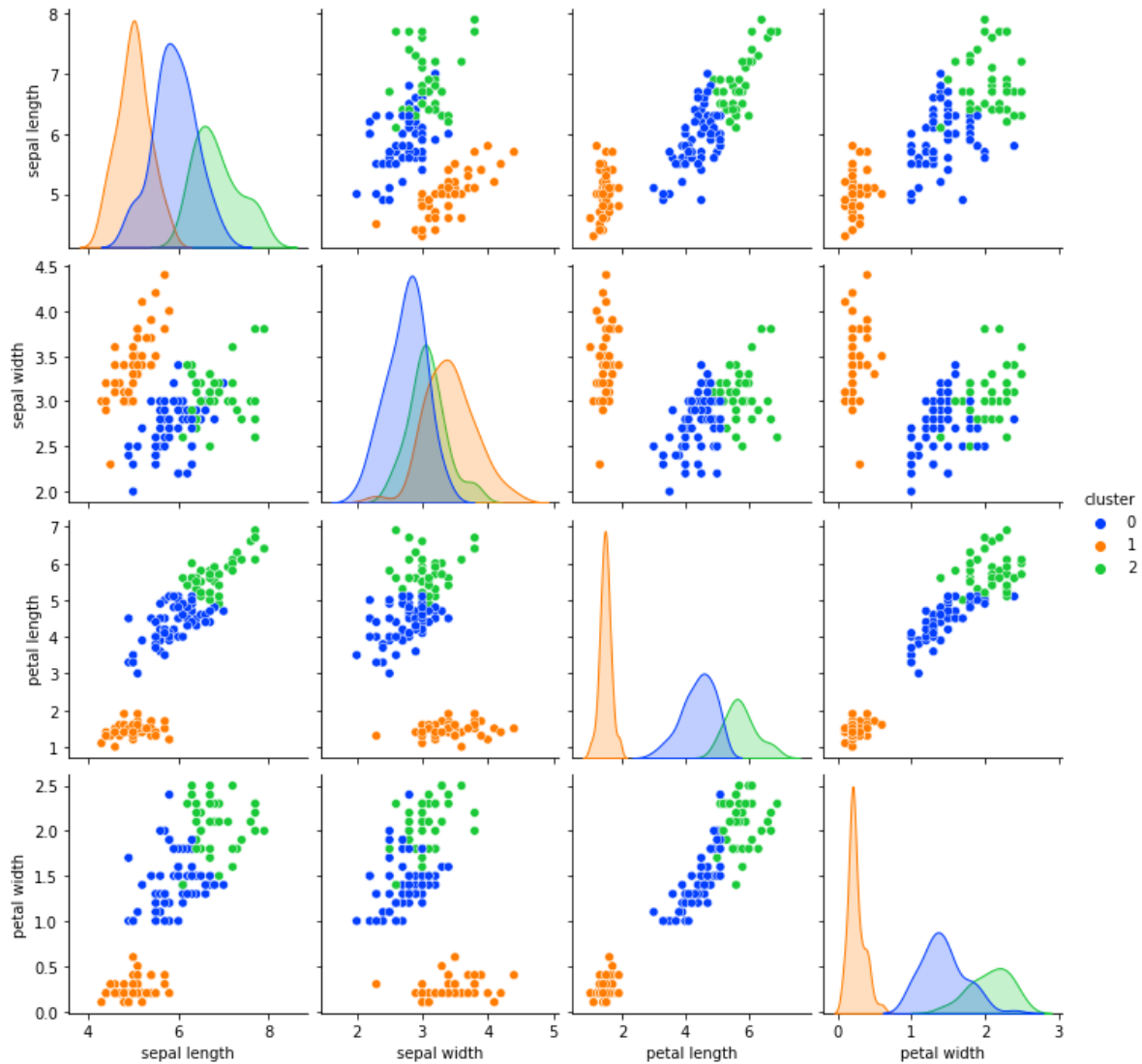
```
#최적의 k로 K-Means 군집화 실행
km = KMeans(n_clusters = 3, random_state=1)
km.fit(X)
```

```
# 할당된 군집을 iris 데이터에 추가
new_labels = km.labels_
iris['cluster'] = new_labels
```

In [11]:

```
# k-means 시각화
# 군집결과 시각화
import seaborn as sns
import matplotlib.pyplot as plt

sns.pairplot(iris,
             diag_kind='kde',
             hue="cluster",
             corner = True, #직각삼각형 모양으로 산점도 행렬 만드려면 주석해제
             palette='bright')
plt.show()
```



In [14]:

```
dbscan_df = X.copy()
```

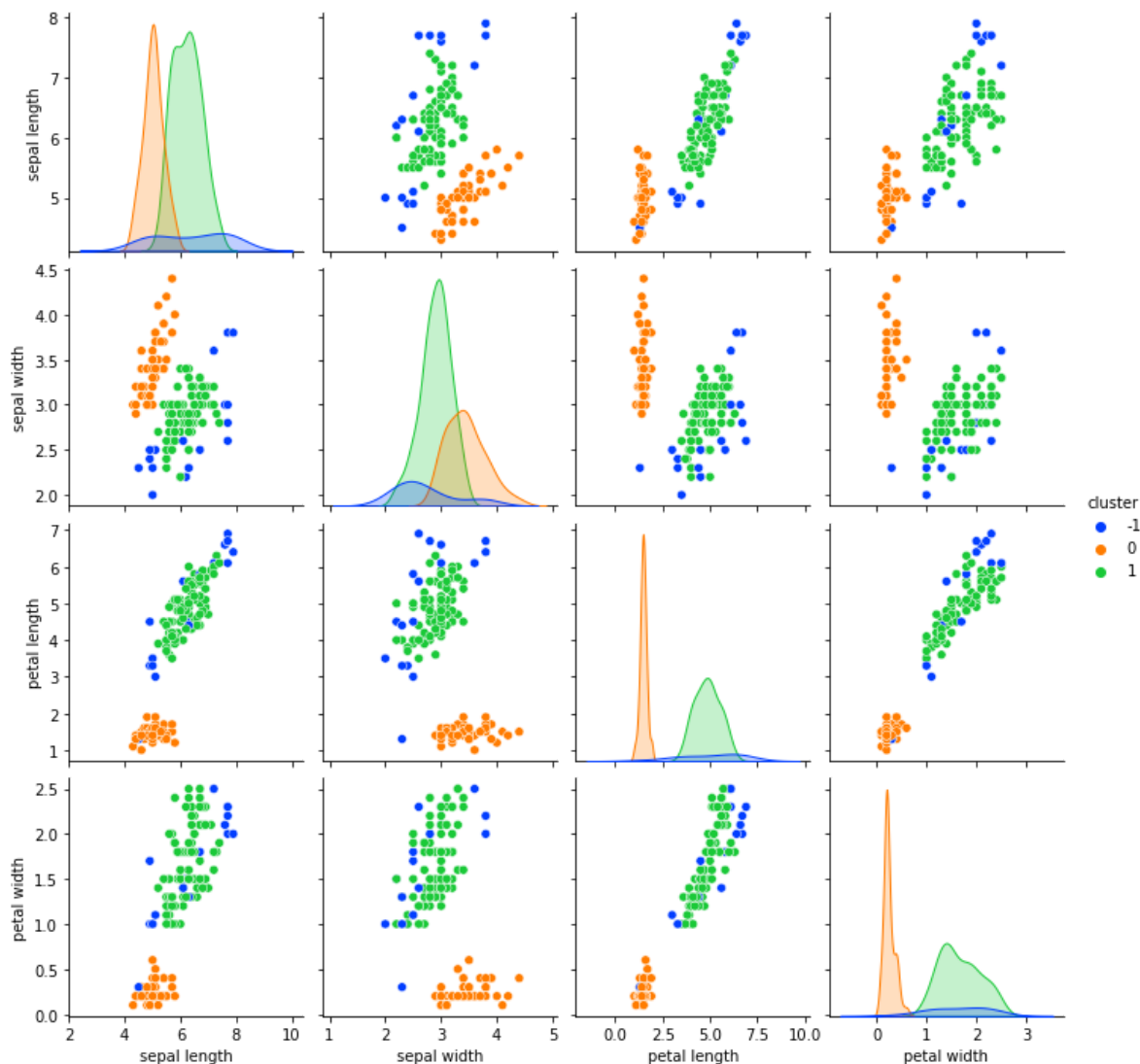
In [15]:

```
dbscan_df['cluster'] = dbscan.labels_
```

In [16]:

```
# dbscan_df 시각화
# 군집결과 시각화
import seaborn as sns
import matplotlib.pyplot as plt

sns.pairplot(dbscan_df,
             diag_kind='kde',
             hue="cluster",
             corner = True, #직각삼각형 모양으로 산점도 행렬 만드려면 주석해제
             palette='bright')
plt.show()
```

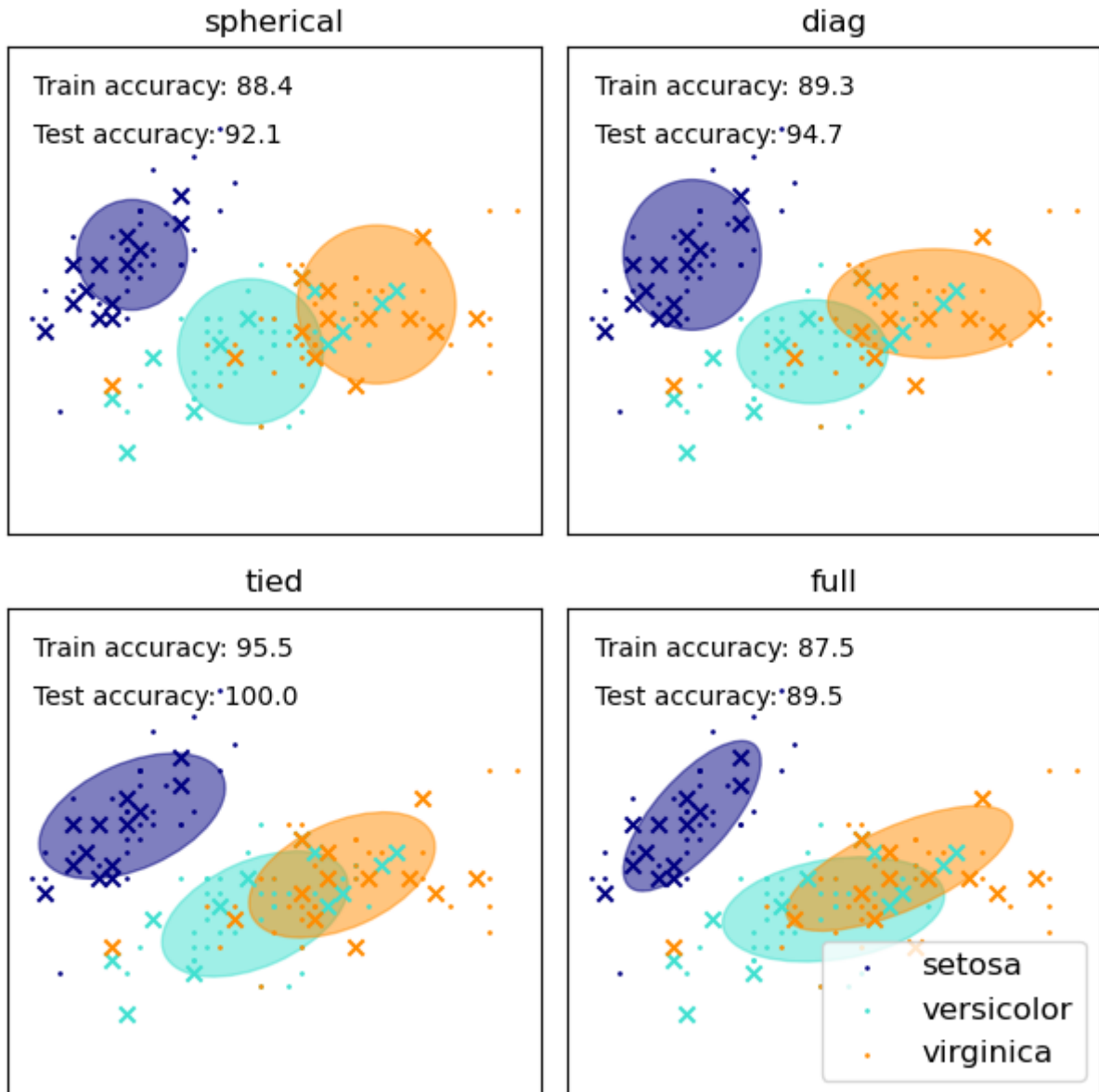


- 위의 결과를 보면, 원본 데이터와 많이 차이가 나는 것을 알 수 있다.
- DBSCAN은 밀도기반 클러스터이기 때문에 위와 같은 데이터 분포의 특징을 잘 반영하지 못한다.

7.4 혼합 분포 군집

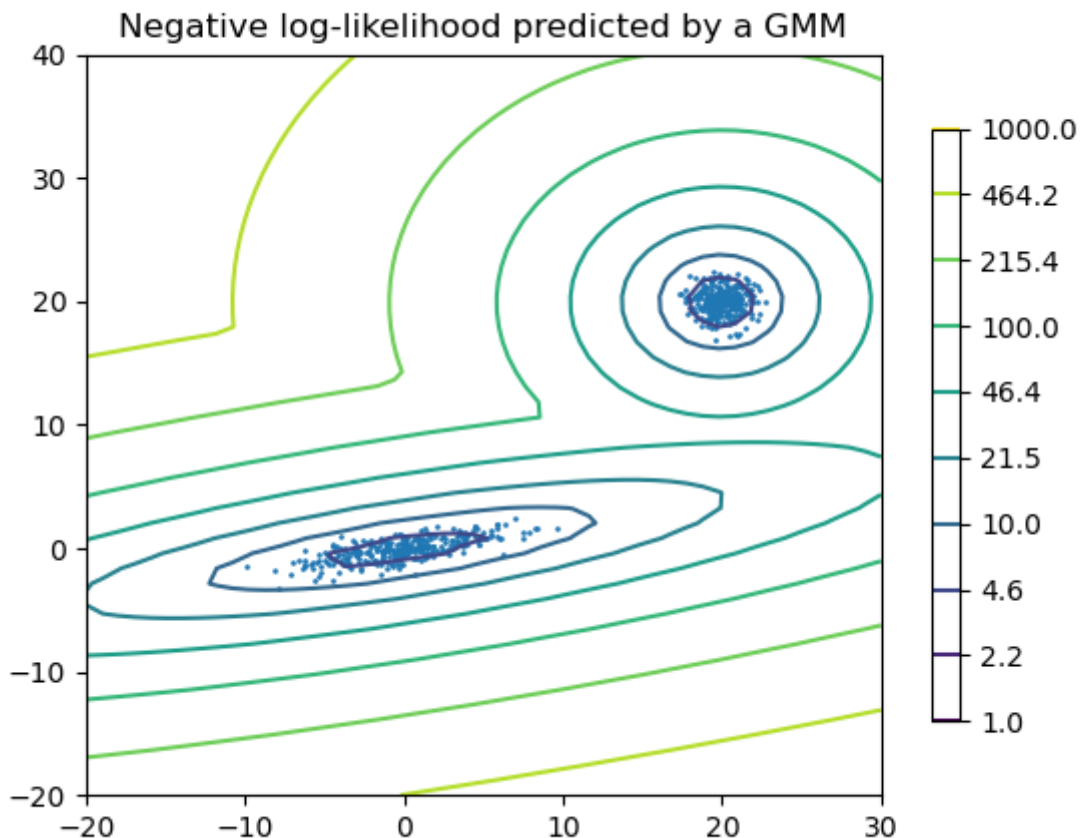
가. 개념

- 모형 기반(model-based) 군집방법
- 데이터가 k 개의 모수적 모형의 가중합으로 표현되는 모집단 모형으로부터 나왔다는 가정
- k 개의 각 모형은 군집이며, 각 데이터는 추정된 k 개의 모형 중 어느 모형으로부터 나왔을 확률이 높은지에 따라 군집의 분류가 만들어짐



나. 혼합분포모형으로 설명할 수 있는 데이터의 형태

- 다봉형 등



다. 사용 알고리즘: EM (Expectation-Maximization)

- 각 자료에 대해 Z 의 조건부분포로부터 조건부 기대값을 구할 수 있음
 - Z 의 조건부분포: 어느 집단에 속할지에 대한 확률분포
- 관측변수 X 와 잠재변수 Z 를 포함하는 (X, Z) 에 대한 로그-가능도함수에 Z 대신 상수값인 Z 의 조건부 기대값을 대입하면, 로그-가능도함수를 최대로 하는 모수를 쉽게 찾을 수 있음
 - 이 추정과정을 "최대가능도추정"이라고 함
- EM 알고리즘의 4단계 정리
 - (1) 초기값 설정: 필요한 모수에 대해 초기값을 선정한다
 - (2) E: 잠재변수 Z 의 기대치 계산 (X 가 특정 군집에 속할 확률을 계산함)
 - (3) M: 잠재변수 Z 의 기대치를 이용해 파라미터를 추정함 (계산된 확률을 통해 모수를 재추정함)
 - (4) 반복 정지: 수렴조건이 만족될 때까지 E와 M을 반복
 - 수렴조건 = 최대가능도(likelihood)가 최대인가?

라. 혼합 분포 군집보형의 특징

- 장점
 - k-means보다 좀 더 통계적으로 엄밀한 결과를 얻을 수 있음 (확률분포 도입)
 - 군집을 몇 개의 모수로 표현할 수 있고, 서로 다른 크기나 모양의 군집을 찾을 수 있음
- 단점
 - 군집의 크기가 너무 작으면 추정의 정도가 떨어짐
 - 데이터가 커지면 EM 알고리즘 적용 시 시간/계산비용 증가 → 사용하기 어려움
 - 이상치자료에 민감하기 때문에 사전에 조치 필요
 - 유형들의 분포가 정규분포와 차이가 크면 결과가 좋지 못함
 - → 그래서 현실에 존재하는 여러 개의 확률 분포를 사용자가 혼합해서 표현함
 - → 몇 개의 확률분포를 혼합할 것인지는 사용자가 지정함. (그 개수에 맞춰서 모델이 만들어짐)

[예제]

iris 데이터의 Species를 제외하고 혼합분포군집분석을 실시하기

In [1]:

```
# 필요한 모듈
import pandas as pd
import numpy as np
import sklearn
from sklearn.preprocessing import StandardScaler
from sklearn.mixture import GaussianMixture
```

In [2]:

```
# 데이터 업로드
iris = pd.read_csv('../data/iris.csv')
df = iris.drop('target',axis=1)
df.head()
```

Out[2]:

	sepal length	sepal width	petal length	petal width
0	5.1	3.5	1.4	0.2
1	4.9	3.0	1.4	0.2
2	4.7	3.2	1.3	0.2
3	4.6	3.1	1.5	0.2
4	5.0	3.6	1.4	0.2

In [3]:

```
scaler = StandardScaler()
```

In [4]:

```
df_scaled = scaler.fit_transform(df)
```

In [5]:

```
df_scaled
```

Out[5]:

```
array([[ -9.00681170e-01,  1.03205722e+00, -1.34127240e+00,
        -1.31297673e+00],
       [-1.14301691e+00, -1.24957601e-01, -1.34127240e+00,
        -1.31297673e+00],
       [-1.38535265e+00,  3.37848329e-01, -1.39813811e+00,
        -1.31297673e+00],
       [-1.50652052e+00,  1.06445364e-01, -1.28440670e+00,
        -1.31297673e+00],
       [-1.02184904e+00,  1.26346019e+00, -1.34127240e+00,
        -1.31297673e+00],
       [-5.37177559e-01,  1.95766909e+00, -1.17067529e+00,
        -1.05003079e+00],
       [-1.50652052e+00,  8.00654259e-01, -1.34127240e+00,
        -1.18150376e+00],
       [-1.02184904e+00,  8.00654259e-01, -1.28440670e+00,
        -1.31297673e+00],
       [-1.74885626e+00, -3.56360566e-01, -1.34127240e+00,
        -1.31297673e+00]].
```

In [6]:

```
# 가우시안 혼합모델 구축
gmm = GaussianMixture(n_components = 3)
gmm.fit(df_scaled)
gmm_labels = gmm.predict(df_scaled)
```

In [7]:

```
gmm_labels
```

Out[7]:

```
array([1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
       1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 2, 0, 2, 0, 2, 0, 0, 0, 0, 2, 0, 0, 0, 0, 0, 2, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2,
       2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2,
       2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2], dtype=int64)
```

In [8]:

```
df['gmm_cluster'] = gmm_labels
df['gmm_cluster'].value_counts()
```

Out[8]:

```
2    55
1    50
0    45
Name: gmm_cluster, dtype: int64
```

In [9]:

```
df.head()
```

Out[9]:

	sepal length	sepal width	petal length	petal width	gmm_cluster
0	5.1	3.5	1.4	0.2	1
1	4.9	3.0	1.4	0.2	1
2	4.7	3.2	1.3	0.2	1
3	4.6	3.1	1.5	0.2	1
4	5.0	3.6	1.4	0.2	1

In [10]:

```
# 군집의 변수별 통계량 확인  
clusters = [0, 1, 2]  
df.groupby('gmm_cluster').mean()
```

Out[10]:

	sepal length	sepal width	petal length	petal width
gmm_cluster				
0	5.904444	2.775556	4.193333	1.293333
1	5.006000	3.418000	1.464000	0.244000
2	6.554545	2.950909	5.489091	1.989091

In [11]:

```
df.groupby('gmm_cluster').var()
```

Out[11]:

	sepal length	sepal width	petal length	petal width
gmm_cluster				
0	0.277707	0.095071	0.195636	0.031091
1	0.124249	0.145180	0.030106	0.011494
2	0.385488	0.111434	0.320620	0.083582

In [12]:

```
# 군집결과 시각화
```

```
import seaborn as sns
```

```
import matplotlib.pyplot as plt
```

```
sns.pairplot(df,
```

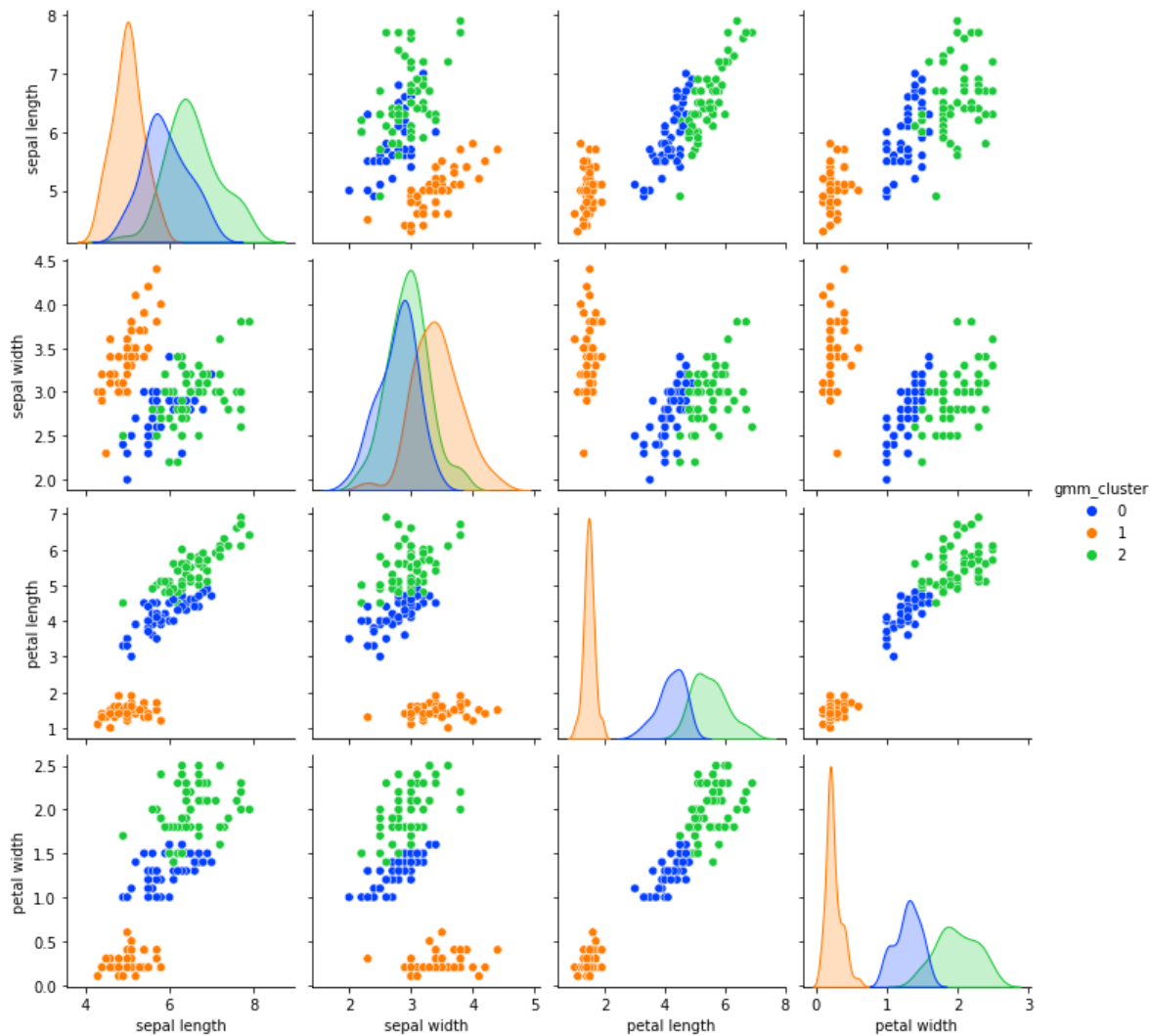
```
    diag_kind='kde',
```

```
    hue="gmm_cluster",
```

```
#    corner = True, #직각삼각형 모양으로 산점도 행렬 만드려면 주석해제
```

```
    palette='bright')
```

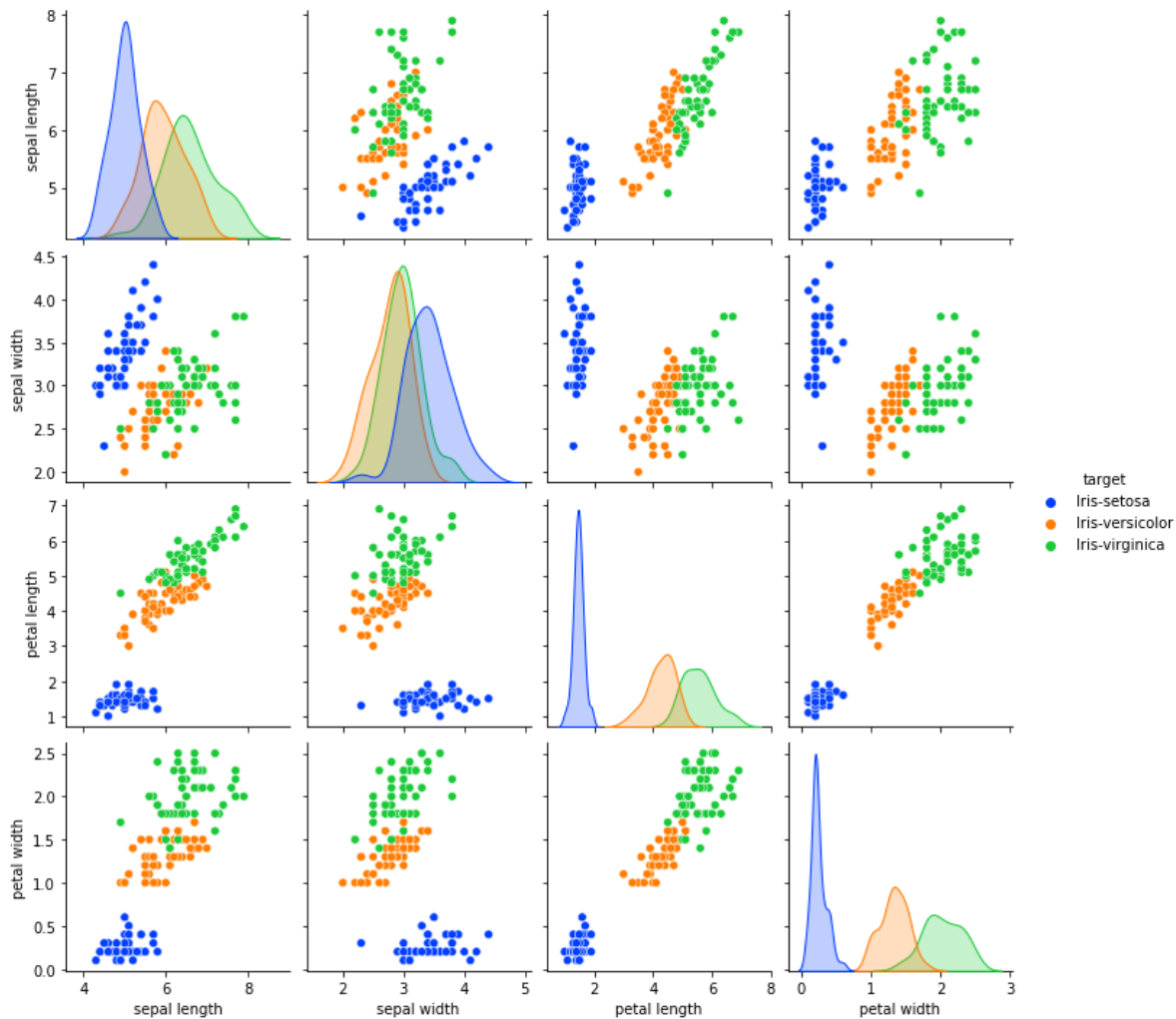
```
plt.show())
```



In [13]:

```
# 원본 target으로 시각화
# 군집결과 시각화
import seaborn as sns
import matplotlib.pyplot as plt

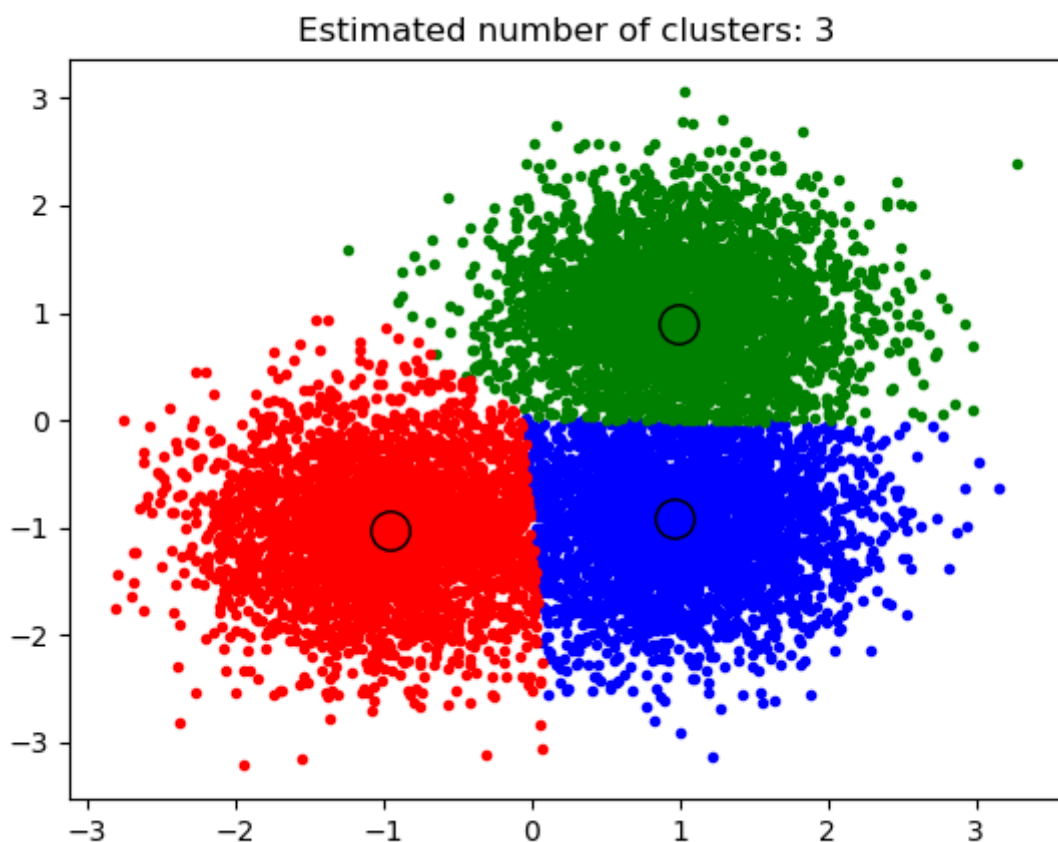
sns.pairplot(iris,
             diag_kind='kde',
             hue="target",
#             corner = True, #직각삼각형 모양으로 산점도 행렬 만드려면 주석해제
             palette='bright')
plt.show()
```



7.5 자기조직화지도(Self-Organizing Map, SOM)

1. 개념

- 대뇌피질과 시각피질의 학습 과정을 모델화한 인공신경망으로써 자율 학습에 의한 클러스터링을 수행하는 알고리즘이다.
- SOM 은 우선 Map size 를 설정해줍니다. (* 군집분석으로 SOM을 사용할 때에는 군집의 수를 차원의 수로 맞추어 주어야 합니다.)
- 이제 이 Default Map 을 조금씩 수정해서(학습해서 = fit) 데이터 분포 형태에 Map 을 근사시킵니다.



7.5.1 예제 : iris_data를 som으로 군집분석 해보세요.

In [1]:

```
# 데이터 로드
import pandas as pd
import numpy as np
iris = pd.read_csv('../data/iris.csv')
iris
```

Out[1]:

	sepal length	sepal width	petal length	petal width	target
0	5.1	3.5	1.4	0.2	Iris-setosa
1	4.9	3.0	1.4	0.2	Iris-setosa
2	4.7	3.2	1.3	0.2	Iris-setosa
3	4.6	3.1	1.5	0.2	Iris-setosa
4	5.0	3.6	1.4	0.2	Iris-setosa
...	...	...	...	...	...
145	6.7	3.0	5.2	2.3	Iris-virginica
146	6.3	2.5	5.0	1.9	Iris-virginica
147	6.5	3.0	5.2	2.0	Iris-virginica
148	6.2	3.4	5.4	2.3	Iris-virginica

In [2]:

```
y = iris['target']
X = iris.drop('target',axis=1)
```

In [3]:

```
# Feature Scaling
from sklearn.preprocessing import MinMaxScaler
scaler = MinMaxScaler(feature_range = (0, 1))
X = scaler.fit_transform(X)
```

In [4]:

```
X
```

Out[4]:

```
array([[0.22222222, 0.625      , 0.06779661, 0.04166667],
       [0.16666667, 0.41666667, 0.06779661, 0.04166667],
       [0.11111111, 0.5        , 0.05084746, 0.04166667],
       [0.08333333, 0.45833333, 0.08474576, 0.04166667],
       [0.19444444, 0.66666667, 0.06779661, 0.04166667],
       [0.30555556, 0.79166667, 0.11864407, 0.125      ],
       [0.08333333, 0.58333333, 0.06779661, 0.08333333],
       [0.19444444, 0.58333333, 0.08474576, 0.04166667],
       [0.02777778, 0.375      , 0.06779661, 0.04166667],
       [0.16666667, 0.45833333, 0.08474576, 0.        ],
       [0.30555556, 0.70833333, 0.08474576, 0.04166667],
       [0.13888889, 0.58333333, 0.10169492, 0.04166667],
       [0.13888889, 0.41666667, 0.06779661, 0.        ],
       [0.        , 0.41666667, 0.01694915, 0.        ],
       [0.41666667, 0.83333333, 0.03389831, 0.04166667],
       [0.38888889, 1.        , 0.08474576, 0.125      ],
       [0.30555556, 0.79166667, 0.05084746, 0.125      ],
       [0.22222222, 0.625      , 0.06779661, 0.08333333],
```

In [5]:

```
# sklearn의 모든 분류기와 마찬가지로 SOM 인스턴스를 빌드하고 SOM .fit()에 맞게 데이터 호출
# Iris Dataset에 3개의 클래스가 있다는 것을 이미 알고 있으므로
# 자체 구성 맵에 3 x 1 구조를 사용하지만 실제로는 데이터에 가장 적합한 것을 찾기 위해 다른 구조를
# 시도해 봐야함
```

```
# !pip install sklearn-som
from sklearn_som.som import SOM
```

In [6]:

```
iris_som = SOM(m=3, n=1, dim=4)
iris_som.fit(X, epochs=100)
```

In [7]:

```
predictions= iris_som.predict(X)
predictions
```

Out[7]:

```
array([0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 2, 2, 2, 1, 2, 1, 2, 1, 2, 1, 1, 1, 1, 2, 1, 2,
       1, 1, 2, 1, 2, 1, 2, 1, 1, 2, 2, 2, 1, 1, 1, 1, 1, 2, 1, 2, 2, 1,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 2, 2, 2, 2, 2, 2, 1, 2, 2, 2,
       2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2,
       2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2], dtype=int64)
```

In [8]:

```
iris["cluster"] = predictions
```

In [9]:

```
iris
```

Out[9]:

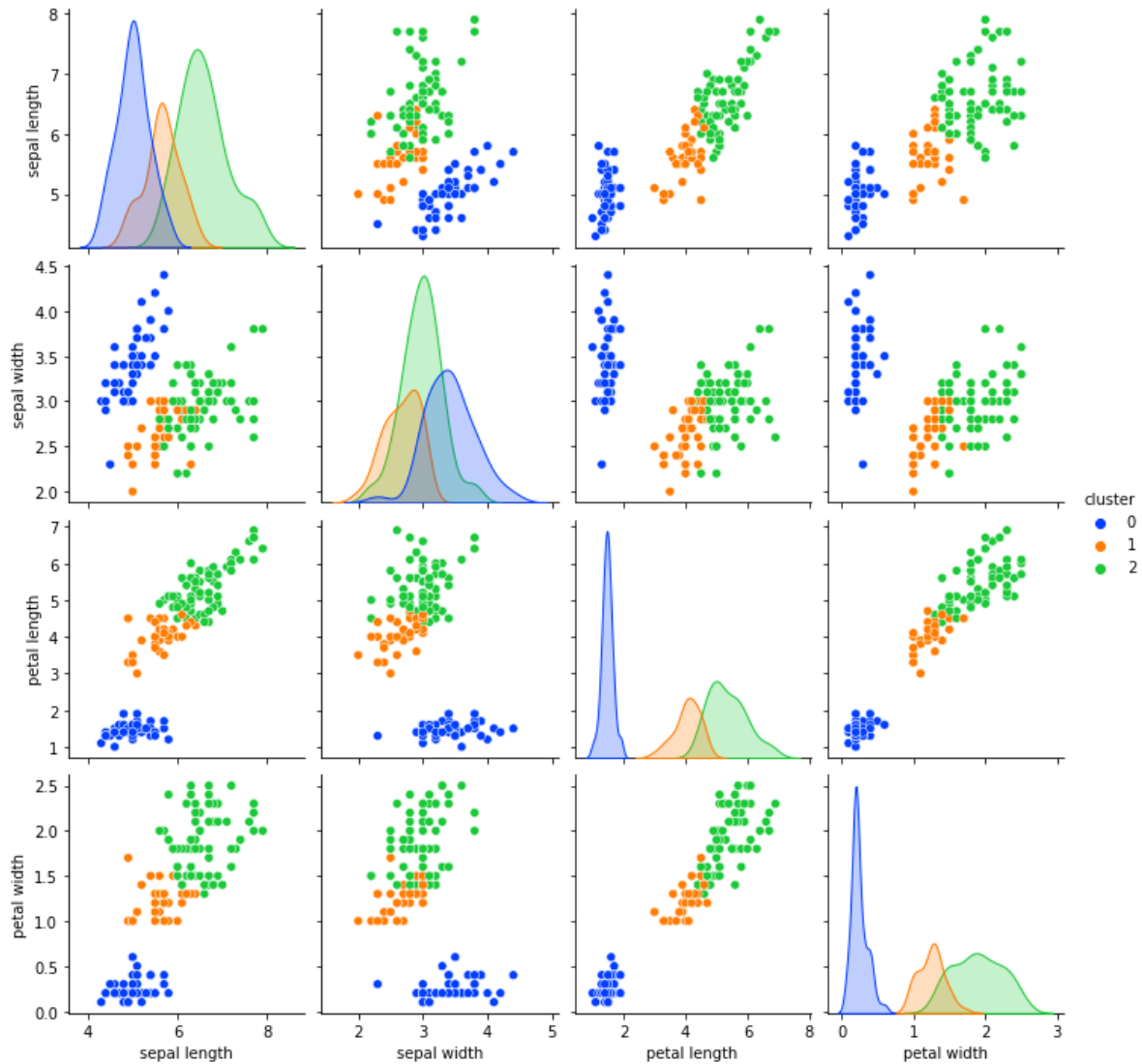
	sepal length	sepal width	petal length	petal width	target	cluster
0	5.1	3.5	1.4	0.2	Iris-setosa	0
1	4.9	3.0	1.4	0.2	Iris-setosa	0
2	4.7	3.2	1.3	0.2	Iris-setosa	0
3	4.6	3.1	1.5	0.2	Iris-setosa	0
4	5.0	3.6	1.4	0.2	Iris-setosa	0
...	...	...	...	...	...	...
145	6.7	3.0	5.2	2.3	Iris-virginica	2
146	6.3	2.5	5.0	1.9	Iris-virginica	2
147	6.5	3.0	5.2	2.0	Iris-virginica	2
148	6.2	3.4	5.4	2.3	Iris-virginica	2
149	5.9	3.0	5.1	1.8	Iris-virginica	2

150 rows × 6 columns

In [10]:

```
# 원본 target으로 시각화
# 군집결과 시각화
import seaborn as sns
import matplotlib.pyplot as plt

sns.pairplot(iris,
             diag_kind='kde',
             hue="cluster",
             corner = True, #직각삼각형 모양으로 산점도 행렬 만드려면 주석해제
             palette='bright')
plt.show()
```



7.5.2 17회 기출

- 데이터를 7:3으로 나누어라
- train 데이터셋을 som 알고리즘으로 군집화하고 test데이터에 대한 오분류표를 구하라

In [11]:

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X,y, stratify=y, test_size=0.3,
random_state=2021)
```

In [12]:

```
som = SOM(m=3, n=1, dim=4)
som.fit(X_train)
```

In [13]:

```
y_pred = som.predict(X_test)
```

In [14]:

```
result = pd.DataFrame()  
result['y_test'] = y_test  
result['y_pred'] = y_pred  
result
```

Out[14]:

	y_test	y_pred
6	Iris-setosa	2
148	Iris-virginica	0
89	Iris-versicolor	1
116	Iris-virginica	0
12	Iris-setosa	2
33	Iris-setosa	2
67	Iris-versicolor	1
119	Iris-virginica	0
30	Iris-setosa	2
59	Iris-versicolor	1

In [15]:

```
result.loc[result['y_test']=='Iris-setosa','target_num'] = 0  
result.loc[result['y_test']=='Iris-virginica','target_num'] = 2  
result.loc[result['y_test']=='Iris-versicolor','target_num'] = 1
```

In [16]:

```
result
```

Out[16]:

	y_test	y_pred	target_num
6	Iris-setosa	2	0.0
148	Iris-virginica	0	2.0
89	Iris-versicolor	1	1.0
116	Iris-virginica	0	2.0
12	Iris-setosa	2	0.0
33	Iris-setosa	2	0.0
67	Iris-versicolor	1	1.0
119	Iris-virginica	0	2.0
30	Iris-setosa	2	0.0
59	Iris-versicolor	1	1.0

In [17]:

```
from sklearn.metrics import accuracy_score, confusion_matrix, roc_auc_score,
classification_report
```

In [18]:

```
accuracy_score(result['target_num'], result['y_pred'])
```

Out[18]:

0.24444444444444444

In [19]:

```
confusion_matrix(result['target_num'], result['y_pred'])
```

Out[19]:

```
array([[ 0,  0, 15],
       [ 4, 11,  0],
       [15,  0,  0]], dtype=int64)
```

In [20]:

```
print(classification_report(result['target_num'], result['y_pred']))
```

	precision	recall	f1-score	support
0.0	0.00	0.00	0.00	15
1.0	1.00	0.73	0.85	15
2.0	0.00	0.00	0.00	15
accuracy			0.24	45
macro avg	0.33	0.24	0.28	45
weighted avg	0.33	0.24	0.28	45

8 연관분석

8.1 Run-Test

- 연속적인 관측값들이 임의적으로 나타난 것인지를 검정하는 방법
- run-test의 귀무가설 : 연속적인 관측값이 임의적이다.
- run-test의 대립가설 : 연속적인 관측값이 임의적이 아니다 (즉, 연관이 있다)

< 22회 기출 >

- 상품 a와 b가 있을 때 다음과 같은 구매 패턴이 있다고 한다.
['b','b','b','b','b','b','a','a','a','a','a','b','b','a','a','a','a','b','b','b']
두 상품의 구매패턴이 연관성이 있는지 검정해라

In [1]:

```
import pandas as pd

data = ['a','a','b','b','a','a','a','a','b','b','b','b','b','a','a','b','b','a','b','b']
```

In [2]:

```
test_df = pd.DataFrame(data,columns=["product"])
test_df
```

Out[2]:

	product
0	a
1	a
2	b
3	b
4	a
5	a
6	a
7	a
8	b
9	b

In [3]:

```
test_df.loc[test_df['product']=='a','product'] = 1
test_df.loc[test_df['product']=='b','product'] = 0
```

In [4]:

```
test_df['product']
```

Out[4]:

```
0    1
1    1
2    0
3    0
4    1
5    1
6    1
7    1
8    0
9    0
10   0
11   0
12   0
13   1
14   1
15   0
16   0
17   1
```

In [8]:

```
from statsmodels.sandbox.stats.runs import runstest_1samp
help(runstest_1samp)
```

Help on function runstest_1samp in module statsmodels.sandbox.stats.runs:

```
runstest_1samp(x, cutoff='mean', correction=True)
    use runs test on binary discretized data above/below cutoff
```

Parameters

x : array_like

data, numeric

cutoff : {'mean', 'median'} or number

This specifies the cutoff to split the data into large and small values.

correction : bool

Following the SAS manual, for samplesize below 50, the test statistic is corrected by 0.5. This can be turned off with correction=False, and was included to match R, tseries, which does not use any correction.

Returns

In [9]:

```
from statsmodels.sandbox.stats.runs import runstest_1samp

#Perform Runs test
# - run-test의 귀무가설 : 연속적인 관측값이 임의적이다.
# - run-test의 대립가설 : 연속적인 관측값이 임의적이 아니다

runstest_1samp(test_df['product'])
```

Out[9]:

(-1.1144881152070183, 0.26506984027306035)

- 분석결과 : 유의수준 5% 하에서 p-value값이 0.05보다 크므로, 상품 a와 b의 구매에는 연관이 없다.

8.2 연관분석

8.2.1 연관 분석(Association analysis)이란?

효율적인 상품 진열, 패키지 상품 개발, 교차판매 전략, 기획상품의 결정 등에 사용되고 있음

8.2.2 연관규칙분석

- 기업의 데이터베이스에서 상품 구매, 서비스 등 일련의 거래 또는 사건들 간의 규칙을 발견하기 위해 적용됨
- 서로 다른 두 아이템 집합이 얼마나 빈번하게 발생하는지를 알 수 있게 함
- 조건과 반응 형태(IF_THEN)로 이루어져 있음
- 장바구니 분석, 서열분석이 있음
 - 장바구니 분석 : 장바구니에 무엇이 같이 들어있는지에 대한 분석
 - 서열 분석 : A를 산 다음에 B를 산다

8.2.2.1 연관규칙 분석에 사용되는 척도

- 지지도 : 전체 거래 중 항목 A와 항목 B를 동시에 포함하는 거래의 비율로 정의
 $\text{support} = \text{A와 B가 동시에 포함된 거래수} / \text{전체 거래수}$
- 신뢰도 : 항목 A를 포함한 거래 중에서 항목 A와 항목 B가 같이 포함될 확률로 연관성의 정도 파악 가능
 $\text{confident} = \text{A,B가 동시에 포함된 거래수} / \text{A를 포함하는 거래수}$
- 향상도 : A가 구매되지 않았을 때 품목 B의 구매확률에 비해 A가 구매되었을 때 품목 B의 구매확률의 증가비
연관규칙 $A \rightarrow B$ 는 품목 A와 품목 B의 구매가 서로 관련이 없는 경우에는 향상도가 1이 됨
 $\text{Lift} = \text{A와 B가 동시에 포함된 거래수} / \text{A를 포함하는 거래수} * \text{B를 포함하는 거래수}$

8.2.3 Apriori 알고리즘

가능한 모든 경우의 수를 탐색하여 지지도, 신뢰도, 향상도가 높은 규칙들을 찾아내는 방식이 가장 이상적이지만 아이템 수가 증가할수록 계산에 소요되는 시간이 기하급수적으로 증가하는 문제가 있음

아이템이 n 개일 때 탐색해야할 모든 경우의 수 : $n*(n-1)$

이 때문에 빈발 집합(frequent item sets)만을 고려해서 연관 규칙을 생성하는 Apriori algorithm이 제안됨

- 최소지지도보다 큰 지지도 값을 갖는 품목의 집합을 빈발항목집단(frequent item set)이라고 함
- Apriori 알고리즘은 모든 품목집합에 대한 지지도를 전부 계산하는 것이 아니라, 최소 지지도 이상의 빈발항목집합을 찾은 후 그것들에 대해서만 연관규칙을 계산하는 것임
- 장점 : 1세대 알고리즘으로 구현과 이해가 쉬움
- 단점 : 지지도가 낮은 후보 집합을 생성 시 아이템의 개수가 많아지면 계산 복잡도가 증가함

$\{A,B\}$ 는 $\{A\}$, $\{B\}$ 의 초월집합(superset)이라고 함

만일 아이템 집합 $\{A,B\}$ 의 지지도가 사용자가 정한 최소 지지도 요건을 충족시키지 못했을 경우

$\{A,B\}$ 는 물론 $\{A,B\}$ 의 초월집합인 $\{A,B,C\}$, $\{A,B,D\}$ 등 8가지 경우의 수를 계산에서 제외하여 계산효율성을 달성

8.3 apriori 함수에 들어갈 트랜잭션 데이터를 만드는 함수

```
from mlxtend.preprocessing import TransactionEncoder
te = TransactionEncoder()
te_result = te.fit(dataset).transform(dataset)
```

In [1]:

```
# # ## 설치 (시험장 설치버전 : 0.17.0)
# !conda config --add channels conda-forge
# !conda install mlxtend==0.17.0
# !pip install mlxtend==0.17.0
```

[예제]

dataset을 트랜잭션(거래) 형태로 변경하고 결과를 확인하라

In [2]:

```
import pandas as pd
from mlxtend.preprocessing import TransactionEncoder

dataset = [['Apple', 'Beer', 'Rice', 'Chicken'],
           ['Apple', 'Beer', 'Rice'],
           ['Apple', 'Beer'],
           ['Apple', 'Bananas'],
           ['Milk', 'Beer', 'Rice', 'Chicken'],
           ['Milk', 'Beer', 'Rice'],
           ['Milk', 'Beer'],
           ['Apple', 'Bananas']]

te = TransactionEncoder()
te_ary = te.fit_transform(dataset)

print(te.columns_)
te_ary
```

```
['Apple', 'Bananas', 'Beer', 'Chicken', 'Milk', 'Rice']
```

Out[2]:

```
array([[ True, False,  True,  True, False,  True],
       [ True, False,  True, False, False,  True],
       [ True, False,  True, False, False, False],
       [ True,  True, False, False, False, False],
       [False, False,  True,  True,  True,  True],
       [False, False,  True, False,  True,  True],
       [False, False,  True, False,  True, False],
       [ True,  True, False, False, False, False]])
```

In [3]:

```
df = pd.DataFrame(te_ary, columns = te.columns_)  
df
```

Out[3]:

	Apple	Bananas	Beer	Chicken	Milk	Rice
0	True	False	True	True	False	True
1	True	False	True	False	False	True
2	True	False	True	False	False	False
3	True	True	False	False	False	False
4	False	False	True	True	True	True
5	False	False	True	False	True	True
6	False	False	True	False	True	False
7	True	True	False	False	False	False

8.3.0.0.1 mlxtend의 apriori 함수

One-Hot 형식의 DataFrame에서 빈번 항목 집합을 출력하는 함수

```
from mlxtend.frequent_patterns import apriori  
apriori(df)
```

- df : 값이 0/1 또는 True / False인 pandas DataFrame
- min_support : 최소 지지도 (default = 0.5)
- use_colnames : True이면 열 인덱스 대신 반환된 DataFrame의 열 이름을 사용 (default = False)
- max_len : 생성된 항목 세트의 최대 길이로 None인 경우 가능한 모든 항목 세트 길이를 평가 (default = None)
- low_memory : 메모리 리소스가 제한된 경우에 사용할 수 있으나 속도가 기존의 3~6배 느림 (default = False)

In [4]:

```
## 지지도 60 % 이상인 품목만 추출

from mlxtend.frequent_patterns import apriori

apriori(df, min_support=0.6)
```

Out[4]:

	support	itemsets
0	0.625	(0)
1	0.750	(2)

In [5]:

```
apriori(df, min_support=0.6, use_colnames=True)
```

Out[5]:

	support	itemsets
0	0.625	(Apple)
1	0.750	(Beer)

In [6]:

```
## 규칙의 길이를 추출하는 방법

frequent_itemsets = apriori(df, min_support=0.3, use_colnames=True)
frequent_itemsets['length'] = frequent_itemsets['itemsets'].apply(lambda x: len(x))
frequent_itemsets
```

Out[6]:

	support	itemsets	length
0	0.625	(Apple)	1
1	0.750	(Beer)	1
2	0.375	(Milk)	1
3	0.500	(Rice)	1
4	0.375	(Apple, Beer)	2
5	0.375	(Milk, Beer)	2
6	0.500	(Rice, Beer)	2

In [7]:

```
frequent_itemsets['itemsets'][0]
```

Out[7]:

```
frozenset({'Apple'})
```

In [8]:

```
len(frequent_itemsets['itemsets'][0])
```

Out[8]:

1

[예제]

Groceries 데이터셋으로 연관규칙분석을 실시하라

In [9]:

```
df= pd.read_csv('../data/groceries.csv',)
df
```

Out[9]:

	citrus fruit,semi-finished bread,margarine,ready soups
0	tropical fruit,yogurt,coffee
1	whole milk
2	pip fruit,yogurt,cream cheese,meat spreads
3	other vegetables,whole milk,condensed milk,lon...
4	whole milk,butter,yogurt,rice,abrasive cleaner
...	...
9829	sausage,chicken,beef,hamburger meat,citrus fru...
9830	cooking chocolate
9831	chicken,citrus fruit,other vegetables,butter,y...
9832	semi-finished bread,bottled water,soda,bottled...

In [10]:

```
df.loc[9834, "citrus fruit,semi-finished bread,margarine,ready soups"] = df.columns.values[0]
df
```

Out[10]:

citrus fruit,semi-finished bread,margarine,ready soups	
0	tropical fruit,yogurt,coffee
1	whole milk
2	pip fruit,yogurt,cream cheese,meat spreads
3	other vegetables,whole milk,condensed milk,lon...
4	whole milk,butter,yogurt,rice,abrasive cleaner
...	...
9830	cooking chocolate
9831	chicken,citrus fruit,other vegetables,butter,y...
9832	semi-finished bread,bottled water,soda,bottled...
9833	chicken,tropical fruit,other vegetables,vinega...
9834	citrus fruit,semi-finished bread,margarine,rea...

9835 rows × 1 columns

In [11]:

```
df_split = df.iloc[:,0].str.split(' ',expand=True)
df_split
```

Out[11]:

	0	1	2	3	4	5	6	7	8	
0	tropical fruit	yogurt	coffee	None	None	None	None	None	None	1
1	whole milk	None	None	None	None	None	None	None	None	1
2	pip fruit	yogurt	cream cheese	meat spreads	None	None	None	None	None	1
3	other vegetables	whole milk	condensed milk	long life bakery product	None	None	None	None	None	1
4	whole milk	butter	yogurt	rice	abrasive cleaner	None	None	None	None	1
...	...	...	...	...	...	...	...	...	...	
9830	cooking chocolate	None	None	None	None	None	None	None	None	1
9831	chicken	citrus fruit	other vegetables	butter	yogurt	frozen dessert	domestic eggs	rolls/buns	rum	film/
9832	semi-finished bread	bottled water	soda	bottled beer	None	None	None	None	None	1
9833	chicken	tropical fruit	other vegetables	vinegar	shopping bags	None	None	None	None	1
9834	citrus fruit	semi-finished bread	margarine	ready soups	None	None	None	None	None	1

9835 rows × 32 columns

In [12]:

```
df_split.values
```

Out[12]:

```
array([[ 'tropical fruit', 'yogurt', 'coffee', ..., None, None, None],
      [ 'whole milk', None, None, ..., None, None, None],
      [ 'pip fruit', 'yogurt', 'cream cheese', ..., None, None, None],
      ...,
      [ 'semi-finished bread', 'bottled water', 'soda', ..., None, None,
        None],
      [ 'chicken', 'tropical fruit', 'other vegetables', ..., None, None,
        None],
      [ 'citrus fruit', 'semi-finished bread', 'margarine', ..., None,
        None, None]], dtype=object)
```

In [13]:

```
df_split_ary = df_split.values
```

In [14]:

```
list(filter(None,df_split_ary[0]))
```

Out[14]:

```
['tropical fruit', 'yogurt', 'coffee']
```

In [15]:

```
groceries = []  
for i in range(len(df_split_ary)) :  
    temp = list(filter(None,df_split_ary[i]))  
    groceries.append(temp)
```

In [16]:

```
groceries
```

Out[16]:

```
[['tropical fruit', 'yogurt', 'coffee'],  
 ['whole milk'],  
 ['pip fruit', 'yogurt', 'cream cheese', 'meat spreads'],  
 ['other vegetables',  
  'whole milk',  
  'condensed milk',  
  'long life bakery product'],  
 ['whole milk', 'butter', 'yogurt', 'rice', 'abrasive cleaner'],  
 ['rolls/buns'],  
 ['other vegetables',  
  'UHT-milk',  
  'rolls/buns',  
  'bottled beer',  
  'liquor (appetizer)'],  
 ['potted plants'],  
 ['whole milk', 'cereals'],  
 ['tropical fruit',  
  'other vegetables']
```

In [17]:

```
from mlxtend.preprocessing import TransactionEncoder
import pandas as pd

te = TransactionEncoder()
groceries_tr = te.fit(groceries).transform(groceries)
groceries_tr = pd.DataFrame(groceries_tr, columns=te.columns_)
groceries_tr
```

Out[17]:

	Instant food products	UHT- milk	abrasive cleaner	artif. sweetener	baby cosmetics	baby food	bags	baking powder	bathroom cleaner	beef
0	False	False	False	False	False	False	False	False	False	False
1	False	False	False	False	False	False	False	False	False	False
2	False	False	False	False	False	False	False	False	False	False
3	False	False	False	False	False	False	False	False	False	False
4	False	False	True	False	False	False	False	False	False	False
...	...	...	...	...	...	...	...	...	...	...
9830	False	False	False	False	False	False	False	False	False	False
9831	False	False	False	False	False	False	False	False	False	False
9832	False	False	False	False	False	False	False	False	False	False
9833	False	False	False	False	False	False	False	False	False	False
9834	False	False	False	False	False	False	False	False	False	False

9835 rows × 169 columns



In [18]:

```
from mlxtend.frequent_patterns import apriori

# 지지도가 5% 이상인 빈번 항목 집합 탐색
groceries_ap = apriori(groceries_tr, min_support=0.01, use_colnames=True)
groceries_ap
```

Out [18]:

	support	itemsets
0	0.033452	(UHT-milk)
1	0.017692	(baking powder)
2	0.052466	(beef)
3	0.033249	(berries)
4	0.026029	(beverages)
...	...	...
328	0.011998	(tropical fruit, root vegetables, whole milk)
329	0.014540	(root vegetables, yogurt, whole milk)
330	0.010473	(soda, yogurt, whole milk)
331	0.015150	(tropical fruit, yogurt, whole milk)
332	0.010880	(whipped/sour cream, yogurt, whole milk)

333 rows × 2 columns

8.3.0.0.2 mlxtend의 association_rules 함수

'score(점수)', 'confidence(신뢰도)' 및 'Lift(상승도)'를 포함하는 연관 규칙의 DataFrame 생성

```
from mlxtend.frequent_patterns import association_rules
association_rules(df, metric='confidence', min_threshold=0.8, support_only=False)
```

- df : ['support', 'itemsets'] 열이 있는 빈번 항목 집합의 DataFrame
- metric : {'support', 'confidence', 'lift'}
support_only = True 인 경우 자동으로 'support'로 설정됨
- min_threshold : metric에 지정된 rule의 최소값 (default: 0.8)
- support_only : support만 계산하고 다른 메트릭 열을 NaN으로 출력

In [19]:

```
from mlxtend.frequent_patterns import association_rules
# 신뢰도가 0.3 이상인 빈번 항목 집합 탐색
association_rules(groceries_ap, metric="confidence", min_threshold=0.3)
```

Out[19]:

	antecedents	consequents	antecedent support	consequent support	support	confidence	lift	level
0	(beef)	(other vegetables)	0.052466	0.193493	0.019725	0.375969	1.943066	0.001
1	(beef)	(root vegetables)	0.052466	0.108998	0.017387	0.331395	3.040367	0.01
2	(beef)	(whole milk)	0.052466	0.255516	0.021251	0.405039	1.585180	0.001
3	(berries)	(other vegetables)	0.033249	0.193493	0.010269	0.308869	1.596280	0.001
4	(berries)	(whole milk)	0.033249	0.255516	0.011795	0.354740	1.388328	0.001
...	...	...	...	...	...	...	...	...
120	(soda, yogurt)	(whole milk)	0.027351	0.255516	0.010473	0.382900	1.498535	0.001
121	(tropical fruit, yogurt)	(whole milk)	0.029283	0.255516	0.015150	0.517361	2.024770	0.001
122	(tropical fruit, whole milk)	(yogurt)	0.042298	0.139502	0.015150	0.358173	2.567516	0.001
123	(whipped/sour cream, yogurt)	(whole milk)	0.020742	0.255516	0.010880	0.524510	2.052747	0.001
124	(whipped/sour cream, whole milk)	(yogurt)	0.032232	0.139502	0.010880	0.337539	2.419607	0.001

125 rows × 9 columns



[예시] 여러 개의 기준을 충족하는 빈번 항목 집합 탐색

In [20]:

```
rules = association_rules(groceries_ap, metric="lift", min_threshold=3)
rules["antecedent_len"] = rules["antecedents"].apply(lambda x: len(x))
rules
```

Out[20]:

	antecedents	consequents	antecedent support	consequent support	support	confidence	lift	lever:
0	(root vegetables)	(beef)	0.108998	0.052466	0.017387	0.159515	3.040367	0.011
1	(beef)	(root vegetables)	0.052466	0.108998	0.017387	0.331395	3.040367	0.011
2	(citrus fruit, root vegetables)	(other vegetables)	0.017692	0.193493	0.010371	0.586207	3.029608	0.006
3	(citrus fruit, other vegetables)	(root vegetables)	0.028876	0.108998	0.010371	0.359155	3.295045	0.007
4	(root vegetables)	(citrus fruit, other vegetables)	0.108998	0.028876	0.010371	0.095149	3.295045	0.007
5	(other vegetables)	(citrus fruit, root vegetables)	0.193493	0.017692	0.010371	0.053600	3.029608	0.006
6	(yogurt, whole milk)	(curd)	0.056024	0.053279	0.010066	0.179673	3.372304	0.007
7	(curd)	(yogurt, whole milk)	0.053279	0.056024	0.010066	0.188931	3.372304	0.007
8	(tropical fruit, root vegetables)	(other vegetables)	0.021047	0.193493	0.012303	0.584541	3.020999	0.008
9	(tropical fruit, other vegetables)	(root vegetables)	0.035892	0.108998	0.012303	0.342776	3.144780	0.008
10	(root vegetables)	(tropical fruit, other vegetables)	0.108998	0.035892	0.012303	0.112873	3.144780	0.008
11	(other vegetables)	(tropical fruit, root vegetables)	0.193493	0.021047	0.012303	0.063584	3.020999	0.008
12	(yogurt, other vegetables)	(whipped/sour cream)	0.043416	0.071683	0.010168	0.234192	3.267062	0.007
13	(whipped/sour cream)	(yogurt, other vegetables)	0.071683	0.043416	0.010168	0.141844	3.267062	0.007

In [21]:

```
## 규칙의 길이가 2 이상, 신뢰도가 0.4이상, 신뢰도가 1이상인 규칙만 추출
```

```
rules[ (rules['antecedent_len'] >= 2) &  
       (rules['confidence'] >= 0.4) &  
       (rules['lift'] >= 1) ]
```

Out[21]:

	antecedents	consequents	antecedent support	consequent support	support	confidence	lift	leverag
2	(citrus fruit, root vegetables)	(other vegetables)	0.017692	0.193493	0.010371	0.586207	3.029608	0.00694
8	(tropical fruit, root vegetables)	(other vegetables)	0.021047	0.193493	0.012303	0.584541	3.020999	0.00823

9. 시계열분석

9.1 시계열 분석 개념

시계열 분석 개념

일정한 시간 간격으로 표시된 자료의 특성을 파악하여 미래를 예측하는 분석 방법

추세 변동	계절 변동	순환 변동	불규칙 변동
추세변동이란 X(시간)에 따라 y(종속변수) 값이 달라진다는 것	계절변동은 특정 주기별로 같은 패턴을 보이 는 파동의 형태	추세 변동 + 계절 변동의 개념 : 일정한 주기를 가지고 추세를 따 라감	시간과 종속변수 와의 관계로는 설 명할 수 없는 변동 Ex) 천재지변, 태 풍, 홍수 등



시계열 분해(time series decomposition)는 시계열 자료를 추세(Trend), 계절성(Seasonality), 잔차(Residual)로 분해하는 기법

9.2 시계열 분해 방법

- (1) 데이터를 보고 시계열의 주기적 반복/계절성이 있는지, 가법 모형(additive model, $y = t + s + r$)과 승법 모형(multiplicative model, $y = t * s * r$) 중 무엇이 더 적합할지 판단을 합니다.
- (2) 시계열 자료에서 추세(trend)를 뽑아내기 위해서 중심 이동 평균(centered moving average)을 이용합니다.
- (3) 원 자료에서 추세 분해값을 빼줍니다(detrend). 그러면 계절 요인과 불규칙 요인만 남게 됩니다.
- (4) 다음에 계절 주기 (seasonal period) 로 detrend 이후 남은 값의 합을 나누어주면 계절 평균(average seasonality)을 구할 수 있습니다. (예: 01월 계절 평균 = $(2020-01 + 2021-01 + 2022-01 + 2023-01)/4$, 02월 계절 평균 = $(2020-02 + 2021-02 + 2022-02 + 2023-02)/4$).
- (5) 원래의 값에서 추세와 계절성 분해값을 빼주면 불규칙 요인(random, irregular factor)이 남게 됩니다.
- 시계열 분해 후에 추세와 계절성을 제외한 잔차는 특정 패턴 없이 무작위 분포를 띠고 작은 값이면 추세와 계절성으로 모형화가 잘 이루어진 데이터임 이는 시계열 자료의 특성을 이해하고 예측하는데 활용할 수 있습니다.
- 만약 시계열 분해 후의 잔차에 특정 패턴 (가령, 주기적인 파동을 그린다거나, 분산이 점점 커진다거나 등..)이 존재한다면 잔차에 대해서만 다른 모형을 추가로 적합할 수도 있겠습니다.

```
seasonal_decompose(ts, model='{ "additive" , "multiplicative" }, optional)
```

In [1]:

```
import pandas as pd

data = pd.read_csv('../data/arima_data.csv', names = ['day', 'price'])
data.head(3)
```

Out[1]:

	day	price
0	2013-01-01	3794
1	2013-02-01	3863
2	2013-03-01	5190

In [2]:

```
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 60 entries, 0 to 59
Data columns (total 2 columns):
#   Column  Non-Null Count  Dtype
---  ------  -
0   day      60 non-null      object
1   price    60 non-null      int64
dtypes: int64(1), object(1)
memory usage: 1.1+ KB
```

In [3]:

```
data['day'] = pd.to_datetime(data['day'], format="%Y-%m-%d")
data.set_index('day', inplace=True)
data.head(3)
```

Out[3]:

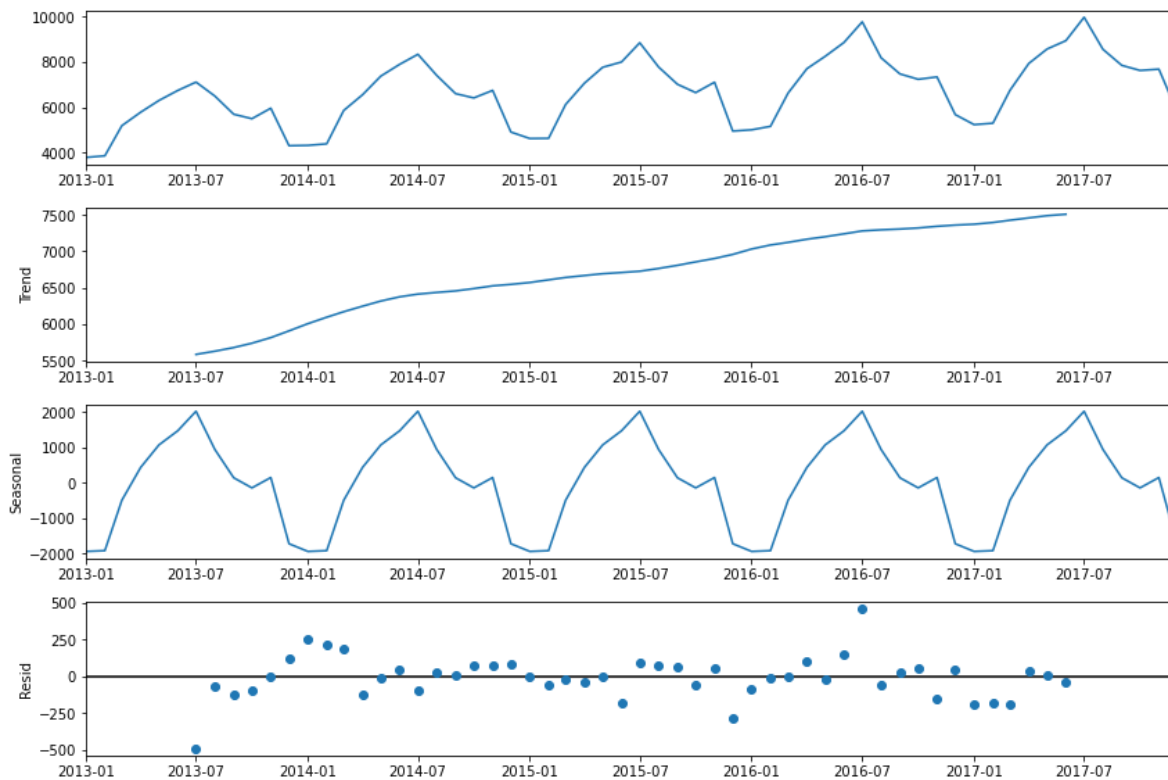
	price
day	
2013-01-01	3794
2013-02-01	3863
2013-03-01	5190

In [4]:

```
from statsmodels.tsa.seasonal import seasonal_decompose
import matplotlib.pyplot as plt
```

```
ts = data
result = seasonal_decompose(ts, model='additive')
```

```
plt.rcParams['figure.figsize'] = [12, 8]
result.plot()
plt.show()
```



9.3 정상성 검정 및 확인

9.3.1 정상성이란?

- 정상성은 추세와 계절성을 갖지 않는 진동하는 데이터이다.
- 주기성 행동을 가지고 있지만 주기가 특정 패턴을 갖지 않고 시계열을 관측하는 어떠한 시점에서도 똑같은 모양일 것이다..
- 그러므로 추세나 계절성이 없는 데이터 == > ARIMA 모델을 적용하기 위한 조건
- 정상성을 띄지 않는다면 정상성(stationary)을 갖춘 형태로 변경해줘야 한다.

9.3.2 정상성 검정

Augmented Dickey-Fuller Test

- 정상성을 알아보기 위한 단위근 검정 방법
- 귀무가설 : 데이터가 정상성(stationary)을 갖지 않는다. (변동이 있다.)
- 대립가설 : 데이터가 정상성(stationary)을 갖는다

```
adfuller(x, maxlag, regression, autolag)
```

- x: 시계열 자료
- regression: 검정에 사용할 모형
{'nc': 추세 없음, 'c':상수항만 사용, 'ct':상수항&1차 추세, 'ctt':상수항&2차 추세}
- autolag: 검정 AR 모형의 차수를 자동 결정하는 알고리즘
{'AIC', 'BIC', 't-stat', None}

return 값 :

- 0. 검정통계량
- 1. p-value
- 2. 사용된 시차의 수
- 3. 분석에 사용한 자료 수
- 4. 1%, 5%, 10% 수준에 해당되는 검정통계량 값

In [1]:

```
import pandas as pd

data = pd.read_csv('../data/arima_data.csv', names = ['day', 'price'])
data.head(3)
```

Out[1]:

	day	price
0	2013-01-01	3794
1	2013-02-01	3863
2	2013-03-01	5190

In [2]:

```
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 60 entries, 0 to 59
Data columns (total 2 columns):
 #   Column  Non-Null Count  Dtype
---  --
 0   day     60 non-null     object
 1   price   60 non-null     int64
dtypes: int64(1), object(1)
memory usage: 1.1+ KB
```

In [3]:

```
data['day'] = pd.to_datetime(data['day'], format="%Y-%m-%d")
data.set_index('day', inplace=True)
data.head(3)
```

Out[3]:

	price
day	
2013-01-01	3794
2013-02-01	3863
2013-03-01	5190

In [4]:

```
train_len = int(len(data) * 0.8)
training = data[:train_len]
test = data.drop(training.index)
```

In [5]:

```
## 귀무가설 : 데이터가 정상성(stationary)을 갖지 않는다. (변동이 있다.)
## 대립가설 : 데이터가 정상성(stationary)을 갖는다
```

```
from statsmodels.tsa.stattools import adfuller
```

```
adf = adfuller(training, regression='ct')
```

```
print('ADF Statistic: {}'.format(adf[0]))
print('p-value : {}'.format(adf[1]))
```

```
ADF Statistic: -1.9997199341327796
p-value : 0.6015863303794179
```

단위근 검정에서 p-value가 0.05보다 크므로 정상성을 갖지 않는 것으로 판단됨

차분 후 단위근 검정 재 실시

차분 : 비정상성을 정상성으로 만들기 위해, 관측값들의 차이를 계산하는 것

9.3.3 시계열 데이터 정상성 만족시키기

데이터 프레임을 n차 차분하고 acf, pacf를 다시 구함

```
DF.diff(periods=차분횟수).iloc[1:]
```

alpha : 신뢰구간 (파란색 구간의 폭으로 나타남)

lags : 그래프의 X축 범위

In [6]:

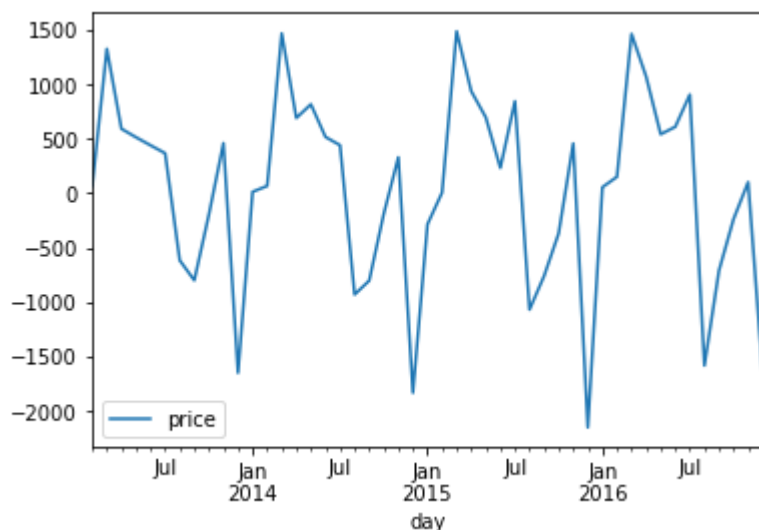
```
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf

diff_data = training.diff(1)
diff_data = diff_data.dropna()

diff_data.plot()
```

Out[6]:

<AxesSubplot: xlabel='day'>



In [7]:

```
from statsmodels.tsa.stattools import adfuller

adf = adfuller(diff_data)

print('ADF Statistic: {}'.format(adf[0]))
print('p-value : {}'.format(adf[1]))
```

ADF Statistic: -12.094547576926441

p-value : 2.085160639960858e-22

9.3.4 [Plot] ARIMA(p,d,q)의 파라미터 정하기

ACF plot와 PACF plot을 통해 AR 및 MA의 모수를 추정함

- ACF: Lag에 따른 관측치들 사이의 관련성을 측정하는 함수

- PACF: k 이외의 모든 다른 시점 관측치의 영향력을 배제한 후의 두 관측치의 관련성을 측정하는 함수

요약 : AR/MA 모형과 ACF/PACF 관계

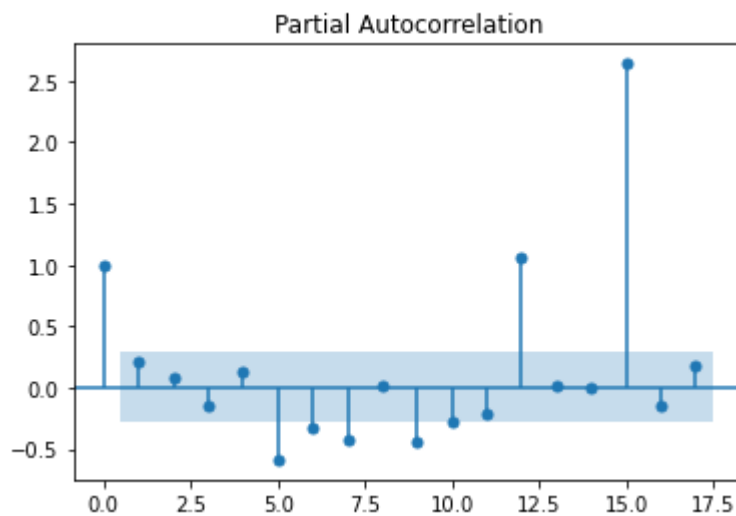
	AR(p)	MA(q)
ACF	점차적으로 감소	시차 q 이후에 0
PACF	시차 p 이후에 0	점차적으로 감소

In [8]:

```
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf
import numpy as np
import matplotlib.pyplot as plt

plot_pacf(diff_data) #AR(p)의 값 확인 가능 ==2
plot_acf(diff_data) #MA(q)의 값 확인 가능 == 2
plt.show()
```

C:\Users\Wj\Wanaconda3\Wenvs\WADP_Class\lib\site-packages\statsmodels\regression
linear_model.py:1406: RuntimeWarning: invalid value encountered in sqrt
return rho, np.sqrt(sigmasq)



acf 와 pacf 모두 2에서 절단점을 가짐

9.4 Arima 분석

ARIMA : Autoregressive integrated Moving Average

과거의 데이터를 사용하는 것을 넘어 과거의 데이터가 지니고 있던 추세까지 반영함

AR(자기상관) : 이전의 값이 이후의 값에 영향을 미치고 있는 상황

MA(이동평균) : 랜덤 변수의 평균값이 지속적으로 증가하거나 감소하는 추세

차분 : 비정상성을 정상성으로 만들기 위해, 관측값들의 차이를 계산

In [1]:

```
import pandas as pd
import matplotlib.pyplot as plt

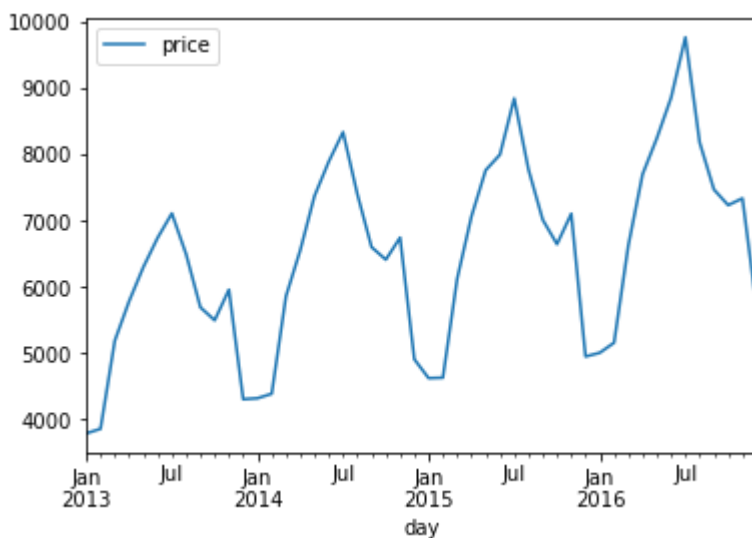
data = pd.read_csv('../data/arima_data.csv', names = ['day', 'price'])
data['day'] = pd.to_datetime(data['day'], format="%Y-%m-%d")
data.set_index('day', inplace=True)

#Train, Test 데이터 구분
train_len = int(len(data) * 0.8)
training = data[:train_len]
test = data.drop(training.index)
```

9.4.1 [Plot] 시계열 데이터 확인하기

In [2]:

```
training.plot()
plt.show()
```



In [3]:

```
from statsmodels.tsa.stattools import adfuller

adf = adfuller(data, regression='ct')

print('ADF Statistic: {}'.format(adf[0]))
print('p-value : {}'.format(adf[1]))
```

ADF Statistic: -1.6180598959846377
p-value : 0.7852805486100636

In [4]:

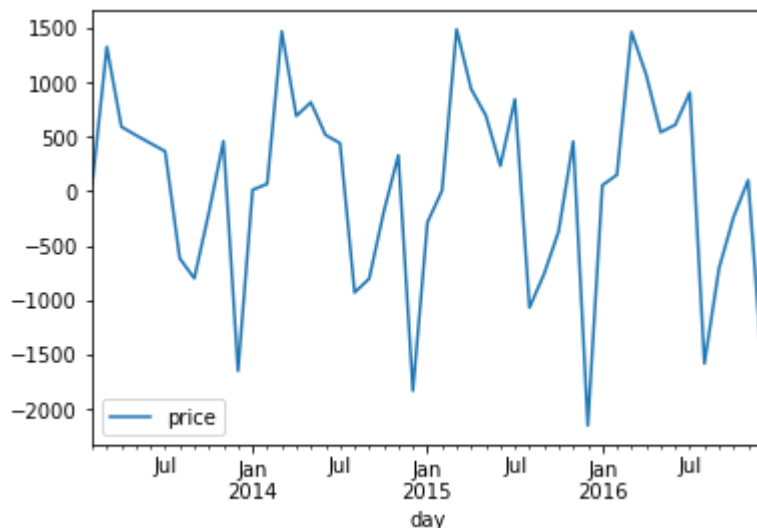
```
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf

diff_data = training.diff(1)
diff_data = diff_data.dropna()

diff_data.plot()
```

Out[4]:

<AxesSubplot: xlabel='day'>



In [5]:

```
from statsmodels.tsa.stattools import adfuller

adf = adfuller(diff_data, regression='ct')

print('ADF Statistic: {}'.format(adf[0]))
print('p-value : {}'.format(adf[1]))
```

ADF Statistic: -13.318152920970459
p-value : 5.556621043495163e-21

9.4.2 [Plot] ARIMA(p,d,q)의 파라미터 정하기

ACF plot와 PACF plot을 통해 AR 및 MA의 모수를 추정함

- ACF: Lag에 따른 관측치들 사이의 관련성을 측정하는 함수

- PACF: k 이외의 모든 다른 시점 관측치의 영향력을 배제한 후의 두 관측치의 관련성을 측정하는 함수

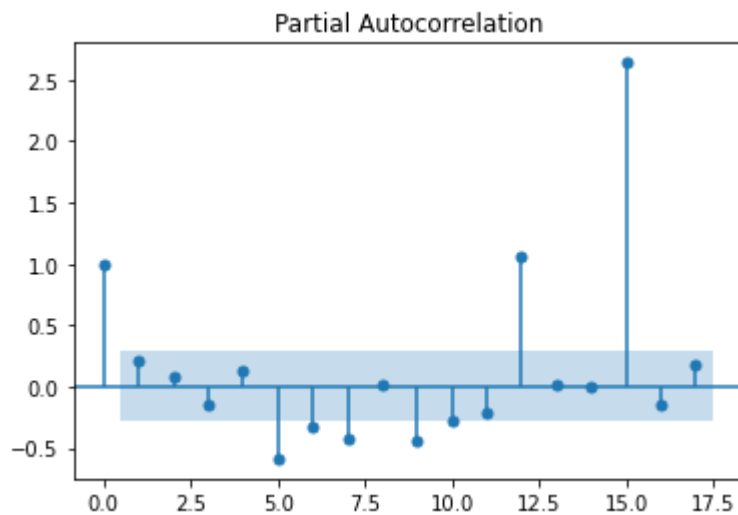
요약 : AR/MA 모형과 ACF/PACF 관계

	AR(p)	MA(q)
ACF	점차적으로 감소	시차 q 이후에 0
PACF	시차 p 이후에 0	점차적으로 감소

In [6]:

```
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf
plot_pacf(diff_data) ## AR (p) 값 확인 가능
plot_acf(diff_data) ## MA (q) 값 확인 가능
plt.show()
```

C:\Users\Wjjang\Anaconda3\envs\WADP_Class\lib\site-packages\statsmodels\regression
linear_model.py:1406: RuntimeWarning: invalid value encountered in sqrt
return rho, np.sqrt(sigmasq)



In [7]:

```
len(diff_data)
diff_data
```

Out[7]:

	price
day	
2013-02-01	69.0
2013-03-01	1327.0
2013-04-01	593.0
2013-05-01	515.0
2013-06-01	441.0
2013-07-01	368.0
2013-08-01	-615.0
2013-09-01	-799.0
2013-10-01	-196.0
2013-11-01	461.0
2013-12-01	-1649.0
2014-01-01	14.0
2014-02-01	68.0
2014-03-01	1469.0
2014-04-01	692.0
2014-05-01	818.0
2014-06-01	518.0
2014-07-01	442.0
2014-08-01	-929.0
2014-09-01	-802.0
2014-10-01	-188.0
2014-11-01	331.0
2014-12-01	-1832.0
2015-01-01	-284.0
2015-02-01	7.0
2015-03-01	1488.0
2015-04-01	939.0
2015-05-01	698.0
2015-06-01	234.0
2015-07-01	846.0
2015-08-01	-1068.0
2015-09-01	-759.0
2015-10-01	-367.0

	price
day	
2015-11-01	458.0
2015-12-01	-2150.0
2016-01-01	56.0
2016-02-01	152.0
2016-03-01	1464.0
2016-04-01	1073.0
2016-05-01	543.0
2016-06-01	611.0
2016-07-01	907.0
2016-08-01	-1582.0
2016-09-01	-709.0
2016-10-01	-236.0
2016-11-01	103.0
2016-12-01	-1656.0

In [8]:

```
diff_data.iloc[2:47]- diff_data.iloc[0:47-2].values
```

Out[8]:

	price
day	
2013-04-01	524.0
2013-05-01	-812.0
2013-06-01	-152.0
2013-07-01	-147.0
2013-08-01	-1056.0
2013-09-01	-1167.0
2013-10-01	419.0
2013-11-01	1260.0
2013-12-01	-1453.0
2014-01-01	-447.0
2014-02-01	1717.0
2014-03-01	1455.0
2014-04-01	624.0
2014-05-01	-651.0
2014-06-01	-174.0
2014-07-01	-376.0
2014-08-01	-1447.0
2014-09-01	-1244.0
2014-10-01	741.0
2014-11-01	1133.0
2014-12-01	-1644.0
2015-01-01	-615.0
2015-02-01	1839.0
2015-03-01	1772.0
2015-04-01	932.0
2015-05-01	-790.0
2015-06-01	-705.0
2015-07-01	148.0
2015-08-01	-1302.0
2015-09-01	-1605.0
2015-10-01	701.0
2015-11-01	1217.0
2015-12-01	-1783.0

	price
day	
2016-01-01	-402.0
2016-02-01	2302.0
2016-03-01	1408.0
2016-04-01	921.0
2016-05-01	-921.0
2016-06-01	-462.0
2016-07-01	364.0
2016-08-01	-2193.0
2016-09-01	-1616.0
2016-10-01	1346.0
2016-11-01	812.0
2016-12-01	-1420.0

In [9]:

```
import statsmodels.api as sm
sm.tsa.stattools.acf(diff_data, nlags=10, fft=False)
```

Out[9]:

```
array([ 1.          ,  0.21314393,  0.11138369, -0.10128232,  0.07394015,
        -0.46120534, -0.38712099, -0.41183432,  0.04214028, -0.20715218,
         0.0799447  ])
```

Type *Markdown* and LaTeX: α^2

9.4.3 모형 만들기 (ARIMA)

ARIMA(...)

- endog (array_like, optional) : 시계열 y 값
- exog (array_like, optional) : 시계열 이외의 다른 변수 X들
- order ((p,d,q), optional) : 각각 AR, diff, and MA 값 (acf, pacf 그래프 확인 후 설정)
- dates (array_like, optional) : datetime 객체의 배열과 유사한 객체로,
pandas 객체가 endog 또는 exog에 제공되면 DateIndex가 있는 것으로 간주
- freq (('B', 'D', 'W', 'M', 'A', 'Q'), optional) : 시계열의 빈도. 날짜가 제공되는 경우 선택 사항

ARIMA().fit(...)

Kalman 필터를 통해 정확한 최대 가능성으로 ARIMA (p, d, q) 모델을 맞춤

- transparams (bool, optional) : 정상성 보장을 위해 매개변수를 변환할지 여부
- method ({'css-mle', 'mle', 'css'}, optional) : 최대화할 로그 가능성으로 start_params 를 시작 매개 변수로 사용

- `trend ({'c','nc'}, optional)` : 상수 포함 여부로 'c'는 상수를 포함하고 'nc'는 상수가 없음

In [10]:

```
training.price.values
```

Out[10]:

```
array([3794, 3863, 5190, 5783, 6298, 6739, 7107, 6492, 5693, 5497, 5958,  
       4309, 4323, 4391, 5860, 6552, 7370, 7888, 8330, 7401, 6599, 6411,  
       6742, 4910, 4626, 4633, 6121, 7060, 7758, 7992, 8838, 7770, 7011,  
       6644, 7102, 4952, 5008, 5160, 6624, 7697, 8240, 8851, 9758, 8176,  
       7467, 7231, 7334, 5678], dtype=int64)
```

In [11]:

```
from statsmodels.tsa.arima.model import ARIMA
import statsmodels.api as sm

model = ARIMA(training.price.values, order=(2,1,2),trend='ct')

res = model.fit()

res.summary()
```

C:\Users\Wjjang\Anaconda3\envs\WADP_Class\lib\site-packages\statsmodels\tsa\statespace\Wsarimax.py:963: UserWarning: Non-stationary starting autoregressive parameters found. Using zeros as starting parameters.
warn('Non-stationary starting autoregressive parameters')
C:\Users\Wjjang\Anaconda3\envs\WADP_Class\lib\site-packages\statsmodels\tsa\statespace\Wsarimax.py:975: UserWarning: Non-invertible starting MA parameters found. Using zeros as starting parameters.
warn('Non-invertible starting MA parameters found.')
C:\Users\Wjjang\Anaconda3\envs\WADP_Class\lib\site-packages\statsmodels\base\model.py:568: ConvergenceWarning: Maximum Likelihood optimization failed to converge. Check mle_retvals
"Check mle_retvals", ConvergenceWarning)

Out[11]:

SARIMAX Results

Dep. Variable:	y	No. Observations:	48			
Model:	ARIMA(2, 1, 2)	Log Likelihood	-375.863			
Date:	Sun, 06 Feb 2022	AIC	765.725			
Time:	22:21:30	BIC	778.676			
Sample:	0	HQIC	770.599			
	- 48					
Covariance Type:	opg					
	coef	std err	z	P> z	[0.025	0.975]
const	1.0181	4212.891	0.000	1.000	-8256.096	8258.132
x1	36.9581	173.295	0.213	0.831	-302.694	376.610
ar.L1	-1.3081	0.232	-5.640	0.000	-1.763	-0.854
ar.L2	-0.3106	0.232	-1.336	0.181	-0.766	0.145
ma.L1	1.9535	0.265	7.370	0.000	1.434	2.473
ma.L2	0.9787	0.264	3.714	0.000	0.462	1.495
sigma2	4.472e+05	2.373	1.88e+05	0.000	4.47e+05	4.47e+05
Ljung-Box (Q):	102.12	Jarque-Bera (JB):	0.35			
Prob(Q):	0.00	Prob(JB):	0.84			
Heteroskedasticity (H):	1.52	Skew:	-0.20			
Prob(H) (two-sided):	0.41	Kurtosis:	2.88			

Warnings:

- [1] Covariance matrix calculated using the outer product of gradients (complex-step).
- [2] Covariance matrix is singular or near-singular, with condition number 4.08e+22.

In [12]:

```
sm.__version__
```

Out[12]:

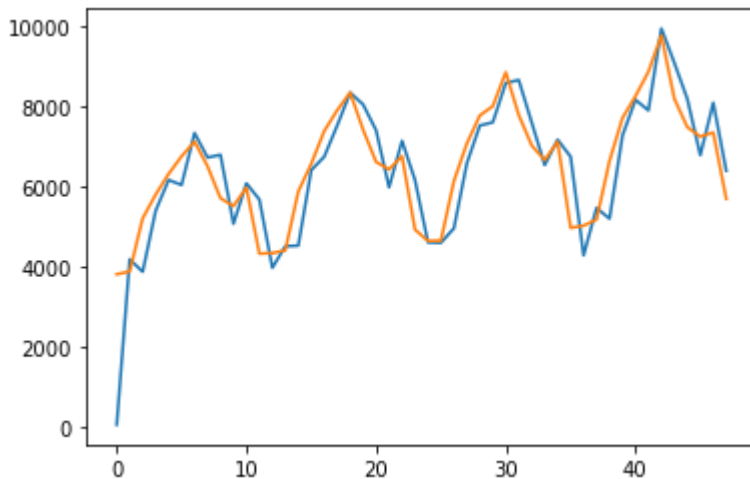
```
'0.11.1'
```

In [13]:

```
plt.plot(res.predict())  
plt.plot(training.price.values)
```

Out[13]:

```
[<matplotlib.lines.Line2D at 0x1990eec8e88>]
```



In [14]:

```
forecast_data = res.forecast(steps=len(test), alpha=0.1) # 학습 데이터셋으로부터 test 데이터 길이  
만큼 예측합니다.
```

```
pred_y= forecast_data  
pred_y
```

Out[14]:

```
array([4073.72697431, 3793.73618792, 4204.07130546, 3851.04485484,  
       4282.18632089, 3924.6264941 , 4355.23599682, 3999.78005817,  
       4427.80262684, 4075.07728735, 4500.33135269, 4150.37947941])
```

In [15]:

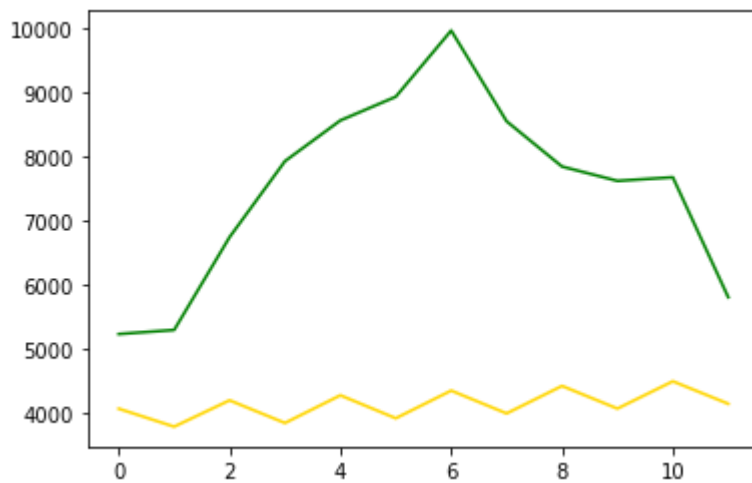
```
test_y = test.values # 실제 데이터  
test_y
```

Out[15]:

```
array([[5236],  
       [5299],  
       [6744],  
       [7927],  
       [8561],  
       [8930],  
       [9960],  
       [8548],  
       [7843],  
       [7620],  
       [7676],  
       [5809]], dtype=int64)
```

In [16]:

```
plt.plot(pred_y, color="gold") # 모델이 예상한 가격 그래프  
plt.plot(test_y, color="green") # 실제 가격 그래프  
  
plt.show()
```



In [17]:

```
from sklearn.metrics import r2_score  
r2_score(pred_y, test_y)
```

Out[17]:

```
-282.6298918669729
```

In [18]:

```
regid = pred_y - test_y
```

In [19]:

```
import numpy as np  
np.sqrt(np.sum(regid**2)/len(test_y))
```

Out[19]:

12704.92600132563

9.5 계절성 시계열 데이터에 대한 분석 : SARIMA

SARIMA : 데이터가 지닌 계절성(주기에 따라 유사한 양상으로 변화하는 모양)까지 고려한 ARIMA 모델

9.5.1 데이터 로드

In [1]:

```
import pandas as pd
import matplotlib.pyplot as plt

data = pd.read_csv('../data/arima_data.csv', names = ['day', 'price'])
data['day'] = pd.to_datetime(data['day'], format="%Y-%m-%d")
data.set_index('day', inplace=True)
s_data = data
```

In [2]:

s_data

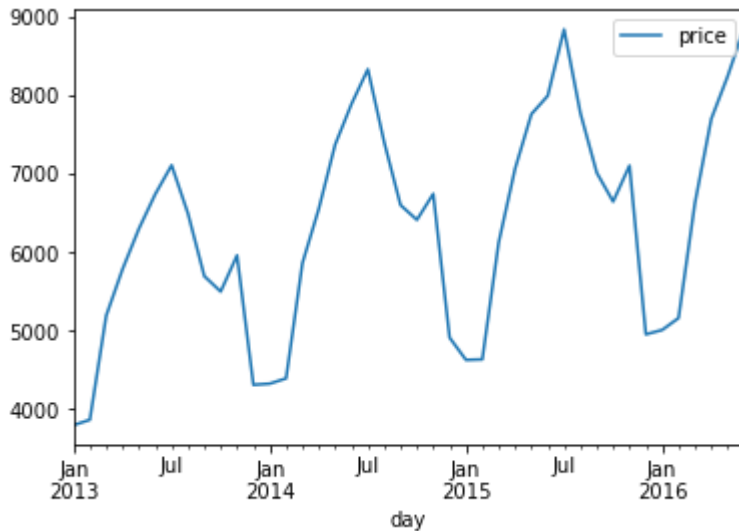
Out[2]:

	price
day	
2013-01-01	3794
2013-02-01	3863
2013-03-01	5190
2013-04-01	5783
2013-05-01	6298
2013-06-01	6739
2013-07-01	7107
2013-08-01	6492
2013-09-01	5693

In [3]:

```
train_len = int(len(s_data) * 0.7)
training = s_data[:train_len]
test = s_data.drop(training.index)
```

```
import matplotlib.pyplot as plt
training.plot()
plt.show()
```



```
SARIMAX(endog, order=(1,1,1), seasonal_order=(1,1,1,12),
        enforce_stationarity=False, enforce_invertibility=False)
```

- endog : 시계열 y값
- order ((p,d,q), optional) : 각각 AR, diff, and MA 값
- seasonal_order ((P,D,Q,s), optional) : s는 계절의 기간 수를 제공하는 정수
- trend (str{'n','c','t','ct'}, optional) : 'n'은 없음, 'c'는 상수, 't'는 선형 추세, 'ct'는 둘 다

In [4]:

```
import statsmodels.api as sm
from statsmodels.tsa.statespace.sarimax import SARIMAX

model = SARIMAX(training.price.values,
                 order=(2,1,2), seasonal_order=(1,1,1,12),
                 trend='ct',
                 enforce_stationarity=False,
                 enforce_invertibility=False)

results = model.fit()
results.summary()
```

C:\Users\Wjjang\Anaconda3\envs\WADP_Class\lib\site-packages\statsmodels\tsa\statespace\Wsarimax.py:868: UserWarning: Too few observations to estimate starting parameters for seasonal ARMA. All parameters except for variances will be set to zeros.
' zeros.' % warning_description)
C:\Users\Wjjang\Anaconda3\envs\WADP_Class\lib\site-packages\statsmodels\base\model.py:568: ConvergenceWarning: Maximum Likelihood optimization failed to converge. Check mle_retvals
"Check mle_retvals", ConvergenceWarning)

Out[4]:

SARIMAX Results

Dep. Variable:	y				No. Observations:		42
Model:	SARIMAX(2, 1, 2)x(1, 1, [1], 12)				Log Likelihood		-90.141
Date:	Sun, 06 Feb 2022				AIC		198.281
Time:	22:22:29				BIC		204.033
Sample:	0				HQIC		197.749
	- 42						
Covariance Type:	opg						
	coef	std err	z	P> z	[0.025	0.975]	
intercept	-266.3161	2243.254	-0.119	0.905	-4663.012	4130.380	
drift	8.4685	52.534	0.161	0.872	-94.497	111.434	
ar.L1	-0.8374	1.085	-0.772	0.440	-2.963	1.289	
ar.L2	0.0155	1.292	0.012	0.990	-2.517	2.548	
ma.L1	-0.0080	10.396	-0.001	0.999	-20.383	20.367	
ma.L2	-0.9240	9.773	-0.095	0.925	-20.079	18.230	
ar.S.L12	0.0233	5.074	0.005	0.996	-9.922	9.969	
ma.S.L12	-0.2263	7.163	-0.032	0.975	-14.266	13.813	
sigma2	2.922e+04	2.56e+05	0.114	0.909	-4.73e+05	5.31e+05	
Ljung-Box (Q):	11.04	Jarque-Bera (JB):	1.16				
Prob(Q):	0.61	Prob(JB):	0.56				
Heteroskedasticity (H):	0.90	Skew:	-0.62				
Prob(H) (two-sided):	0.92	Kurtosis:	3.66				

Warnings:

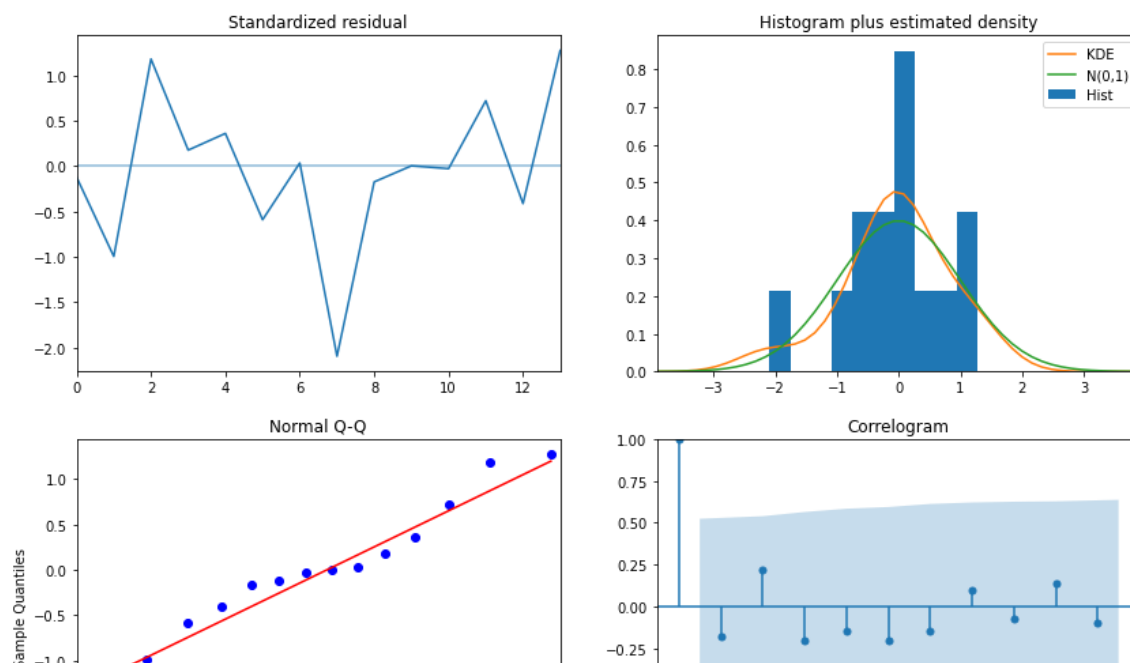
[1] Covariance matrix calculated using the outer product of gradients (complex-step).

9.5.2 데이터 확인

1. 시간 경과에 따른 표준화 잔차
2. 히스토그램과 표준화 된 잔차의 추정 된 밀도, 참조를 위해 그려진 Normal (0,1) 밀도.
3. 일반 기준선이있는 일반 QQ 플롯.
4. 상관도

In [5]:

```
results.plot_diagnostics(figsize=(14, 10))  
plt.show()
```



9.5.3 예측값, mse, 신뢰구간의 최소&최댓값

In [6]:

```
forecast_values = results.get_forecast(steps=len(test))  
forecast_values.summary_frame()
```

Out[6]:

y	mean	mean_se	mean_ci_lower	mean_ci_upper
0	9483.491029	173.846798	9142.757566	9824.224492
1	8463.395254	175.597424	8119.230628	8807.559880
2	7791.121131	175.576941	7446.996650	8135.245611
3	7493.318393	176.426971	7147.527883	7839.108902
4	8028.405739	176.372152	7682.722673	8374.088806
5	6004.383133	176.973328	5657.521784	6351.244481
6	6105.994735	176.925506	5759.227114	6452.762355
7	6307.696500	177.433616	5959.933003	6655.459998
8	7885.017608	177.413575	7537.293390	8232.741826
9	9003.677387	177.727231	8655.338415	9352.016359

9.5.4 예측값을 그리는 함수

In [7]:

```
forecast_values
```

Out[7]:

<statsmodels.tsa.statespace.mlemodel.PredictionResultsWrapper at 0x16ec65345c8>

In [8]:

```
forecast_values.conf_int()
```

Out[8]:

```
array([[ 9142.75756613,  9824.22449226],
       [ 8119.23062751,  8807.55988012],
       [ 7446.99665049,  8135.24561055],
       [ 7147.52788314,  7839.10890206],
       [ 7682.72267264,  8374.08880571],
       [ 5657.5217843 ,  6351.24448149],
       [ 5759.22711443,  6452.76235503],
       [ 5959.93300268,  6655.45999768],
       [ 7537.29339045,  8232.74182586],
       [ 8655.33841505,  9352.01635875],
       [ 9350.27399493, 10046.97419375],
       [ 9997.51891464, 10695.21143821],
       [10678.54945659, 11580.5427925 ],
       [ 9770.41092117, 10679.49338891],
       [ 9214.93384369, 10123.54567564],
       [ 9038.91075309,  9951.66820828],
       [ 9700.17124883, 10612.70231871],
       [ 7808.64148934,  8724.36997564]])
```

In [9]:

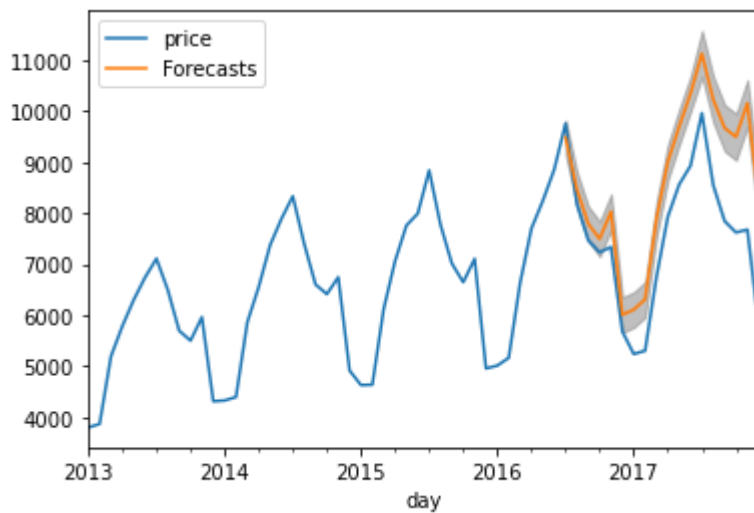
```
#pred_ci = forecast_values.conf_int()
ax = s_data.plot() ## 실제 값

pred_ci = pd.DataFrame(forecast_values.conf_int())
pred_ci.index = list(test.index)

ax.fill_between(pred_ci.index,
               pred_ci.iloc[:, 0],
               pred_ci.iloc[:, 1], color='gray', alpha=0.5)

predicted = pd.DataFrame(forecast_values.predicted_mean)
predicted.index = list(test.index)
predicted.columns = ['Forecasts']
predicted.plot(ax=ax, label="Forecasts")

plt.legend()
plt.show()
```



In [10]:

```
from sklearn.metrics import r2_score
r2_score(predicted, test)
```

Out[10]:

0.2006815424790297

9.6 auto arima

자동으로 sarima 분석을 수행할 수 있다.

In [1]:

```
import pandas as pd
import matplotlib.pyplot as plt

data = pd.read_csv('../data/arima_data.csv', names = ['day', 'price'])
data['day'] = pd.to_datetime(data['day'], format="%Y-%m-%d")
data.set_index('day', inplace=True)
s_data = data
```

In [2]:

```
#!pip install pmdarima
#!conda install pmdarima -y
```

In [3]:

```
train_len = int(len(s_data)*0.8)
training = s_data[:train_len]
test = s_data.drop(training.index)
```

In [4]:

```
from pmdarima import auto_arima

auto_model = auto_arima(training, start_P=1, D=1, Q=1,
                        max_P=5, max_D=5, max_Q=5, m=12,
                        seasonal=True, information_criterion='aic',
                        trace=True, stepwise=True)
```

Performing stepwise search to minimize aic

ARIMA(2,0,2)(1,1,1)[12] intercept	: AIC=500.898, Time=0.48 sec
ARIMA(0,0,0)(0,1,0)[12] intercept	: AIC=513.245, Time=0.01 sec
ARIMA(1,0,0)(1,1,0)[12] intercept	: AIC=496.519, Time=0.06 sec
ARIMA(0,0,1)(0,1,1)[12] intercept	: AIC=500.500, Time=0.17 sec
ARIMA(0,0,0)(0,1,0)[12]	: AIC=569.050, Time=0.01 sec
ARIMA(1,0,0)(0,1,0)[12] intercept	: AIC=495.874, Time=0.06 sec
ARIMA(1,0,0)(0,1,1)[12] intercept	: AIC=496.035, Time=0.17 sec
ARIMA(1,0,0)(1,1,1)[12] intercept	: AIC=498.479, Time=0.07 sec
ARIMA(2,0,0)(0,1,0)[12] intercept	: AIC=497.831, Time=0.09 sec
ARIMA(1,0,1)(0,1,0)[12] intercept	: AIC=497.839, Time=0.09 sec
ARIMA(0,0,1)(0,1,0)[12] intercept	: AIC=501.105, Time=0.04 sec
ARIMA(2,0,1)(0,1,0)[12] intercept	: AIC=499.873, Time=0.07 sec
ARIMA(1,0,0)(0,1,0)[12]	: AIC=510.570, Time=0.03 sec

Best model: ARIMA(1,0,0)(0,1,0)[12] intercept

Total fit time: 1.357 seconds

In [5]:

```
auto_model.summary()
```

Out[5]:

SARIMAX Results

Dep. Variable:	y			No. Observations:	48	
Model:	SARIMAX(1, 0, 0)x(0, 1, 0, 12)			Log Likelihood	-244.937	
Date:	Sun, 06 Feb 2022			AIC	495.874	
Time:	22:22:57			BIC	500.625	
Sample:	0			HQIC	497.532	
	- 48					
Covariance Type:	opg					
	coef	std err	z	P> z	[0.025	0.975]
intercept	239.6144	71.575	3.348	0.001	99.331	379.898
ar.L1	0.5607	0.093	6.009	0.000	0.378	0.744
sigma2	4.342e+04	1.27e+04	3.411	0.001	1.85e+04	6.84e+04
Ljung-Box (Q):	27.66	Jarque-Bera (JB):	1.08			
Prob(Q):	0.81	Prob(JB):	0.58			
Heteroskedasticity (H):	1.05	Skew:	-0.09			
Prob(H) (two-sided):	0.93	Kurtosis:	2.17			

Warnings:

[1] Covariance matrix calculated using the outer product of gradients (complex-step).

각 요소의 P-value와 AIC 확인

In [6]:

```
# 학습 데이터셋으로부터 test 데이터 길이만큼 예측
prediction= pd.DataFrame(auto_model.predict(n_periods=len(test)),
                        index=test.index)
prediction
```

Out[6]:

0	
day	
2017-01-01	5654.661643
2017-02-01	5762.178918
2017-03-01	7201.238739
2017-04-01	8260.255501
2017-05-01	8795.415504
2017-06-01	9402.019844
2017-07-01	10306.555324
2017-08-01	8723.173539
2017-09-01	8013.398812

In [7]:

```
prediction.columns = ['predicted_price']
prediction
```

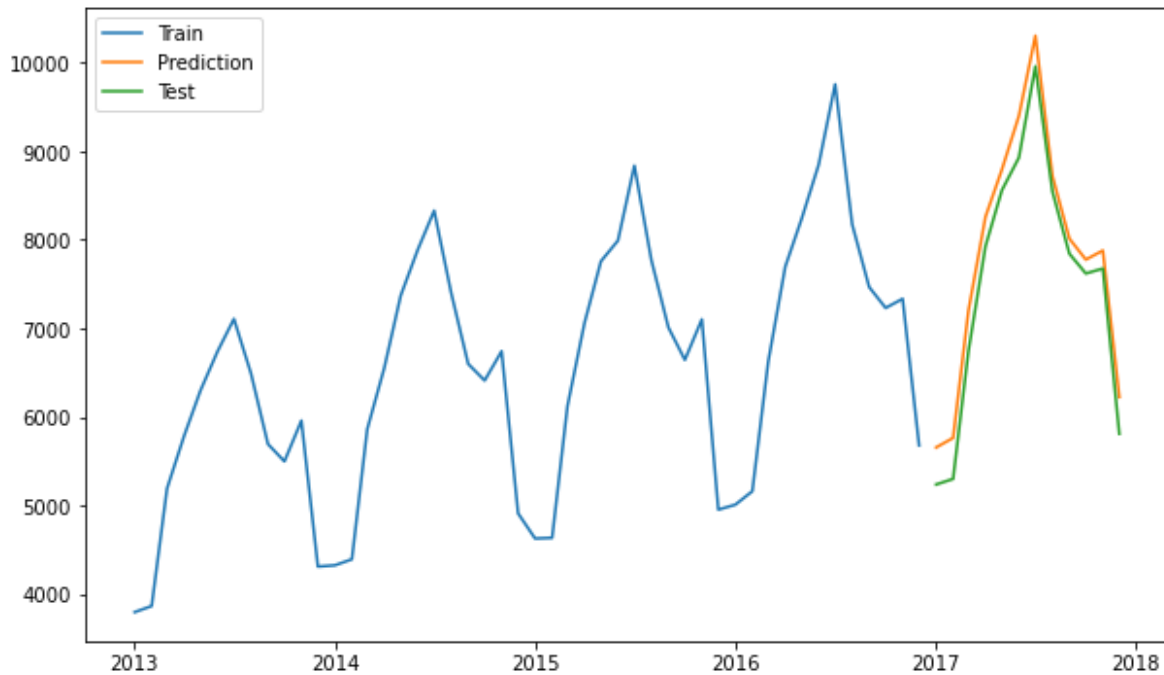
Out[7]:

predicted_price	
day	
2017-01-01	5654.661643
2017-02-01	5762.178918
2017-03-01	7201.238739
2017-04-01	8260.255501
2017-05-01	8795.415504
2017-06-01	9402.019844
2017-07-01	10306.555324
2017-08-01	8723.173539
2017-09-01	8013.398812
2017-10-01	7776.964445
2017-11-01	7879.720908
2017-12-01	6223.584364

In [8]:

```
plt.figure(figsize=(10,6))
plt.plot(training, label="Train") # Train 데이터
plt.plot(prediction, label="Prediction") # 모델이 예상한 그래프
plt.plot(test, label="Test") # 실제 가격 그래프

plt.legend(loc='upper left')
plt.show()
```



In [9]:

```
from sklearn.metrics import r2_score, mean_squared_error
test['predicted_price'] = prediction
r2_score(test['price'], test['predicted_price'])
```

Out[9]:

0.9415943476073537

In [10]:

```
import numpy as np
import math
rmse = np.sqrt(mean_squared_error(test['price'], test['predicted_price']))
rmse
```

Out[10]:

342.23949855666933

In [11]:

```
len(test['price'])
```

Out[11]:

12

9.6.1 결과 해석

- 1) 시계열 데이터 분석한 결과 1년 주기의 계절성을 가지고 있음을 확인할 수 있었다.
- 2) 또한 우상향하는 추세가 있음을 알 수 있었다.
- 3) 추세를 가지고 있으므로 1차 차분을 해줘야 한다.
- 계절성을 가지고 있으므로 예측 모형은 SARIMA를 사용하였다.
- 최적의 매개변수를 갖는 SARIMAX(1, 0, 0)x(0, 1, 0, 12) 모델의 설명력은 94.1%로 예측의 정확도가 매우 높았다.
- 예측값과 실제값의 차이는 평균적으로 342 정도 난다고 할 수 있다.

In []: